

Red Hat OpenShift Data Foundation 4.22

Managing hybrid and multicloud resources

Instructions for how to manage storage resources across a hybrid cloud or multicloud environment using the Multicloud Object Gateway (NooBaa).

Last Updated: 2026-10-12

Red Hat OpenShift Data Foundation 4.22 Managing hybrid and multicloud resources

Instructions for how to manage storage resources across a hybrid cloud or multicloud environment using the Multicloud Object Gateway (NooBaa).

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document explains how to manage storage resources across a hybrid cloud or multicloud environment.

Table of Contents

PREFACE 6

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION 7

CHAPTER 1. ABOUT THE MULTICLOUD OBJECT GATEWAY 8

CHAPTER 2. ACCESSING THE MULTICLOUD OBJECT GATEWAY WITH YOUR APPLICATIONS 9
2.1. ACCESSING THE MULTICLOUD OBJECT GATEWAY FROM THE TERMINAL 9
2.2. ACCESSING THE MULTICLOUD OBJECT GATEWAY FROM THE COMMAND-LINE INTERFACE 11
2.3. SUPPORT OF MULTICLOUD OBJECT GATEWAY DATA BUCKET APIS 13

CHAPTER 3. ADDING STORAGE RESOURCES FOR HYBRID OR MULTICLOUD 16
3.1. CREATING A NEW BACKING STORE 16
3.2. OVERRIDING THE DEFAULT BACKING STORE 17
3.3. ADDING STORAGE RESOURCES FOR HYBRID OR MULTICLOUD USING THE COMMAND-LINE INTERFACE 18
3.3.1. Creating an AWS-backed backingstore 19
3.3.2. Creating an AWS-STS-backed backingstore 20
3.3.2.1. Creating an AWS role using a script 20
3.3.3. Installing OpenShift Data Foundation operator in AWS STS OpenShift cluster 23
3.3.4. Creating a new AWS STS backingstore 23
3.3.5. Creating an IBM COS-backed backingstore 24
3.3.6. Creating an Azure-backed backingstore 26
3.3.7. Creating an Azure STS-backed backingstore 27
3.3.7.1. Creating an Azure managed identity for NooBaa 27
3.3.7.2. Installing OpenShift Data Foundation operator in Azure STS OpenShift cluster 29
3.3.7.3. Creating a new Azure STS backingstore 30
3.3.8. Creating a GCP-backed backingstore 30
3.3.9. Creating a local Persistent Volume-backed backingstore 32
3.4. CREATING AN S3 COMPATIBLE MULTICLOUD OBJECT GATEWAY BACKINGSTORE 33
3.5. CREATING A NEW BUCKET CLASS 35
3.6. EDITING A BUCKET CLASS 36
3.7. EDITING BACKING STORES FOR BUCKET CLASS 37

CHAPTER 4. CREATING AND MANAGING BUCKETS USING THE OBJECT BROWSER 38
4.1. ACCESSING THE OBJECT BROWSER 38
4.1.1. Signing in to the object browser as a developer user 38
4.2. CREATING AND MANAGING BUCKETS 39
4.2.1. Creating new buckets using the object browser 39
4.2.2. Listing bucket details using the object browser 40
4.2.3. Deleting or emptying bucket using object browser 40
4.3. MANAGING OBJECTS 40
4.3.1. Listing objects and object details using the object browser 40
4.3.2. Downloading or previewing objects using the object browser 41
4.3.3. Generating the presigned URL to share an object using the object browser 41
4.3.4. Uploading objects to the bucket using the object browser 42
4.3.5. Creating folder using the object browser 42
4.3.6. Deleting objects using the object browser 43
4.3.7. Enabling or suspending object versioning in a bucket using the object browser 43
4.3.8. Listing object versions using the object browser 44
4.4. CREATING AND MANAGING LIFECYCLE RULES 44
4.4.1. Creating new lifecycle rules using the object browser 45
4.4.2. Editing lifecycle rules using the object browser 46

4.4.3. Creating Cross Origin Resource Sharing (CORS) rule 46
4.4.4. Editing Cross Origin Resource Sharing (CORS) rule 47
4.4.5. Using bucket policy to grant public or restricted access to objects 47
4.4.6. Setting up public access limit to S3 resources using the object browser 48
4.5. CREATING AND MANAGING IAM USER 49
4.5.1. Creating an IAM user using the object browser 49
4.5.2. Generating additional access key for an IAM user 50
4.5.3. Activating/Deactivating Access key 50
4.5.4. Deleting access key for an IAM user 51
4.5.5. Deleting an IAM User 51

CHAPTER 5. MANAGING NAMESPACE BUCKETS 53
5.1. AMAZON S3 API ENDPOINTS FOR OBJECTS IN NAMESPACE BUCKETS 53
5.2. ADDING A NAMESPACE BUCKET USING THE MULTICLOUD OBJECT GATEWAY CLI AND YAML 54
5.2.1. Adding an AWS S3 namespace bucket using YAML 54
5.2.2. Adding an IBM COS namespace bucket using YAML 57
5.2.3. Adding an AWS S3 namespace bucket using the Multicloud Object Gateway CLI 60
5.2.4. Adding an IBM COS namespace bucket using the Multicloud Object Gateway CLI 62
5.3. ADDING A NAMESPACE BUCKET USING THE OPENSHIFT CONTAINER PLATFORM USER INTERFACE 63
5.4. SHARING LEGACY APPLICATION DATA WITH CLOUD NATIVE APPLICATION USING S3 PROTOCOL 65
5.4.1. Creating a NamespaceStore to use a file system 65
5.4.2. Creating accounts with NamespaceStore filesystem configuration 65
5.4.3. Accessing legacy application data from the openshift-storage namespace 67
5.4.3.1. Changing the default SELinux label on the legacy application project to match the one in the openshift-storage project 76
5.4.3.2. Modifying the SELinux label only for the deployment config that has the pod which mounts the legacy application PVC 77
5.5. CONFIGURING OR MODIFYING THE PUBLICACCESSBLOCK CONFIGURATION FOR S3 BUCKET 79

CHAPTER 6. SECURING MULTICLOUD OBJECT GATEWAY 82
6.1. CHANGING THE DEFAULT ACCOUNT CREDENTIALS TO ENSURE BETTER SECURITY IN THE MULTICLOUD OBJECT GATEWAY 82
6.2. REGENERATING THE S3 CREDENTIALS FOR THE ACCOUNTS 82
6.3. REGENERATING THE S3 CREDENTIALS FOR THE OBC 83
6.4. ENABLING SECURED MODE DEPLOYMENT FOR MULTICLOUD OBJECT GATEWAY 85
6.5. DISABLING ROUTES THAT ENABLE EXTERNAL ACCESS TO MULTICLOUD OBJECT GATEWAY 86
6.6. DISABLING HTTP ACCESS FOR NOOBAA S3 ROUTE 87
6.7. DISABLING HTTP ROUTE FOR RADOS OBJECT GATEWAY 88

CHAPTER 7. MIRRORING DATA FOR HYBRID AND MULTICLOUD BUCKETS 90
7.1. CREATING BUCKET CLASSES TO MIRROR DATA USING THE COMMAND-LINE INTERFACE 90
7.2. CREATING BUCKET CLASSES TO MIRROR DATA USING A YAML 90

CHAPTER 8. BUCKET POLICIES IN THE MULTICLOUD OBJECT GATEWAY 92
8.1. INTRODUCTION TO BUCKET POLICIES 92
8.2. USING BUCKET POLICIES IN MULTICLOUD OBJECT GATEWAY 92
8.3. CREATING A USER IN THE MULTICLOUD OBJECT GATEWAY 93

CHAPTER 9. BUCKET NOTIFICATION IN MULTICLOUD OBJECT GATEWAY 95
9.1. CONFIGURING BUCKET NOTIFICATION IN MULTICLOUD OBJECT GATEWAY 95

CHAPTER 10. MULTICLOUD OBJECT GATEWAY BUCKET REPLICATION 101
10.1. REPLICATING A BUCKET TO ANOTHER BUCKET 101
10.1.1. Replicating a bucket to another bucket using the command-line interface 101

10.1.2. Replicating a bucket to another bucket using a YAML 102
10.2. SETTING A BUCKET CLASS REPLICATION POLICY 103
10.2.1. Setting a bucket class replication policy using the command-line interface 103
10.2.2. Setting a bucket class replication policy using a YAML 104
10.3. ENABLING LOG BASED BUCKET REPLICATION 105
10.3.1. Enabling log based bucket replication for new namespace buckets using OpenShift Web Console in Amazon Web Service environment 105
10.3.2. Enabling log based bucket replication for existing namespace buckets using YAML 106
10.3.3. Enabling log based bucket replication in Microsoft Azure 106
10.3.4. Enabling log-based bucket replication deletion 108
10.4. BUCKET LOGGING FOR MULTICLOUD OBJECT GATEWAY 108
10.4.1. Enabling bucket logging for Multicloud Object Gateway using the Best-effort option 109
10.4.2. Enabling bucket logging using the Guaranteed option 110
10.5. SYNCHRONIZING VERSIONS IN MULTICLOUD OBJECT GATEWAY BUCKET REPLICATION 111
10.6. OBTAINING METRICS TO REFLECT BUCKET REPLICATION STATE 113

CHAPTER 11. MANAGING S3 VECTORS 115
11.1. S3 VECTORS IN MULTICLOUD OBJECT GATEWAY 115
11.2. CREATING A VECTOR BUCKETCLASS 115
11.3. CREATING VECTOR BUCKETS USING OBJECTBUCKETCLAIM 116
11.4. CREATING VECTOR BUCKETS USING S3 API 117
11.5. MANAGING VECTOR INDEXES 118
11.5.1. Listing vector indexes 118
11.5.2. Creating a vector index 119
11.5.3. Viewing vector index details 120
11.5.4. Deleting a vector index 120
11.6. DELETING VECTOR BUCKETS 121

CHAPTER 12. OBJECT BUCKET CLAIM 123
12.1. OBJECT BUCKET CLAIM 123
12.2. DYNAMIC OBJECT BUCKET CLAIM 123
12.3. CREATING AN OBJECT BUCKET CLAIM USING THE COMMAND-LINE INTERFACE 125
12.4. CREATING AN OBJECT BUCKET CLAIM USING THE OPENSHIFT WEB CONSOLE 128
12.5. ATTACHING AN OBJECT BUCKET CLAIM TO A DEPLOYMENT 129
12.6. VIEWING OBJECT BUCKETS USING THE OPENSHIFT WEB CONSOLE 129
12.7. DELETING OBJECT BUCKET CLAIMS 130

CHAPTER 13. MANAGING BUCKET QUOTA 131
13.1. BUCKET QUOTA IN MULTICLOUD OBJECT GATEWAY 131
13.1.1. Quota types 131
13.1.2. Quota configuration levels 131
13.1.3. Quota precedence rules 132
13.1.4. Supported bucket types 132
13.2. SETTING QUOTA ON A BUCKETCLASS 132
13.2.1. Creating a BucketClass with quota using YAML 132
13.2.2. Creating a BucketClass with quota using the CLI 133
13.3. SETTING QUOTA ON OBJECT BUCKET CLAIM USING YAML 134
13.3.1. Updating quota on an existing Object Bucket Claim 135
13.4. SETTING QUOTA ON OBJECT BUCKET CLAIM USING THE CLI 135
13.5. SETTING QUOTA ON BUCKETS 136
13.5.1. Creating a bucket with quota 137
13.5.2. Updating quota on an existing bucket 137

CHAPTER 14. CACHING POLICY FOR OBJECT BUCKETS 139

14.1. CREATING AN AWS CACHE BUCKET 139
14.2. CREATING AN IBM COS CACHE BUCKET 141

CHAPTER 15. LIFECYCLE BUCKET CONFIGURATION IN MULTICLOUD OBJECT GATEWAY 143

CHAPTER 16. SCALING MULTICLOUD OBJECT GATEWAY PERFORMANCE 144
16.1. AUTOMATIC SCALING OF MULTICLOUD OBJECT GATEWAY ENDPOINTS 144
16.2. AUTOSCALING RGW IN OPENSHIFT DATA FOUNDATION USING HPA AND KEDA 145
16.3. INCREASING CPU AND MEMORY FOR PV POOL RESOURCES 147

CHAPTER 17. ACCESSING THE RADOS OBJECT GATEWAY S3 ENDPOINT 149

CHAPTER 18. USING TLS CERTIFICATES FOR APPLICATIONS ACCESSING RGW 150
18.1. ACCESSING EXTERNAL RGW SERVER IN OPENSHIFT DATA FOUNDATION 150

CHAPTER 19. USING THE MULTICLOUD OBJECT GATEWAY'S SECURITY TOKEN SERVICE TO ASSUME THE ROLE OF ANOTHER USER 152

PREFACE

This document describes how to configure and manage the Multicloud Object Gateway (MCG) for hybrid and multicloud resource management.

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your input on our documentation. Do let us know how we can make it better.

To give feedback, create a Jira ticket:

Procedure

1. Log in to the Jira.
2. Click **Create** in the top navigation bar
3. Enter a descriptive title in the Summary field.
4. Enter your suggestion for improvement in the Description field. Include links to the relevant parts of the documentation.
5. Select **Documentation** in the Components field.
6. Click Create at the bottom of the dialogue.

CHAPTER 1. ABOUT THE MULTICLOUD OBJECT GATEWAY

The Multicloud Object Gateway (MCG) is a lightweight object storage service for OpenShift, allowing users to start small and then scale as needed on-premise, in multiple clusters, and with cloud-native storage.

CHAPTER 2. ACCESSING THE MULTICLOUD OBJECT GATEWAY WITH YOUR APPLICATIONS

You can access the object service with any application targeting AWS S3 or code that uses AWS S3 Software Development Kit (SDK). Applications need to specify the Multicloud Object Gateway (MCG) endpoint, an access key, and a secret access key. You can use your terminal or the MCG CLI to retrieve this information.

Prerequisites

- A running OpenShift Data Foundation Platform.

Additional resources

- Accessing the RADOS Object Gateway S3 endpoint

2.1. ACCESSING THE MULTICLOUD OBJECT GATEWAY FROM THE TERMINAL

Procedure

1. Run the **describe** command to view information about the Multicloud Object Gateway (MCG) endpoint, including its access key (**AWS_ACCESS_KEY_ID** value) and secret access key (**AWS_SECRET_ACCESS_KEY** value).

   ```
   # oc describe noobaa -n openshift-storage
   ```

   The output will look similar to the following:

   ```
   Name:         noobaa
   Namespace:    openshift-storage
   Labels:       <none>
   Annotations:  <none>
   API Version:  noobaa.io/v1alpha1
   Kind:         NooBaa
   Metadata:
     Creation Timestamp:  2019-07-29T16:22:06Z
     Generation:          1
     Resource Version:    6718822
     Self Link:           /apis/noobaa.io/v1alpha1/namespaces/openshift-storage/noobaas/noobaa
     UID:                 019cfb4a-b21d-11e9-9a02-06c8de012f9e
   Spec:
   Status:
     Accounts:
       Admin:
         Secret Ref:
           Name:          noobaa-admin
           Namespace:     openshift-storage
     Actual Image:        noobaa/noobaa-core:4.0
     Observed Generation:  1
     Phase:               Ready
     Readme:
   ```

```
Welcome to NooBaa!
-----------------

Welcome to NooBaa!
-----------------
NooBaa Core Version:
NooBaa Operator Version:

Lets get started:

Test S3 client:

  kubectl port-forward -n openshift-storage service/s3 10443:443 &
(1)
  NOOBAA_ACCESS_KEY=$(kubectl get secret noobaa-admin -n openshift-storage -o
json | jq -r '.data.AWS_ACCESS_KEY_ID|@base64d')
(2)
  NOOBAA_SECRET_KEY=$(kubectl get secret noobaa-admin -n openshift-storage -o
json | jq -r '.data.AWS_SECRET_ACCESS_KEY|@base64d')
  alias s3='AWS_ACCESS_KEY_ID=$NOOBAA_ACCESS_KEY
AWS_SECRET_ACCESS_KEY=$NOOBAA_SECRET_KEY aws --endpoint
https://localhost:10443 --no-verify-ssl s3'
  s3 ls


Services:
  Service Mgmt:
    External DNS:
      https://noobaa-mgmt-openshift-storage.apps.mycluster-cluster.qe.rh-ocs.com
      https://a3406079515be11eaa3b70683061451e-1194613580.us-east-
2.elb.amazonaws.com:443
    Internal DNS:
      https://noobaa-mgmt.openshift-storage.svc:443
    Internal IP:
      https://172.30.235.12:443
    Node Ports:
      https://10.0.142.103:31385
    Pod Ports:
      https://10.131.0.19:8443
  serviceS3:
    External DNS: (3)
      https://s3-openshift-storage.apps.mycluster-cluster.qe.rh-ocs.com
      https://a340f4e1315be11eaa3b70683061451e-943168195.us-east-
2.elb.amazonaws.com:443
    Internal DNS:
      https://s3.openshift-storage.svc:443
    Internal IP:
      https://172.30.86.41:443
    Node Ports:
      https://10.0.142.103:31011
    Pod Ports:
      https://10.131.0.19:6443
```

- **<1>** access key (**AWS_ACCESS_KEY_ID** value)
- **<2>** secret access key (**AWS_SECRET_ACCESS_KEY** value)

- **<3>** MCG endpoint



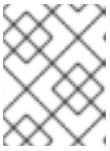
NOTE

The output from the **oc describe noobaa** command lists the internal and external DNS names that are available. When using the internal DNS, the traffic is free. The external DNS uses Load Balancing to process the traffic, and therefore has a cost per hour.

2.2. ACCESSING THE MULTICLOUD OBJECT GATEWAY FROM THE COMMAND-LINE INTERFACE

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Run the **status** command to access the endpoint, access key, and secret access key:

```
odf noobaa status -n openshift-storage
```

The output will look similar to the following:

```
INFO[0000] Namespace: openshift-storage
INFO[0000]
INFO[0000] CRD Status:
INFO[0003]  Exists: CustomResourceDefinition "noobaas.noobaa.io"
INFO[0003]  Exists: CustomResourceDefinition "backingstores.noobaa.io"
INFO[0003]  Exists: CustomResourceDefinition "bucketclasses.noobaa.io"
INFO[0004]  Exists: CustomResourceDefinition "objectbucketclaims.objectbucket.io"
INFO[0004]  Exists: CustomResourceDefinition "objectbuckets.objectbucket.io"
INFO[0004]
INFO[0004] Operator Status:
INFO[0004]  Exists: Namespace "openshift-storage"
INFO[0004]  Exists: ServiceAccount "noobaa"
INFO[0005]  Exists: Role "ocs-operator.v0.0.271-6g45f"
INFO[0005]  Exists: RoleBinding "ocs-operator.v0.0.271-6g45f-noobaa-f9vpj"
INFO[0006]  Exists: ClusterRole "ocs-operator.v0.0.271-fjhgh"
INFO[0006]  Exists: ClusterRoleBinding "ocs-operator.v0.0.271-fjhgh-noobaa-pdxn5"
INFO[0006]  Exists: Deployment "noobaa-operator"
INFO[0006]
INFO[0006] System Status:
INFO[0007]  Exists: NooBaa "noobaa"
INFO[0007]  Exists: StatefulSet "noobaa-core"
INFO[0007]  Exists: Service "noobaa-mgmt"
INFO[0008]  Exists: Service "s3"
```

```
INFO[0008]   Exists: Secret "noobaa-server"
INFO[0008]   Exists: Secret "noobaa-operator"
INFO[0008]   Exists: Secret "noobaa-admin"
INFO[0009]   Exists: StorageClass "openshift-storage.noobaa.io"
INFO[0009]   Exists: BucketClass "noobaa-default-bucket-class"
INFO[0009]   (Optional) Exists: BackingStore "noobaa-default-backing-store"
INFO[0010]   (Optional) Exists: CredentialsRequest "noobaa-cloud-creds"
INFO[0010]   (Optional) Exists: PrometheusRule "noobaa-prometheus-rules"
INFO[0010]   (Optional) Exists: ServiceMonitor "noobaa-service-monitor"
INFO[0011]   (Optional) Exists: Route "noobaa-mgmt"
INFO[0011]   (Optional) Exists: Route "s3"
INFO[0011]   Exists: PersistentVolumeClaim "db-noobaa-core-0"
INFO[0011]   System Phase is "Ready"
INFO[0011]   Exists:  "noobaa-admin"

#------------------#
#- Mgmt Addresses -#
#------------------#

ExternalDNS : [https://noobaa-mgmt-openshift-storage.apps.mycluster-cluster.qe.rh-ocs.com
https://a3406079515be11eaa3b70683061451e-1194613580.us-east-2.elb.amazonaws.com:443]
ExternalIP  : []
NodePorts   : [https://10.0.142.103:31385]
InternalDNS : [https://noobaa-mgmt.openshift-storage.svc:443]
InternalIP  : [https://172.30.235.12:443]
PodPorts    : [https://10.131.0.19:8443]

#--------------------#
#- Mgmt Credentials -#
#--------------------#

email    : admin@noobaa.io
password : HKLbH1rSuVU0I/souIkSiA==

#----------------#
#- S3 Addresses -#
#----------------#

(1)
ExternalDNS : [https://s3-openshift-storage.apps.mycluster-cluster.qe.rh-ocs.com
https://a340f4e1315be11eaa3b70683061451e-943168195.us-east-2.elb.amazonaws.com:443]
ExternalIP  : []
NodePorts   : [https://10.0.142.103:31011]
InternalDNS : [https://s3.openshift-storage.svc:443]
InternalIP  : [https://172.30.86.41:443]
PodPorts    : [https://10.131.0.19:6443]

#------------------#
#- S3 Credentials -#
#------------------#

(2)
AWS_ACCESS_KEY_ID     : jVmAsu9FsvRHYmfjTiHV
(3)
AWS_SECRET_ACCESS_KEY : E//420VNedJfATvVSmDz6FMtsSAzuBv6z180PT5c
```

```
#------------------#
#- Backing Stores -#
#------------------#

NAME                          TYPE     TARGET-BUCKET                                      PHASE   AGE
noobaa-default-backing-store   aws-s3   noobaa-backing-store-15dc896d-7fe0-4bed-9349-
5942211b93c9   Ready   141h35m32s

#------------------#
#- Bucket Classes -#
#------------------#

NAME                          PLACEMENT                                                  PHASE   AGE
noobaa-default-bucket-class   {Tiers:[{Placement: BackingStores:[noobaa-default-backing-store]}]}
Ready   141h35m33s

#------------------#
#- Bucket Claims -#
#------------------#

No OBC's found.
```

<1> endpoint **<2>** access key **<3>** secret access key

+ You have the relevant endpoint, access key, and secret access key in order to connect to your applications.

+ If AWS S3 CLI is the application, the following command will list the buckets in OpenShift Data Foundation:

+

```
AWS_ACCESS_KEY_ID=<AWS_ACCESS_KEY_ID>
AWS_SECRET_ACCESS_KEY=<AWS_SECRET_ACCESS_KEY>
aws --endpoint <ENDPOINT> --no-verify-ssl s3 ls
```

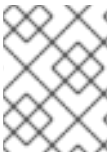
2.3. SUPPORT OF MULTICLOUD OBJECT GATEWAY DATA BUCKET APIS

The following table lists the Multicloud Object Gateway (MCG) data bucket APIs and their support levels.

Data buckets	**Support**	
List buckets	Supported	
Delete bucket	Supported	Replication configuration is part of MCG bucket class configuration

Create bucket	Supported	A distinct set of canned ACLs is available exclusively for NS-S3 and NS-Cache buckets when layered on top of AWS S3 or AWS S3–compatible storage.
Post bucket	Not supported	
Put bucket	Partially supported	Replication configuration is part of MCG bucket class configuration
Bucket lifecycle	Partially supported	Object expiration only
Policy (Buckets, Objects)	Partially supported	Bucket policies are supported
Bucket Website	Supported	
Bucket ACLs (Get, Put)	Supported	A distinct set of canned ACLs is available exclusively for NS-S3 and NS-Cache buckets when layered on top of AWS S3 or AWS S3–compatible storage.
Bucket Location	Partially	Returns a default value only
Bucket Notification	Not supported	
Bucket Object Versions	Supported	
Get Bucket Info (HEAD)	Supported	
Bucket Request Payment	Partially supported	Returns the bucket owner
Put Object	Supported	
Delete Object	Supported	
Get Object	Supported	
Object ACLs (Get, Put)	Supported	
Get Object Info (HEAD)	Supported	
POST Object	Supported	
Copy Object	Supported	

Multipart Uploads	Supported	
Object Tagging	Supported	
Storage Class	Not supported	



NOTE

No support for **cors, metrics, inventory, analytics, inventory, logging, notifications, accelerate, replication, request payment, locks verbs**

CHAPTER 3. ADDING STORAGE RESOURCES FOR HYBRID OR MULTICLOUD

Connect AWS, Azure, GCP, IBM COS, and local persistent volumes as backing stores in the Multicloud Object Gateway to enable hybrid and multicloud object storage for OpenShift Data Foundation.

3.1. CREATING A NEW BACKING STORE

Use this procedure to create a new backing store in OpenShift Data Foundation.

Prerequisites

- Administrator access to OpenShift Data Foundation.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Backing Store** tab.
3. Click **Create Backing Store**.
4. On the **Create New Backing Store** page, perform the following:
   a. Enter a **Backing Store Name**.
   b. Select a **Provider**.
   c. Select a **Region**.
   d. Optional: Enter an **Endpoint**.
   e. Select a **Secret** from the drop-down list, or create your own secret. Optionally, you can **Switch to Credentials** view which lets you fill in the required secrets.
   For more information on creating an OCP secret, see the section Creating the secret in the *Openshift Container Platform* documentation.

   Each backingstore requires a different secret. For more information on creating the secret for a particular backingstore, see the Section 3.3, "Adding storage resources for hybrid or Multicloud using the command-line interface" and follow the procedure for the addition of storage resources using a YAML.

   NOTE

   This menu is relevant for all providers except Google Cloud and local PVC.

   f. Enter the **Target bucket**. The target bucket is a container storage that is hosted on the remote cloud service. It allows you to create a connection that tells the MCG that it can use this bucket for the system.
5. Click **Create Backing Store**.

Verification

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Backing Store** tab to view all the backing stores.

3.2. OVERRIDING THE DEFAULT BACKING STORE

You can use the **manualDefaultBackingStore** flag to override the default NooBaa backing store and remove it if you do not want to use the default backing store configuration. This provides flexibility to customize your backing store configuration and tailor it to your specific needs. By leveraging this feature, you can further optimize your system and enhance its performance.

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

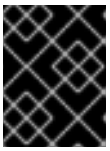
Procedure

1. Check if **noobaa-default-backing-store** is present:

```
$ oc get backingstore
NAME TYPE PHASE AGE
noobaa-default-backing-store pv-pool Creating 102s
```

2. Patch the NooBaa CR to enable **manualDefaultBackingStore**:

```
$ oc patch noobaa/noobaa --type json --
patch='[{"op":"add","path":"/spec/manualDefaultBackingStore","value":true}]'
```



IMPORTANT

Use the Multicloud Object Gateway CLI to create a new backing store and update accounts.

3. Create a new default backing store to override the default backing store. For example:

```
$ odf noobaa backingstore create pv-pool _NEW-DEFAULT-BACKING-STORE_ --num-
volumes 1 --pv-size-gb 16
```

   a. Replace ***NEW-DEFAULT-BACKING-STORE*** with the name you want for your new default backing store.

4. Update the admin account to use the new default backing store as its default resource:

```
$ odf noobaa account update admin@noobaa.io --new_default_resource=_NEW-DEFAULT-
BACKING-STORE_
```

a. Replace ***NEW-DEFAULT-BACKING-STORE*** with the name of the backing store from the previous step.
Updating the default resource for admin accounts ensures that the new configuration is used throughout your system.

5. Configure the default-bucketclass to use the new default backingstore:

```
$ oc patch Bucketclass noobaa-default-bucket-class -n openshift-storage --type=json --patch='[{"op": "replace", "path": "/spec/placementPolicy/tiers/0/backingStores/0", "value": "NEW-DEFAULT-BACKING-STORE"}]'
```

6. Optional: Delete the noobaa-default-backing-store.

a. Delete all instances of and buckets associated with **noobaa-default-backing-store** and update the accounts using it as resource.

b. Delete the noobaa-default-backing-store:

```
$ oc delete backingstore noobaa-default-backing-store -n openshift-storage | oc patch -n openshift-storage backingstore/noobaa-default-backing-store --type json --patch='[ { "op": "remove", "path": "/metadata/finalizers" } ]'
```

You must enable the **manualDefaultBackingStore** flag before proceeding. Additionally, it is crucial to update all accounts that use the default resource and delete all instances of and buckets associated with the default backing store to ensure a smooth transition.

3.3. ADDING STORAGE RESOURCES FOR HYBRID OR MULTICLOUD USING THE COMMAND-LINE INTERFACE

The Multicloud Object Gateway (MCG) simplifies the process of spanning data across the cloud provider and clusters.

Add a backing storage that can be used by the MCG.

Depending on the type of your deployment, you can choose one of the following procedures to create a backing storage:

- For creating an AWS-backed backingstore, see Section 3.3.1, “Creating an AWS-backed backingstore”
- For creating an AWS-STS-backed backingstore, see Section 3.3.2, “Creating an AWS-STS-backed backingstore”
- For creating an IBM COS-backed backingstore, see Section 3.3.5, “Creating an IBM COS-backed backingstore”
- For creating an Azure-backed backingstore, see Section 3.3.6, “Creating an Azure-backed backingstore”
- For creating an Azure STS-backed backingstore, see Section 3.3.7, “Creating an Azure STS-backed backingstore”
- For creating a GCP-backed backingstore, see Section 3.3.8, “Creating a GCP-backed backingstore”

- For creating a local Persistent Volume-backed backingstore, see Section 3.3.9, "Creating a local Persistent Volume-backed backingstore"

For VMware deployments, skip to Section 3.4, "Creating an s3 compatible Multicloud Object Gateway backingstore" for further instructions.

3.3.1. Creating an AWS-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:

```
odf noobaa backingstore create aws-s3 <backingstore_name> --access-key=<AWS ACCESS KEY> --secret-key=<AWS SECRET ACCESS KEY> --target-bucket <bucket-name> -n openshift-storage
```

<backingstore_name>

The name of the backingstore.

<AWS ACCESS KEY> and <AWS SECRET ACCESS KEY>

The AWS access key ID and secret access key you created for this purpose.

<bucket-name>

The existing AWS bucket name. This argument indicates to the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration. The output will be similar to the following:

```
INFO[0001]  Exists: NooBaa "noobaa"
INFO[0002]  Created: BackingStore "aws-resource"
INFO[0002]  Created: Secret "backing-store-secret-aws-resource"
```

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
  namespace: openshift-storage
type: Opaque
data:
  AWS_ACCESS_KEY_ID: <AWS ACCESS KEY ID ENCODED IN BASE64>
  AWS_SECRET_ACCESS_KEY: <AWS SECRET ACCESS KEY ENCODED IN BASE64>
```

<AWS ACCESS KEY> and ***<AWS SECRET ACCESS KEY>***

Supply and encode your own AWS access key ID and secret access key using Base64, and use the results for **<AWS ACCESS KEY ID ENCODED IN BASE64>** and **<AWS SECRET ACCESS KEY ENCODED IN BASE64>**.

<backingstore-secret-name>

The name of the backingstore secret created in the previous step.

1. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  awsS3:
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBucket: <bucket-name>
  type: aws-s3
```

<bucket-name>

The existing AWS bucket name.

<backingstore-secret-name>

The name of the backingstore secret created in the previous step.

3.3.2. Creating an AWS-STS-backed backingstore

Amazon Web Services Security Token Service (AWS STS) is an AWS capability that provides authentication through short-lived, temporary credentials. Creating an AWS-STS-backed backingstore is supported only on clusters deployed on AWS using STS. The process involves the following steps:

- Creating an AWS role using a script, which helps to get the temporary security credentials for the role session
- Installing OpenShift Data Foundation operator in AWS STS OpenShift cluster
- Creating backingstore in AWS STS OpenShift cluster

3.3.2.1. Creating an AWS role using a script

You need to create a role and pass the role Amazon resource name (ARN) while installing the OpenShift Data Foundation operator.

Prerequisites

- Configure Red Hat OpenShift Container Platform cluster with AWS STS. For more information, see Configuring an AWS cluster to use short-term credentials .

Procedure

- Create an AWS role using a script that matches OpenID Connect (OIDC) configuration for Multicloud Object Gateway (MCG) on OpenShift Data Foundation.
  The following example shows the details that are required to create the role:

```
{
    “Version”: “2012-10-17",
    “Statement”: [
        {
            “Effect”: “Allow”,
            “Principal”: {
                “Federated”: “arn:aws:iam::123456789123:oidc-provider/mybucket-oidc.s3.us-east-2.amazonaws.com”
            },
            “Action”: “sts:AssumeRoleWithWebIdentity”,
            “Condition”: {
                “StringEquals”: {
                    “mybucket-oidc.s3.us-east-2.amazonaws.com:sub”: [
                        “system:serviceaccount:openshift-storage:noobaa”,
                        "system:serviceaccount:openshift-storage:noobaa-core",
                        “system:serviceaccount:openshift-storage:noobaa-endpoint”
                    ]
                }
            }
        }
    ]
}
```

where

123456789123

Is the AWS account ID

mybucket

Is the bucket name (using public bucket configuration)

us-east-2

Is the AWS region

openshift-storage

Is the namespace name

Sample script

```
#!/bin/bash
set -x

# This is a sample script to help you deploy MCG on AWS STS cluster.
# This script shows how to create role-policy and then create the role in AWS.
# For more information see: https://docs.openshift.com/rosa/authentication/assuming-an-aws-iam-role-for-a-service-account.html
```

```
# WARNING: This is a sample script. You need to adjust the variables based on your
requirement.

# Variables :
# user variables - REPLACE these variables with your values:
ROLE_NAME="<role-name>" # role name that you pick in your AWS account
NAMESPACE="<namespace>" # namespace name where MCG is running. For
OpenShift Data Foundation, it is openshift-storage.

# MCG variables
SERVICE_ACCOUNT_NAME_1="noobaa" # The service account name of deployment
operator
SERVICE_ACCOUNT_NAME_2="noobaa-endpoint" # The service account name of
deployment endpoint
SERVICE_ACCOUNT_NAME_3="noobaa-core" # The service account name of
statefulset core

# AWS variables
# Make sure these values are not empty (AWS_ACCOUNT_ID, OIDC_PROVIDER)
# AWS_ACCOUNT_ID is your AWS account number
AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query "Account" --output text)
# If you want to create the role before using the cluster, replace this field too.
# The OIDC provider is in the structure:
# 1) <OIDC-bucket>.s3.<aws-region>.amazonaws.com. for OIDC bucket configurations
are in an S3 public bucket
# 2) `<characters>.cloudfront.net` for OIDC bucket configurations in an S3 private bucket
with a public CloudFront distribution URL
OIDC_PROVIDER=$(oc get authentication cluster -ojson | jq -r
.spec.serviceAccountIssuer | sed -e "s/^https:\/\//")
# the permission (S3 full access)
POLICY_ARN_STRINGS="arn:aws:iam::aws:policy/AmazonS3FullAccess"

# Creating the role (with AWS command-line interface)

read -r -d '' TRUST_RELATIONSHIP <<EOF
{
 "Version": "2012-10-17",
 "Statement": [
   {
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::${AWS_ACCOUNT_ID}:oidc-
provider/${OIDC_PROVIDER}"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
     "${OIDC_PROVIDER}:sub": [
       "system:serviceaccount:${NAMESPACE}:${SERVICE_ACCOUNT_NAME_1}",
       "system:serviceaccount:${NAMESPACE}:${SERVICE_ACCOUNT_NAME_2}",
        "system:serviceaccount:${NAMESPACE}:${SERVICE_ACCOUNT_NAME_3}"
      ]
    }
  }
   }
 ]
```

```
}
EOF

echo "${TRUST_RELATIONSHIP}" > trust.json

aws iam create-role --role-name "$ROLE_NAME" --assume-role-policy-document
file://trust.json --description "role for demo"

while IFS= read -r POLICY_ARN; do
  echo -n "Attaching $POLICY_ARN ... "
  aws iam attach-role-policy \
   --role-name "$ROLE_NAME" \
   --policy-arn "${POLICY_ARN}"
  echo "ok."
done <<< "$POLICY_ARN_STRINGS"
```

3.3.3. Installing OpenShift Data Foundation operator in AWS STS OpenShift cluster

Prerequisites

- Configure Red Hat OpenShift Container Platform cluster with AWS STS. For more information, see Configuring an AWS cluster to use short-term credentials .
- Create an AWS role using a script that matches OpenID Connect (OIDC) configuration. For more information, see Creating an AWS role using a script .

Procedure

- Install OpenShift Data Foundation Operator from the Software Catalog.
  - During the installation add the role ARN in the **ARN Details** field.
  - Make sure that the **Update approval** field is set to **Manual**.

3.3.4. Creating a new AWS STS backingstore

Prerequisites

- Configure Red Hat OpenShift Container Platform cluster with AWS STS. For more information, see Configuring an AWS cluster to use short-term credentials .
- Create an AWS role using a script that matches OpenID Connect (OIDC) configuration. For more information, see Creating an AWS role using a script .
- Install OpenShift Data Foundation Operator. For more information, see Installing OpenShift Data Foundation operator in AWS STS OpenShift cluster.

Procedure

1. Install Multicloud Object Gateway (MCG).
   It is installed with the default backingstore by using the short-lived credentials.

2. After the MCG system is ready, you can create more backingstores of the type **aws-sts-s3** using the following command-line interface command:

```
$ odf noobaa backingstore create aws-sts-s3 <backingstore-name> --aws-sts-arn=<aws-sts-role-arn> --region=<region> --target-bucket=<target-bucket>
```

where

backingstore-name

Name of the backingstore

aws-sts-role-arn

The AWS STS role ARN which will assume role

region

The AWS bucket region

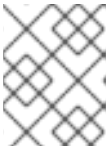
target-bucket

The target bucket name on the cloud

3.3.5. Creating an IBM COS-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. From the command-line interface, run the following command:

```
odf noobaa backingstore create ibm-cos <backingstore_name> --access-key=<IBM ACCESS KEY> --secret-key=<IBM SECRET ACCESS KEY> --endpoint=<IBM COS ENDPOINT> --target-bucket <bucket-name> -n openshift-storage
```

<backingstore_name>

The name of the backingstore.

<IBM ACCESS KEY>, ***<IBM SECRET ACCESS KEY>***, and ***<IBM COS ENDPOINT>***

An IBM access key ID, secret access key and the appropriate regional endpoint that corresponds to the location of the existing IBM bucket.
To generate the above keys on IBM cloud, you must include HMAC credentials while creating the service credentials for your target bucket.

<bucket-name>

An existing IBM bucket name. This argument indicates MCG about the bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "ibm-resource"
INFO[0002]   Created: Secret "backing-store-secret-ibm-resource"
```

2. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
  namespace: openshift-storage
type: Opaque
data:
  IBM_COS_ACCESS_KEY_ID: <IBM COS ACCESS KEY ID ENCODED IN BASE64>
  IBM_COS_SECRET_ACCESS_KEY: <IBM COS SECRET ACCESS KEY ENCODED IN
BASE64>
```

<IBM COS ACCESS KEY ID ENCODED IN BASE64> and <IBM COS SECRET ACCESS KEY ENCODED IN BASE64>

Provide and encode your own IBM COS access key ID and secret access key using Base64, and use the results in place of these attributes respectively.

<backingstore-secret-name>

The name of the backingstore secret.

3. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  ibmCos:
    endpoint: <endpoint>
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBucket: <bucket-name>
  type: ibm-cos
```

<bucket-name>

an existing IBM COS bucket name. This argument indicates to MCG about the bucket to use as a target bucket for its backingstore, and subsequently, data storage and administration.

<endpoint>

A regional endpoint that corresponds to the location of the existing IBM bucket name. This argument indicates to MCG about the endpoint to use for its backingstore, and subsequently, data storage and administration.

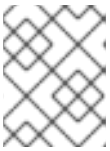
<backingstore-secret-name>

The name of the secret created in the previous step.

3.3.6. Creating an Azure-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:

```
odf noobaa backingstore create azure-blob <backingstore_name> --account-key=<AZURE ACCOUNT KEY> --account-name=<AZURE ACCOUNT NAME> --target-blob-container <blob container name> -n openshift-storage
```

<backingstore_name>

The name of the backingstore.

<AZURE ACCOUNT KEY> and <AZURE ACCOUNT NAME>

An AZURE account key and account name you created for this purpose.

<blob container name>

An existing Azure blob container name. This argument indicates to MCG about the bucket to use as a target bucket for its backingstore, and subsequently, data storage and administration.
The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "azure-resource"
INFO[0002]   Created: Secret "backing-store-secret-azure-resource"
```

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
type: Opaque
data:
  AccountName: <AZURE ACCOUNT NAME ENCODED IN BASE64>
  AccountKey: <AZURE ACCOUNT KEY ENCODED IN BASE64>
```

<AZURE ACCOUNT NAME ENCODED IN BASE64> and <AZURE ACCOUNT KEY ENCODED IN BASE64>

Supply and encode your own Azure Account Name and Account Key using Base64, and use the results in place of these attributes respectively.

<backingstore-secret-name>

A unique name of backingstore secret.

1. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  azureBlob:
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBlobContainer: <blob-container-name>
  type: azure-blob
```

<blob-container-name>

An existing Azure blob container name. This argument indicates to the MCG about the bucket to use as a target bucket for its backingstore, and subsequently, data storage and administration.

<backingstore-secret-name>

with the name of the secret created in the previous step.

3.3.7. Creating an Azure STS-backed backingstore

Azure Identity (also known as Azure Workload Identity) is an Azure capability that provides authentication through short-lived, temporary credentials. Creating an Azure STS-backed backingstore is supported on any OpenShift cluster with Azure Workload Identity and OIDC provider configured. The process involves the following steps:

- Creating an Azure managed identity for NooBaa
- Installing OpenShift Data Foundation operator in Azure STS OpenShift cluster
- Creating a new Azure STS backingstore

3.3.7.1. Creating an Azure managed identity for NooBaa

You need to create a user-assigned managed identity with federated credentials and pass the managed identity client ID while installing the OpenShift Data Foundation operator.

Prerequisites

- Red Hat OpenShift Container Platform cluster configured with Azure Workload Identity and OIDC provider. For more information, see Configuring an Azure cluster to use short-term credentials.

- Azure CLI (**az**) installed and authenticated.
- OpenShift CLI (**oc**) installed and logged in to the cluster.

Procedure

1. Set the required environment variables:

```
CLUSTER_NAME=<cluster_name>
RESOURCE_GROUP=<cluster_resource_group>
SUBSCRIPTION_ID=<azure_subscription_id>
REGION=<azure_region>
NAMESPACE=openshift-storage
MI_NAME=${CLUSTER_NAME}-noobaa-mi
```

where

cluster_name
The name of your OpenShift cluster

cluster_resource_group
The Azure resource group containing your cluster

azure_subscription_id
Your Azure subscription ID

azure_region
The Azure region where your cluster is deployed

2. Create a user-assigned managed identity:

```
az identity create \
  --name ${MI_NAME} \
  --resource-group ${RESOURCE_GROUP} \
  --location ${REGION} \
  --subscription ${SUBSCRIPTION_ID}
```

3. Retrieve the managed identity's client ID and principal ID:

```
MI_CLIENT_ID=$(az identity show --name ${MI_NAME} --resource-group
${RESOURCE_GROUP} --subscription ${SUBSCRIPTION_ID} --query clientId -o tsv)
MI_PRINCIPAL_ID=$(az identity show --name ${MI_NAME} --resource-group
${RESOURCE_GROUP} --subscription ${SUBSCRIPTION_ID} --query principalId -o tsv)
```

4. Assign the required roles to the managed identity:

```
SCOPE="/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RESOURCE_GROUP}"
az role assignment create \
  --assignee-object-id ${MI_PRINCIPAL_ID} \
  --assignee-principal-type ServicePrincipal \
  --role "Storage Account Contributor" \
  --scope ${SCOPE} \
  --subscription ${SUBSCRIPTION_ID}
az role assignment create \
  --assignee-object-id ${MI_PRINCIPAL_ID} \
```

```
    --assignee-principal-type ServicePrincipal \
    --role "Storage Blob Data Contributor" \
    --scope ${SCOPE} \
    --subscription ${SUBSCRIPTION_ID}
```

5. Retrieve the OIDC issuer URL from the cluster:

```
OIDC_ISSUER=$(oc get authentication cluster -o jsonpath='{.spec.serviceAccountIssuer}')
```

6. Create federated credentials for the NooBaa service accounts (**noobaa**, **noobaa-core**, and **noobaa-endpoint**):

```
for SA in noobaa noobaa-core noobaa-endpoint; do
  az identity federated-credential create \
    --name ${MI_NAME}-${SA} \
    --identity-name ${MI_NAME} \
    --resource-group ${RESOURCE_GROUP} \
    --issuer ${OIDC_ISSUER} \
    --subject "system:serviceaccount:${NAMESPACE}:${SA}" \
    --audiences openshift \
    --subscription ${SUBSCRIPTION_ID}
done
```

7. Note the managed identity client ID (**MI_CLIENT_ID**) - you will need it when installing the OpenShift Data Foundation operator.

```
echo "Managed Identity Client ID: ${MI_CLIENT_ID}"
```

3.3.7.2. Installing OpenShift Data Foundation operator in Azure STS OpenShift cluster

Prerequisites

- Red Hat OpenShift Container Platform cluster configured with Azure Workload Identity and OIDC provider. For more information, see Configuring an Azure cluster to use short-term credentials.
- Create an Azure managed identity with federated credentials for NooBaa service accounts. For more information, see Creating an Azure managed identity for NooBaa .

Procedure

- Install OpenShift Data Foundation Operator from the Software Catalog.
  - During the installation, enter the following values in their respective fields:
    - **Azure Client ID**: The managed identity's client ID from the procedure in the prerequisites (not the service principal's client ID)
    - **Azure Tenant ID**: Your Azure Active Directory tenant ID
    - **Azure Subscription ID**: Your Azure subscription ID
    - **Azure Resource Group**: The cluster's resource group name

- Make sure that the **Update approval** field is set to **Manual**.

3.3.7.3. Creating a new Azure STS backingstore

Prerequisites

- Red Hat OpenShift Container Platform cluster configured with Azure Workload Identity and OIDC provider. For more information, see Configuring an Azure cluster to use short-term credentials.
- Create an Azure managed identity with federated credentials for NooBaa service accounts. For more information, see Creating an Azure managed identity for NooBaa .
- Install OpenShift Data Foundation operator in Azure STS OpenShift cluster. For more information, see Installing OpenShift Data Foundation operator in Azure STS OpenShift cluster .

Procedure

1. Install Multicloud Object Gateway (MCG).
   It is installed with the default backingstore by using the short-lived Azure Identity credentials.
2. After the MCG system is ready, you can create more backingstores of the type **azure-sts-blob** using the following command-line interface command:

```
$ odf noobaa backingstore create azure-sts-blob <backingstore-name> \
  --account-name <storage-account-name> \
  --target-blob-container <container-name> \
  --tenant-id <tenant-id> \
  --client-id <client-id>
```

   where

   backingstore-name
   Name of the backingstore

   storage-account-name
   The Azure Storage Account name

   container-name
   The target blob container name in Azure

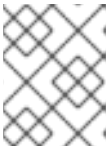
   tenant-id
   The Azure Active Directory Tenant ID

   client-id
   The Azure Managed Identity Client ID

3.3.8. Creating a GCP-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:

```
odf noobaa backingstore create google-cloud-storage <backingstore_name> --private-key-json-file=<PATH TO GCP PRIVATE KEY JSON FILE> --target-bucket <GCP bucket name> -n openshift-storage
```

<backingstore_name>

Name of the backingstore.

<PATH TO GCP PRIVATE KEY JSON FILE>

A path to your GCP private key created for this purpose.

<GCP bucket name>

An existing GCP object storage bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "google-gcp"
INFO[0002]   Created: Secret "backing-store-google-cloud-storage-gcp"
```

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <backingstore-secret-name>
type: Opaque
data:
  GoogleServiceAccountPrivateKeyJson: <GCP PRIVATE KEY ENCODED IN BASE64>
```

<GCP PRIVATE KEY ENCODED IN BASE64>

Provide and encode your own GCP service account private key using Base64, and use the results for this attribute.

<backingstore-secret-name>

A unique name of the backingstore secret.

1. Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
```

```
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  googleCloudStorage:
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    targetBucket: <target bucket>
  type: google-cloud-storage
```

<target bucket>

An existing Google storage bucket. This argument indicates to the MCG about the bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

<backingstore-secret-name>

The name of the secret created in the previous step.

3.3.9. Creating a local Persistent Volume-backed backingstore

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.

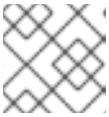


NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- From the command-line interface, run the following command:



NOTE

This command must be run from within the **openshift-storage** namespace.

```
$ odf noobaa -n openshift-storage backingstore create pv-pool <backingstore_name> --num-volumes <NUMBER OF VOLUMES>  --pv-size-gb <VOLUME SIZE> --request-cpu <CPU REQUEST> --request-memory <MEMORY REQUEST> --limit-cpu <CPU LIMIT> --limit-memory <MEMORY LIMIT> --storage-class <LOCAL STORAGE CLASS>
```

- Apply the following YAML for a specific backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
```

```
    app: noobaa
  name: <backingstore_name>
  namespace: openshift-storage
spec:
  pvPool:
   numVolumes: <NUMBER OF VOLUMES>
   resources:
    requests:
     storage: <VOLUME SIZE>
     cpu: <CPU REQUEST>
     memory: <MEMORY REQUEST>
    limits:
     cpu: <CPU LIMIT>
     memory: <MEMORY LIMIT>
   storageClass: <LOCAL STORAGE CLASS>
  type: pv-pool
```

<backingstore_name>

The name of the backingstore.

<NUMBER OF VOLUMES>

The number of volumes you would like to create. Note that increasing the number of volumes scales up the storage.

<VOLUME SIZE>

Required size in GB of each volume.

<CPU REQUEST>

Guaranteed amount of CPU requested in CPU unit **m**.

<MEMORY REQUEST>

Guaranteed amount of memory requested.

<CPU LIMIT>

Maximum amount of CPU that can be consumed in CPU unit **m**.

<MEMORY LIMIT>

Maximum amount of memory that can be consumed.

<LOCAL STORAGE CLASS>

The local storage class name, recommended to use **ocs-storagecluster-ceph-rbd**.

The output will be similar to the following:

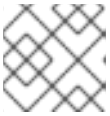
```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Exists: BackingStore "local-mcg-storage"
```

3.4. CREATING AN S3 COMPATIBLE MULTICLOUD OBJECT GATEWAY BACKINGSTORE

The Multicloud Object Gateway (MCG) can use any S3 compatible object storage as a backing store, for example, Red Hat Ceph Storage's RADOS Object Gateway (RGW). The following procedure shows how to create an S3 compatible MCG backing store for Red Hat Ceph Storage's RGW. Note that when the RGW is deployed, OpenShift Data Foundation operator creates an S3 compatible backingstore for MCG automatically.

Procedure

1. From the command-line interface, run the following command:



NOTE

This command must be run from within the **openshift-storage** namespace.

```
odf noobaa backingstore create s3-compatible rgw-resource --access-key=<RGW ACCESS KEY> --secret-key=<RGW SECRET KEY> --target-bucket=<bucket-name> --endpoint=<RGW endpoint> -n openshift-storage
```

   a. To get the **<RGW ACCESS KEY>** and **<RGW SECRET KEY>**, run the following command using your RGW user secret name:

```
oc get secret <RGW USER SECRET NAME> -o yaml -n openshift-storage
```

   b. Decode the access key ID and the access key from Base64 and keep them.

   c. Replace **<RGW USER ACCESS KEY>** and **<RGW USER SECRET ACCESS KEY>** with the appropriate, decoded data from the previous step.

   d. Replace **<bucket-name>** with an existing RGW bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

   e. To get the **<RGW endpoint>**, see Accessing the RADOS Object Gateway S3 endpoint . The output will be similar to the following:

```
INFO[0001]   Exists: NooBaa "noobaa"
INFO[0002]   Created: BackingStore "rgw-resource"
INFO[0002]   Created: Secret "backing-store-secret-rgw-resource"
```

2. Alternatively, create the backingstore using a YAML. Create a **CephObjectStore** user. This also creates a secret containing the RGW credentials:

```
apiVersion: ceph.rook.io/v1
kind: CephObjectStoreUser
metadata:
  name: <RGW-Username>
  namespace: openshift-storage
spec:
  store: ocs-storagecluster-cephobjectstore
  displayName: "<Display-name>"
```

   a. Replace **<RGW-Username>** and **<Display-name>** with a unique username and display name.

3. Apply the following YAML for an S3-Compatible backing store:

```
apiVersion: noobaa.io/v1alpha1
kind: BackingStore
metadata:
```

```
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <backingstore-name>
  namespace: openshift-storage
spec:
  s3Compatible:
    endpoint: <RGW endpoint>
    secret:
      name: <backingstore-secret-name>
      namespace: openshift-storage
    signatureVersion: v4
    targetBucket: <RGW-bucket-name>
  type: s3-compatible
```

a. Replace **<backingstore-secret-name>** with the name of the secret that was created with **CephObjectStore** in the previous step.

b. Replace **<bucket-name>** with an existing RGW bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

c. To get the **<RGW endpoint>**, see Accessing the RADOS Object Gateway S3 endpoint .

3.5. CREATING A NEW BUCKET CLASS

Bucket class is a CRD representing a class of buckets that defines tiering policies and data placements for an Object Bucket Class.

Use this procedure to create a bucket class in OpenShift Data Foundation.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab.
3. Click **Create Bucket Class**.
4. On the **Create new Bucket Class** page, perform the following:
   a. Select the bucket class type and enter a bucket class name.
      i. Select the **BucketClass type**. Choose one of the following options:
         - **Standard**: data will be consumed by a Multicloud Object Gateway (MCG), deduped, compressed and encrypted.
         - **Namespace**: data is stored on the NamespaceStores without performing de-duplication, compression or encryption.
         - **Vector**: specialized bucket class for storing vector embeddings for machine learning and AI applications. Vector bucket classes require an NSFS NamespaceStore and support only the Lance vector database type.
           By default, **Standard** is selected.

ii. Enter a **Bucket Class Name**.

iii. Click **Next**.



NOTE

If you select **Vector** as the bucket class type, the Placement Policy step is hidden, and you will be prompted to select an NSFS NamespaceStore resource. For more information about vector bucket classes, see Creating a vector BucketClass .

b. In **Placement Policy**, select **Tier 1 - Policy Type** and click **Next**. You can choose either one of the options as per your requirements.

- **Spread** allows spreading of the data across the chosen resources.
- **Mirror** allows full duplication of the data across the chosen resources.
- Click **Add Tier** to add another policy tier.

c. Select at least one **Backing Store** resource from the available list if you have selected **Tier 1 - Policy Type** as **Spread** and click **Next**. Alternatively, you can also create a new backing store.



NOTE

You need to select at least 2 backing stores when you select Policy Type as Mirror in previous step.

d. Review and confirm Bucket Class settings.

e. Click **Create Bucket Class**.

Verification

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab and search the new Bucket Class.

3.6. EDITING A BUCKET CLASS

Use the following procedure to edit the bucket class components through the YAML file by clicking the **edit** button on the Openshift web console.

Prerequisites

- Administrator access to OpenShift Web Console.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab.

3. Click the Action Menu **(⋮)** next to the Bucket class you want to edit.
4. Click **Edit Bucket Class**.
5. You are redirected to the **YAML** file, make the required changes in this file and click **Save**.

3.7. EDITING BACKING STORES FOR BUCKET CLASS

Use the following procedure to edit an existing Multicloud Object Gateway (MCG) bucket class to change the underlying backing stores used in a bucket class.

Prerequisites

- Administrator access to OpenShift Web Console.
- A bucket class.
- Backing stores.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. Click the **Bucket Class** tab.
3. Click the Action Menu **(⋮)** next to the Bucket class you want to edit.
4. Click **Edit Bucket Class Resources**.
5. On the **Edit Bucket Class Resources** page, edit the bucket class resources either by adding a backing store to the bucket class or by removing a backing store from the bucket class. You can also edit bucket class resources created with one or two tiers and different placement policies.
    - To add a backing store to the bucket class, select the name of the backing store.
    - To remove a backing store from the bucket class, uncheck the name of the backing store.
6. Click **Save**.

CHAPTER 4. CREATING AND MANAGING BUCKETS USING THE OBJECT BROWSER

The object browser in the OpenShift Web Console enables you to create, view, and manage buckets and objects across different storage endpoints, including Multicloud Object Gateway (MCG) and internal mode RADOS Gateway (RGW) object storage.

A storage endpoint selector is available under Storage → Object Storage → Buckets, allowing you to switch between the MCG and RGW. After selecting an endpoint, the object browser presents a unified interface for creating and managing buckets, uploading and viewing objects, configuring policies, and performing related operations.

4.1. ACCESSING THE OBJECT BROWSER

The access flow depends on your user role:

- Administrator users – Administrators have direct access to the object browser without additional login steps.
- Developer users – Developers must sign in before accessing the object browser.

4.1.1. Signing in to the object browser as a developer user

Developers must sign in before accessing the object browser.

Prerequisites

- Kubernetes Secret containing S3 credentials.
- Namespace and Secret name details.

Procedure

1. Open the object browser in the OpenShift Web Console.
2. Select the storage endpoint:
   - MCG
   - RGW
     Developers must log in separately for each endpoint.
3. Select the Secret namespace and name where the S3 credentials are stored.
4. (Optional) Select Stay signed in to remain logged in across browser tabs or sessions.
5. Click Sign in.



NOTE

- The UI does not store credentials in local or session storage but only a reference to the Kubernetes Secret is stored.
- Access level depends on the credentials provided. For more information, see <link to openshift documentation>.

4.2. CREATING AND MANAGING BUCKETS

4.2.1. Creating new buckets using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, click **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then click the **Buckets** tab.
3. Choose one of the following options to create the buckets:
   - **Create via Object Bucket Claim**

     NOTE

     For RGW object bucket claims (OBCs), you must use the credentials associated with the OBC to access the newly created bucket. RGW administrator credentials cannot be used to access these buckets.

     a. Select a namespace.

     b. Enter the **ObjectBucketClaim** name.

     c. Select the storage class.

     d. Select the bucket class

     e. Select the **Enable replication** to obtain higher resiliency of the objects stored in the buckets.

     f. Click **Create using ObjectBucketClaim**.

   - **Create via S3 API**

     a. Enter a name for the bucket.

     b. Add tags with different key and value criteria to tag your bucket.

c. Click **Create**.

4.2.2. Listing bucket details using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then click the **Buckets** tab.
3. Select the required bucket and then click the **Details** tab.

4.2.3. Deleting or emptying bucket using object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW and click the **Buckets** tab.
3. From the bucket's options menu, select **Empty bucket**.
4. Enter the bucket name to confirm.
5. Click **Empty bucket**
6. After the bucket is empty, open the options menu again.
7. Select **Delete bucket**.
8. Enter the bucket name to confirm and click **Delete bucket**.

4.3. MANAGING OBJECTS

4.3.1. Listing objects and object details using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW and click the **Buckets** tab.
3. Select the required bucket and click the **Objects** tab.
4. Select the object to open the details in the sidebar.

4.3.2. Downloading or previewing objects using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the required bucket.
3. Click the **Objects** tab.
4. From the object's options menu, select **Download** or **Preview**.



NOTE

Preview behavior depends on browser support. Unsupported file types that cannot be displayed in a separate tab are downloaded automatically.

4.3.3. Generating the presigned URL to share an object using the object browser

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. Click the **Objects** tab.
4. From the object's options menu, select **Share with presigned URL**
5. Select the expiration time to set the validity period of the presigned URL.
6. Click **Create**, then copy the generated URL.

4.3.4. Uploading objects to the bucket using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. Click the **Objects** tab.
4. In the **Add objects** section, do one of the following:
   - Click **Upload** to add a folder.
   - Drag and drop individual files to upload specific files.
5. Click the uploaded object to view its **Overview** and **Versions** tab.

4.3.5. Creating folder using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.

3. Click the **Objects** tab.
4. Click **Create folder**.
5. Enter a folder name.
6. Click **Create**.



NOTE

The folder persists only after at least one object is uploaded inside it.

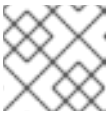
4.3.6. Deleting objects using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. Click the **Objects** tab.
4. To delete objects, do one of the following:
   - **Multiple objects**: Select multiple objects using the checkbox and then from the Actions menu, select **Delete**.
   - **Single object**: Use the object's Action menu and select **Delete**.



NOTE

Folders cannot be deleted directly. Empty folders disappear automatically.

4.3.7. Enabling or suspending object versioning in a bucket using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. Click the **Properties** tab.
4. Toggle **Versioning** to **Enabled** or **Disabled**.
5. Click **Enable** or **Suspend** to confirm depending on your versioning selection.



NOTE

Update lifecycle rules after enabling versioning.

4.3.8. Listing object versions using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. Click the **Objects** tab.
4. Toggle **List all versions**.
   - When **List all versions** is OFF:
     - The current version of the object excluding the delete markers is displayed.
     - Any Action menu options apply to the the current version.
     - To view all versions of one object:
       - Click the object → open the **Versions** tab from the sidebar.
   - When **List all versions** is ON:
     - All object versions appear including deleted objects that have delete markers.
     - Deleting a version is permanent.
     - Removing a delete marker restores the most recent version. If no previous version exists, the object is permanently deleted.

4.4. CREATING AND MANAGING LIFECYCLE RULES

4.4.1. Creating new lifecycle rules using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
   You can also create a new bucket using the steps from the Creating a new bucket using the MCG Object browser section.
3. Click the **Management** tab.
4. Click on **Create lifecycle rule**.
5. Enter a rule name.
6. Select a **Rule scope**:
   - **Targeted** (filters: prefix, tags, size)
   - **Global (Bucket-wide)**
7. Add one or more actions:
   - **Delete an object after a specified time**
     For a versioned bucket, the current version of the object is retained as a noncurrent version. For a nonversioned object, the object is deleted permanently.
   - **Noncurrent versions of the objects**
     This rule which applies to only the versioned objects deletes the older versions of the objects after they become noncurrent.
   - **Incomplete multipart uploads**
     This rule cleans up the abandoned uploads that were initiated but never completed after a specified period of time to prevent unnecessary storage costs. The object tags and size filter that were specified for the object with the targeted rule scope are not applicable for this rule and it deletes all multipart uploads from the bucket regardless of any filter.
   - **Expired object delete markers**
     This rule removes any unnecessary delete markers in versioned buckets that do not have any associated object versions, which might clutter bucket listings. The object tags and size filter that were specified for the object with the targeted rule scope are not applicable for this rule and it deletes all multipart uploads from the bucket regardless of any filter.



NOTE

- The object tags and object size filters are not applicable for this rule. The expired object delete markers are removed from the bucket regardless of any filters that are configured.
- This rule is not enabled when delete an object after a specified time rule is selected.

8. Click **Save**.

4.4.2. Editing lifecycle rules using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. In the bucket view, click the **Management** tab.
4. Locate the rule, then from the Actions menu select **Edit lifecycle rule**.
5. Modify fields and click **Save**.

4.4.3. Creating Cross Origin Resource Sharing (CORS) rule

Use this procedure to create a Cross Origin Resource Sharing (CORS) rule for multicloud object gateway buckets. .Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. In the bucket view, click the **Permission** tab and then click the **CORS** tab.
4. Click **Create CORS rule**.

5. Enter a rule name.
6. Configure allowed origins*.
7. Select **Allowed methods**.
8. Configure allowed headers.
9. Enter the maximum age of preflight request in seconds.
10. Click **Create**.

4.4.4. Editing Cross Origin Resource Sharing (CORS) rule

Use this procedure to edit existing Cross Origin Resource Sharing (CORS) rules for multicloud object gateway buckets. .Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. In the bucket view, click the **Permission** tab and then click the **CORS** tab.
4. Select the CORS rule and from the Actions menu, click **Edit configuration**.
5. Modify the fields and click **Save**.

4.4.5. Using bucket policy to grant public or restricted access to objects

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. In the bucket view, click the **Permission** tab.
4. In **Bucket policy**, do one of the following:

- If no policy exists, do one of the following:
  - Drag a file from another location or upload a file using the **Browse** button.
  - Start from scratch or **Use a predefined policy** configuration.
    a. Create a new policy or choose a policy from **Available policies**.
    b. Update the policy as required.
    c. Click **Apply policy**.
    d. Click **Confirm** to update the policy after reviewing the changes.
- If a policy exists, use a predefined policy configuration by selecting the **Edit bucket policy** from the Action menu.
  a. Select **Use a predefined policy** configuration.
  b. Choose a policy from **Available policies**.
  c. Update the policy as required.
  d. Click **Apply policy**.
  e. Click **Confirm** to update the policy after reviewing the changes.



NOTE

The bucket policy configuration may take up to 120 seconds to take effect after it is applied.

4.4.6. Setting up public access limit to S3 resources using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW, then open the bucket.
3. In the bucket view, click the **Permission** tab.
4. Click **Block public access → Manage public access settings**
5. Choose one or more:
   a. Block all public access.

b. Block public access to buckets and objects granted through new public bucket policies.

c. Block public and cross-account access to buckets and objects through any public bucket policies.

6. Click **Save changes**.

4.5. CREATING AND MANAGING IAM USER

Use this section to create and manage IAM users for object storage access. Note the following considerations when creating and managing an IAM user:

- **Bucket policy principals and ARNs:** Bucket policies can reference either the account ID/ARN (for administrator or NooBaa CLI-created accounts) or the IAM user ARN. On upgraded clusters, bucket-policy principals previously defined using the email-based account format (account@noobaa.io) are automatically converted to the ARN format (arn:aws:iam::account_id:root).
- **S3 access requires a policy:** An IAM user's S3 requests return **AccessDenied** if no IAM policy is attached.
- **OBC account limitations:** Object Bucket Claim (OBC) accounts cannot create IAM users.
- **Default policy review:** IAM users created through the IAM console are automatically assigned a default policy that grants full access.

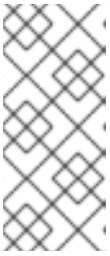
4.5.1. Creating an IAM user using the object browser

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW.
3. Click the **IAM** tab.
4. Click **Create User**.
5. Enter the required details in the **Create User** form.
   After the user is created, a window appears prompting you to download the secret key.
6. Click **Download secret key**, or copy the key and store it securely.



NOTE

- When a new IAM user is created, an inline policy permitting S3 operations is generated automatically.
- Access keys are also created automatically as part of the user creation workflow. No manual configuration is required.

4.5.2. Generating additional access key for an IAM user

You can generate one additional access key for the IAM user. IAM users can generate additional access key for themselves.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW.
3. Click the **IAM** tab.
4. Select the IAM user for whom you want to generate access key.
5. On the user details page, click the **Access keys** tab.
   The existing access key cards are displayed.
6. Click **Generate another access key**.
7. (Optional) Enter a description for the new access key.
8. Click **Generate**.
   The newly generated access key card is displayed.

4.5.3. Activating/Deactivating Access key

You can generate one additional access key

Prerequisites

- **Administrators**: Administrator access to OpenShift Data Foundation.
- **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser.
- Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy.



NOTE

Object Bucket Claim (OBC) based accounts do not have permissions to create IAM users.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW.

3. Click the **IAM** tab.
4. Select the IAM user for whom you want to activate or deactivate access keys.
5. On the user details page, click the **Access keys** tab.
   The existing access key cards are displayed.
6. Select the Actions menu corresponding to the required Access key and select **Activate/Deactivate Access key**.
7. Click **Activate** or **Deactivate** depending on the existing status.

4.5.4. Deleting access key for an IAM user

Prerequisites

- An existing IAM user with at least one access key.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Storage endpoint selector, choose MCG or RGW.
3. Click the **IAM** tab.
4. Select the IAM user for whom you want to delete the access key.
5. On the user details page, click the **Access keys** tab.
   The list of access key cards are displayed.
6. Locate the access key you want to delete and open the Action menu on its card.
7. Select **Delete access key**.
8. You must deactivate the access key before deleting it. Click **Deactivate**.
9. To confirm deletion, type **delete** in the confirmation field.
10. Click **Delete** to permanently remove the access key.

4.5.5. Deleting an IAM User

Prerequisites

Administrators: Administrator access to OpenShift Data Foundation. * **Developers**: Logged in to the object browser using secret namespace and secret name. For more information see Accessing the Object Browser. * Ensure that the selected storage endpoint (MCG or RGW) is deployed and its pods are healthy. * All access keys and inline user policies associated with the user must be removed before deletion.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage**.

2. From the Storage endpoint selector, choose MCG or RGW.
3. Click the **IAM** tab.
4. Select the IAM user you want to delete.
5. Click the Actions menu and select **Delete user**.
6. You must delete any existing access keys before deleting the user.
7. Type delete to confirm the user deletion.

CHAPTER 5. MANAGING NAMESPACE BUCKETS

Namespace buckets unify object storage across multiple cloud providers, letting you read from several sources and write to your preferred provider through a single bucket interface.

Namespace buckets let you connect data repositories on different providers together, so that you can interact with all of your data through a single unified view. Add the object bucket associated with each provider to the namespace bucket, and access your data through the namespace bucket to see all of your object buckets at once. This lets you write to your preferred storage provider while reading from multiple other storage providers, greatly reducing the cost of migrating to a new storage provider.



NOTE

A namespace bucket can only be used if its **write** target is available and functional.

5.1. AMAZON S3 API ENDPOINTS FOR OBJECTS IN NAMESPACE BUCKETS

You can interact with objects in the namespace buckets using the Amazon Simple Storage Service (S3) API.

Ensure that the credentials provided for the Multicloud Object Gateway (MCG) enables you to perform the AWS S3 namespace bucket operations. You can use the AWS tool, **aws-cli** to verify that all the operations can be performed on the target bucket. Also, the list bucket which is using this MCG account shows the target bucket.

Red Hat OpenShift Data Foundation supports the following namespace bucket operations:

- ListBuckets
- ListObjects
- ListMultipartUploads
- ListObjectVersions
- GetObject
- HeadObject
- CopyObject
- PutObject
- CreateMultipartUpload
- UploadPartCopy
- UploadPart
- ListParts
- AbortMultipartUpload
- PubObjectTagging

- DeleteObjectTagging
- GetObjectTagging
- GetObjectAcl
- PutObjectAcl
- DeleteObject
- DeleteObjects

See the Amazon S3 API reference documentation for the most up-to-date information about these operations and how to use them.

Additional resources

- Amazon S3 REST API Reference
- Amazon S3 CLI Reference

5.2. ADDING A NAMESPACE BUCKET USING THE MULTICLOUD OBJECT GATEWAY CLI AND YAML

For more information about namespace buckets, see Managing namespace buckets.

The supported namespacestore types are:

- **aws-s3**
- **aws s3-compatible**
- **azure-blob**
- **ibm-cos**
- **nsfs**

Use **aws-sts-s3** when AWS authentication is performed using STS or **azure-sts-blob** when authentication is performed using Microsoft Entra Workload ID (short-term credential). The **nsfs** type is documented separately later in this document.

Examples for configuring **aws-s3** and **ibm-cos** namespacestores are provided in the following sections. Depending on your deployment type and whether you prefer using YAML or the Multicloud Object Gateway (MCG) CLI, follow one of the procedures to add a namespace bucket.

- Adding an AWS S3 namespace bucket using YAML
- Adding an IBM COS namespace bucket using YAML
- Adding an AWS S3 namespace bucket using the Multicloud Object Gateway CLI
- Adding an IBM COS namespace bucket using the Multicloud Object Gateway CLI

5.2.1. Adding an AWS S3 namespace bucket using YAML

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG).
  For information, see Chapter 2, Accessing the Multicloud Object Gateway with your applications.

Procedure

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
  type: Opaque
data:
  AWS_ACCESS_KEY_ID: <AWS ACCESS KEY ID ENCODED IN BASE64>
  AWS_SECRET_ACCESS_KEY: <AWS SECRET ACCESS KEY ENCODED IN BASE64>
```

   where **<namespacestore-secret-name>** is a unique NamespaceStore name.

   You must provide and encode your own AWS access key ID and secret access key using **Base64**, and use the results in place of **<AWS ACCESS KEY ID ENCODED IN BASE64>** and **<AWS SECRET ACCESS KEY ENCODED IN BASE64>**.

2. Create a NamespaceStore resource using OpenShift custom resource definitions (CRDs).
   A NamespaceStore represents underlying storage to be used as a **read** or **write** target for the data in the MCG namespace buckets.

   To create a NamespaceStore resource, apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <resource-name>
  namespace: openshift-storage
spec:
  awsS3:
    secret:
      name: <namespacestore-secret-name>
      namespace: <namespace-secret>
    targetBucket: <target-bucket>
  type: aws-s3
```

   <resource-name>

   The name you want to give to the resource.

   <namespacestore-secret-name>

   The secret created in the previous step.

<namespace-secret>

The namespace where the secret can be found.

<target-bucket>

The target bucket you created for the NamespaceStore.

3. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy requires a type of either **single** or **multi**.

- A namespace policy of type **single** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Single
    single:
      resource: <resource>
```

<my-bucket-class>

The unique namespace bucket class name.

<resource>

The name of a single NamespaceStore that defines the read and write target of the namespace bucket.

- A namespace policy of type **multi** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1

kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Multi
    multi:
      writeResource: <write-resource>
      readResources:
      - <read-resources>
      - <read-resources>
```

<my-bucket-class>

A unique bucket class name.

<write-resource>

The name of a single NamespaceStore that defines the **write** target of the namespace bucket.

<read-resources>

A list of the names of the NamespaceStores that defines the **read** targets of the namespace bucket.

4. Create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the earlier step using the following YAML:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: <resource-name>
  namespace: openshift-storage
spec:
  generateBucketName: <my-bucket>
  storageClassName: openshift-storage.noobaa.io
  additionalConfig:
    bucketclass: <my-bucket-class>
```

<resource-name>

The name you want to give to the resource.

<my-bucket>

The name you want to give to the bucket.

<my-bucket-class>

The bucket class created in the previous step.
After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and **ConfigMap** with the same name and in the same namespace as that of the OBC.

5.2.2. Adding an IBM COS namespace bucket using YAML

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG), see Chapter 2, Accessing the Multicloud Object Gateway with your applications.

Procedure

1. Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
  type: Opaque
data:
```

```
  IBM_COS_ACCESS_KEY_ID: <IBM COS ACCESS KEY ID ENCODED IN BASE64>
  IBM_COS_SECRET_ACCESS_KEY: <IBM COS SECRET ACCESS KEY ENCODED IN
BASE64>
```

<namespacestore-secret-name>

A unique NamespaceStore name.
You must provide and encode your own IBM COS access key ID and secret access key using **Base64**, and use the results in place of **<IBM COS ACCESS KEY ID ENCODED IN BASE64>** and **<IBM COS SECRET ACCESS KEY ENCODED IN BASE64>**.

2. Create a NamespaceStore resource using OpenShift custom resource definitions (CRDs). A NamespaceStore represents underlying storage to be used as a **read** or **write** target for the data in the MCG namespace buckets.

   To create a NamespaceStore resource, apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: bs
  namespace: openshift-storage
spec:
  s3Compatible:
    endpoint: <IBM COS ENDPOINT>
    secret:
      name: <namespacestore-secret-name>
      namespace: <namespace-secret>
    signatureVersion: v2
    targetBucket: <target-bucket>
  type: ibm-cos
```

<IBM COS ENDPOINT>

The appropriate IBM COS endpoint.

<namespacestore-secret-name>

The secret created in the previous step.

<namespace-secret>

The namespace where the secret can be found.

<target-bucket>

The target bucket you created for the NamespaceStore.

3. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy requires a type of either **single** or **multi**.

   - The namespace policy of type **single** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
```

```
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Single
    single:
      resource: <resource>
```

<my-bucket-class>

The unique namespace bucket class name.

<resource>

The name of a single NamespaceStore that defines the **read** and **write** target of the namespace bucket.

- The namespace policy of type **multi** requires the following configuration:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <my-bucket-class>
  namespace: openshift-storage
spec:
  namespacePolicy:
    type: Multi
    multi:
      writeResource: <write-resource>
      readResources:
      - <read-resources>
      - <read-resources>
```

<my-bucket-class>

The unique bucket class name.

<write-resource>

The name of a single NamespaceStore that defines the write target of the namespace bucket.

<read-resources>

A list of the NamespaceStores names that defines the **read** targets of the namespace bucket.

4. To create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the previous step, apply the following YAML:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: <resource-name>
  namespace: openshift-storage
spec:
```

```
generateBucketName: <my-bucket>
storageClassName: openshift-storage.noobaa.io
additionalConfig:
  bucketclass: <my-bucket-class>
```

<resource-name>

The name you want to give to the resource.

<my-bucket>

The name you want to give to the bucket.

<my-bucket-class>

The bucket class created in the previous step.
After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and **ConfigMap** with the same name and in the same namespace as that of the OBC.

5.2.3. Adding an AWS S3 namespace bucket using the Multicloud Object Gateway CLI

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG), see Chapter 2, Accessing the Multicloud Object Gateway with your applications.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS

Procedure

1. In the command-line interface, create a NamespaceStore resource.
   A NamespaceStore represents an underlying storage to be used as a **read** or **write** target for the data in MCG namespace buckets.

```
$ odf noobaa namespacestore create aws-s3 <namespacestore> --access-key <AWS ACCESS KEY> --secret-key <AWS SECRET ACCESS KEY> --target-bucket <bucket-name> -n openshift-storage
```

<namespacestore>

The name of the NamespaceStore.

<AWS ACCESS KEY> and <AWS SECRET ACCESS KEY>

The AWS access key ID and secret access key you created for this purpose.

<bucket-name>

The existing AWS bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

2. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy can be either **single** or **multi**.

   - To create a namespace bucket class with a namespace policy of type **single**:

     ```
     $ odf noobaa bucketclass create namespace-bucketclass single <my-bucket-class> --resource <resource> -n openshift-storage
     ```

     <resource-name>

     The name you want to give the resource.

     <my-bucket-class>

     A unique bucket class name.

     <resource>

     A single namespace-store that defines the **read** and **write** target of the namespace bucket.

   - To create a namespace bucket class with a namespace policy of type **multi**:

     ```
     $ odf noobaa bucketclass create namespace-bucketclass multi <my-bucket-class> --write-resource <write-resource> --read-resources <read-resources> -n openshift-storage
     ```

     <resource-name>

     The name you want to give the resource.

     <my-bucket-class>

     A unique bucket class name.

     <write-resource>

     A single namespace-store that defines the **write** target of the namespace bucket.

     <read-resources>s

     A list of namespace-stores separated by commas that defines the **read** targets of the namespace bucket.

3. Create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the previous step.

   ```
   $ odf noobaa obc create my-bucket-claim -n openshift-storage --app-namespace my-app --bucketclass <custom-bucket-class>
   ```

   <bucket-name>

   A bucket name of your choice.

   <custom-bucket-class>

   The name of the bucket class created in the previous step.

   After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and a **ConfigMap** with the same name and in the same namespace as that of the OBC.

5.2.4. Adding an IBM COS namespace bucket using the Multicloud Object Gateway CLI

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG), see Chapter 2, Accessing the Multicloud Object Gateway with your applications.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose either Linux(x86_64), Windows, or Mac OS.

Procedure

1. In the command-line interface, create a NamespaceStore resource.
   A NamespaceStore represents an underlying storage to be used as a **read** or **write** target for the data in the MCG namespace buckets.

```
$ odf noobaa namespacestore create ibm-cos <namespacestore> --endpoint <IBM COS ENDPOINT> --access-key <IBM ACCESS KEY> --secret-key <IBM SECRET ACCESS KEY> --target-bucket <bucket-name> -n openshift-storage
```

   <namespacestore>

   The name of the NamespaceStore.

   <IBM ACCESS KEY>, <IBM SECRET ACCESS KEY>, <IBM COS ENDPOINT>

   An IBM access key ID, secret access key, and the appropriate regional endpoint that corresponds to the location of the existing IBM bucket.

   <bucket-name>

   An existing IBM bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.

2. Create a namespace bucket class that defines a namespace policy for the namespace buckets. The namespace policy requires a type of either **single** or **multi**.

   - To create a namespace bucket class with a namespace policy of type **single**:

```
$ odf noobaa bucketclass create namespace-bucketclass single <my-bucket-class> --resource <resource> -n openshift-storage
```

     <resource-name>

     The name you want to give the resource.

     <my-bucket-class>

     A unique bucket class name.

     <resource>

A single NamespaceStore that defines the **read** and **write** target of the namespace bucket.

- To create a namespace bucket class with a namespace policy of type **multi**:

```
$ odf noobaa bucketclass create namespace-bucketclass multi <my-bucket-class> --write-resource <write-resource> --read-resources <read-resources> -n openshift-storage
```

<resource-name>

The name you want to give the resource.

<my-bucket-class>

A unique bucket class name.

<write-resource>

A single NamespaceStore that defines the **write** target of the namespace bucket.

<read-resources>

A comma-separated list of NamespaceStores that defines the **read** targets of the namespace bucket.

3. Create a bucket using an Object Bucket Class (OBC) resource that uses the bucket class defined in the earlier step.

```
$ odf noobaa obc create my-bucket-claim -n openshift-storage --app-namespace my-app --bucketclass <custom-bucket-class>
```

<bucket-name>

A bucket name of your choice.

<custom-bucket-class>

The name of the bucket class created in the previous step.
After the OBC is provisioned by the operator, a bucket is created in the MCG, and the operator creates a **Secret** and **ConfigMap** with the same name and in the same namespace as that of the OBC.

5.3. ADDING A NAMESPACE BUCKET USING THE OPENSHIFT CONTAINER PLATFORM USER INTERFACE

You can add namespace buckets using the OpenShift Container Platform user interface. For information about namespace buckets, see Managing namespace buckets.

Prerequisites

- Ensure that Openshift Container Platform with OpenShift Data Foundation operator is already installed.
- Access to the Multicloud Object Gateway (MCG).

Procedure

1. On the OpenShift Web Console, navigate to **Storage → Object Storage → Namespace Store** tab.

2. Click **Create namespace store** to create a **namespacestore** resources to be used in the namespace bucket.
    a. Enter a **namespacestore** name.
    b. Choose a provider and region.
    c. Either select an existing secret, or click **Switch to credentials** to create a secret by entering a secret key and secret access key.
    d. Enter a target bucket.
    e. Click **Create**.
3. On the Namespace Store tab, verify that the newly created **namespacestore** is in the **Ready** state.
4. Repeat steps 2 and 3 until you have created all the desired amount of resources.
5. Navigate to **Bucket Class** tab and click **Create Bucket Class**.
    a. Choose **Namespace BucketClass type** radio button.
    b. Enter a **BucketClass name** and click **Next**.
    c. Choose a **Namespace Policy Type** for your namespace bucket, and then click **Next**.
        - If your namespace policy type is **Single**, you need to choose a read resource.
        - If your namespace policy type is **Multi**, you need to choose read resources and a write resource.
        - If your namespace policy type is **Cache**, you need to choose a Hub namespace store that defines the read and write target of the namespace bucket.
    d. Select one **Read and Write NamespaceStore** which defines the read and write targets of the namespace bucket and click **Next**.
    e. Review your new bucket class details, and then click **Create Bucket Class**.
6. Navigate to **Bucket Class** tab and verify that your newly created resource is in the **Ready** phase.
7. Navigate to **Object Bucket Claims** tab and click **Create Object Bucket Claim**.
    a. Enter **ObjectBucketClaim Name** for the namespace bucket.
    b. Select **StorageClass** as **openshift-storage.noobaa.io**.
    c. Select the **BucketClass** that you created earlier for your **namespacestore** from the list. By default, **noobaa-default-bucket-class** gets selected.
    d. Click **Create**. The namespace bucket is created along with Object Bucket Claim for your namespace.
8. Navigate to **Object Bucket Claims** tab and verify that the Object Bucket Claim created is in **Bound** state.

9. Navigate to **Object Buckets** tab and verify that the your namespace bucket is present in the list and is in **Bound** state.

5.4. SHARING LEGACY APPLICATION DATA WITH CLOUD NATIVE APPLICATION USING S3 PROTOCOL

Many legacy applications use file systems to share data sets. You can access and share the legacy data in the file system by using the S3 operations. To share data you need to do the following:

- Export the pre-existing file system datasets, that is, RWX volume such as Ceph FileSystem (CephFS) or create a new file system datasets using the S3 protocol.
- Access file system datasets from both file system and S3 protocol.
- Configure S3 accounts and map them to the existing or a new file system unique identifiers (UIDs) and group identifiers (GIDs).

5.4.1. Creating a NamespaceStore to use a file system

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to the Multicloud Object Gateway (MCG).

Procedure

1. Log into the OpenShift Web Console.
2. Click **Storage → Object Storage**.
3. Click the **NamespaceStore** tab to create NamespaceStore resources to be used in the namespace bucket.
4. Click **Create namespacestore**.
5. Enter a name for the NamespaceStore.
6. Choose **Filesystem** as the provider.
7. Choose the Persistent volume claim.
8. Enter a folder name.
   If the folder name exists, then that folder is used to create the NamespaceStore or else a folder with that name is created.
9. Click **Create**.
10. Verify the NamespaceStore is in the Ready state.

5.4.2. Creating accounts with NamespaceStore filesystem configuration

You can either create a new account with NamespaceStore filesystem configuration or convert an existing normal account into a NamespaceStore filesystem account by editing the YAML.



NOTE

You cannot remove a NamespaceStore filesystem configuration from an account.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- Create a new account with NamespaceStore filesystem configuration using the command-line interface.

```
$ odf noobaa account create <noobaa-account-name> [flags]
```

For example:

```
$ odf noobaa account create testaccount --nsfs_account_config --gid 10001 --uid 10001 –
default_resource fs_namespacestore
```

allow_bucket_create	Indicates whether the account is allowed to create new buckets. Supported values are **true** or **false**. Default value is **true**.
default_resource	The NamespaceStore resource on which the new buckets will be created when using the S3 CreateBucket operation. The NamespaceStore must be backed by an RWX (ReadWriteMany) persistent volume claim (PVC).
force_md5_etag	Enables md5 etag calculation for the user account.
new_buckets_path	The filesystem path where directories corresponding to new buckets will be created. The path is inside the filesystem of NamespaceStore filesystem PVCs where new directories are created to act as the filesystem mapping of newly created object bucket classes.
nsfs_account_config	A mandatory field that indicates if the account is used for NamespaceStore filesystem.
nsfs_only	Indicates whether the account is used only for NamespaceStore filesystem or not. Supported values are true or **false**. Default value is **false**. If it is set to 'true', it limits you from accessing other types of buckets.
uid	The user ID of the filesystem to which the MCG account will be mapped and it is used to access and manage data on the filesystem

gid	The group ID of the filesystem to which the MCG account will be mapped and it is used to access and manage data on the filesystem

The MCG system sends a response with the account configuration and its S3 credentials:

```
# NooBaaAccount spec:
allow_bucket_creation: true
default_resource: noobaa-default-namespace-store
Nsfs_account_config:
  gid: 10001
  new_buckets_path: /
  nsfs_only: true
  uid: 10001
INFO[0006]   Exists: Secret "noobaa-account-testaccount"
Connection info:
  AWS_ACCESS_KEY_ID      : <aws-access-key-id>
  AWS_SECRET_ACCESS_KEY  : <aws-secret-access-key>
```

You can list all the custom resource definition (CRD) based accounts by using the following command:

```
$ odf noobaa account list
NAME          ALLOWED_BUCKETS   DEFAULT_RESOURCE             PHASE   AGE
testaccount   [*]               noobaa-default-backing-store   Ready   1m17s
```

If you are interested in a particular account, you can read its custom resource definition (CRD) directly by the account name:

```
$ oc get noobaaaccount/testaccount -o yaml
spec:
  allow_bucket_creation: true
  default_resource: noobaa-default-namespace-store
  nsfs_account_config:
    gid: 10001
    new_buckets_path: /
    nsfs_only: true
    uid: 10001
```

5.4.3. Accessing legacy application data from the openshift-storage namespace

When using the Multicloud Object Gateway (MCG) NamespaceStore filesystem (NSFS) feature, you need to have the Persistent Volume Claim (PVC) where the data resides in the **openshift-storage** namespace. In almost all cases, the data you need to access is not in the **openshift-storage** namespace, but in the namespace that the legacy application uses.



NOTE

When using a non-CephFS storage provider for NSFS PVC, the PVC storage provider must support extended attributes (XATTR).

In order to access data stored in another namespace, you need to create a PVC in the **openshift-storage** namespace that points to the same CephFS volume that the legacy application uses.

Procedure

1. Display the application namespace with **scc**:

```
$ oc get ns <application_namespace> -o yaml | grep scc
```

<application_namespace>

Specify the name of the application namespace.
For example:

```
$ oc get ns testnamespace -o yaml | grep scc

openshift.io/sa.scc.mcs: s0:c26,c5
openshift.io/sa.scc.supplemental-groups: 1000660000/10000
openshift.io/sa.scc.uid-range: 1000660000/10000
```

2. Navigate into the application namespace:

```
$ oc project <application_namespace>
```

For example:

```
$ oc project testnamespace
```

3. Ensure that a ReadWriteMany (RWX) PVC is mounted on the pod that you want to consume from the noobaa S3 endpoint using the MCG NSFS feature:

```
$ oc get pvc

NAME                                          STATUS VOLUME
CAPACITY ACCESS MODES STORAGECLASS              AGE
cephfs-write-workload-generator-no-cache-pv-claim  Bound  pvc-aa58fb91-c3d2-475b-bbee-
68452a613e1a
10Gi     RWX          ocs-storagecluster-cephfs 12s
```

```
$ oc get pod

NAME                                          READY   STATUS            RESTARTS   AGE
cephfs-write-workload-generator-no-cache-1-cv892    1/1     Running           0          11s
```

4. Check the mount point of the Persistent Volume (PV) inside your pod.

   a. Get the volume name of the PV from the pod:

```
$ oc get pods <pod_name> -o jsonpath='{.spec.volumes[]}'
```

<pod_name>

Specify the name of the pod.
For example:

```
$ oc get pods cephfs-write-workload-generator-no-cache-1-cv892 -o
jsonpath='{.spec.volumes[]}'
```

```
{"name":"app-persistent-storage","persistentVolumeClaim":{"claimName":"cephfs-write-workload-generator-no-cache-pv-claim"}}
```

In this example, the name of the volume for the PVC is **cephfs-write-workload-generator-no-cache-pv-claim**.

b. List all the mounts in the pod, and check for the mount point of the volume that you identified in the previous step:

```
$ oc get pods <pod_name> -o jsonpath='{.spec.containers[].volumeMounts}'
```

For example:

```
$ oc get pods cephfs-write-workload-generator-no-cache-1-cv892 -o jsonpath='{.spec.containers[].volumeMounts}'

[{"mountPath":"/mnt/pv","name":"app-persistent-storage"},{"mountPath":"/var/run/secrets/kubernetes.io/serviceaccount","name":"kube-api-access-8tnc5","readOnly":true}]
```

5. Confirm the mount point of the RWX PV in your pod:

```
$ oc exec -it <pod_name> -- df <mount_path>
```

<mount_path>

Specify the path to the mount point that you identified in the previous step.
For example:

```
$ oc exec -it cephfs-write-workload-generator-no-cache-1-cv892 -- df /mnt/pv

main
Filesystem
1K-blocks Used Available  Use%  Mounted on
172.30.202.87:6789,172.30.120.254:6789,172.30.77.247:6789:/volumes/csi/csi-vol-cc416d9e-dbf3-11ec-b286-0a580a810213/edcfe4d5-bdcb-4b8e-8824-8a03ad94d67c
10485760  0    10485760   0%    /mnt/pv
```

6. Ensure that the UID and SELinux labels are the same as the ones that the legacy namespace uses:

```
$ oc exec -it <pod_name> -- ls -latrZ <mount_path>
```

For example:

```
$ oc exec -it cephfs-write-workload-generator-no-cache-1-cv892 -- ls -latrZ /mnt/pv/

total 567
drwxrwxrwx. 3 root       root system_u:object_r:container_file_t:s0:c26,c5      2 May 25 06:35 .
-rw-r--r--. 1 1000660000 root system_u:object_r:container_file_t:s0:c26,c5 580138 May 25
```

```
06:35 fs_write_cephfs-write-workload-generator-no-cache-1-cv892-data.log
drwxrwxrwx. 3 root       root system_u:object_r:container_file_t:s0:c26,c5     30 May 25 06:35
..
```

7. Get the information of the legacy application RWX PV that you want to make accessible from the **openshift-storage** namespace:

```
$ oc get pv | grep <pv_name>
```

<pv_name>

Specify the name of the PV.
For example:

```
$ oc get pv | grep pvc-aa58fb91-c3d2-475b-bbee-68452a613e1a

pvc-aa58fb91-c3d2-475b-bbee-68452a613e1a   10Gi       RWX            Delete
Bound    testnamespace/cephfs-write-workload-generator-no-cache-pv-claim    ocs-
storagecluster-cephfs                 47s
```

8. Ensure that the PVC from the legacy application is accessible from the **openshift-storage** namespace so that one or more noobaa-endpoint pods can access the PVC.

   a. Find the values of the **subvolumePath** and **volumeHandle** from the **volumeAttributes**. You can get these values from the YAML description of the legacy application PV:

```
$ oc get pv <pv_name> -o yaml
```

   For example:

```
$ oc get pv pvc-aa58fb91-c3d2-475b-bbee-68452a613e1a -o yaml

apiVersion: v1
kind: PersistentVolume
metadata:
  annotations:
    pv.kubernetes.io/provisioned-by: openshift-storage.cephfs.csi.ceph.com
  creationTimestamp: "2022-05-25T06:27:49Z"
  finalizers:
  - kubernetes.io/pv-protection
  name: pvc-aa58fb91-c3d2-475b-bbee-68452a613e1a
  resourceVersion: "177458"
  uid: 683fa87b-5192-4ccf-af2f-68c6bcf8f500
spec:
  accessModes:
  - ReadWriteMany
  capacity:
    storage: 10Gi
  claimRef:
    apiVersion: v1
    kind: PersistentVolumeClaim
    name: cephfs-write-workload-generator-no-cache-pv-claim
    namespace: testnamespace
    resourceVersion: "177453"
```

```
    uid: aa58fb91-c3d2-475b-bbee-68452a613e1a
  csi:
    controllerExpandSecretRef:
      name: rook-csi-cephfs-provisioner
      namespace: openshift-storage
    driver: openshift-storage.cephfs.csi.ceph.com
    nodeStageSecretRef:
      name: rook-csi-cephfs-node
      namespace: openshift-storage
    *volumeAttributes:*
      clusterID: openshift-storage
      fsName: ocs-storagecluster-cephfilesystem
      storage.kubernetes.io/csiProvisionerIdentity: 1653458225664-8081-openshift-
storage.cephfs.csi.ceph.com
      subvolumeName: csi-vol-cc416d9e-dbf3-11ec-b286-0a580a810213
      *subvolumePath: /volumes/csi/csi-vol-cc416d9e-dbf3-11ec-b286-
0a580a810213/edcfe4d5-bdcb-4b8e-8824-8a03ad94d67c*
    *volumeHandle: 0001-0011-openshift-storage-0000000000000001-cc416d9e-dbf3-
11ec-b286-0a580a810213*
  persistentVolumeReclaimPolicy: Delete
  storageClassName: ocs-storagecluster-cephfs
  volumeMode: Filesystem
status:
  phase: Bound
```

b. Use the **subvolumePath** and **volumeHandle** values that you identified in the previous step to create a new PV and PVC object in the **openshift-storage** namespace that points to the same CephFS volume as the legacy application PV:
Example YAML file:

```
$ cat << EOF >> pv-openshift-storage.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: cephfs-pv-legacy-openshift-storage
spec:
  storageClassName: ""
  accessModes:
  - ReadWriteMany
  capacity:
    *storage: 10Gi*      1
  csi:
    driver: openshift-storage.cephfs.csi.ceph.com
    nodeStageSecretRef:
      name: rook-csi-cephfs-node
      namespace: openshift-storage
    volumeAttributes:
    # Volume Attributes can be copied from the Source testnamespace PV
      "clusterID": "openshift-storage"
      "fsName": "ocs-storagecluster-cephfilesystem"
      "staticVolume": "true"
    # rootpath is the subvolumePath: you copied from the Source testnamespace PV
      "rootPath": /volumes/csi/csi-vol-cc416d9e-dbf3-11ec-b286-0a580a810213/edcfe4d5-
bdcb-4b8e-8824-8a03ad94d67c
    *volumeHandle: 0001-0011-openshift-storage-0000000000000001-cc416d9e-dbf3-
```

```
11ec-b286-0a580a810213-clone*  2
  persistentVolumeReclaimPolicy: Retain
  volumeMode: Filesystem
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: cephfs-pvc-legacy
  namespace: openshift-storage
spec:
  storageClassName: ""
  accessModes:
  - ReadWriteMany
  resources:
    requests:
      *storage: 10Gi*  3
  volumeMode: Filesystem
  # volumeName should be same as PV name
  volumeName: cephfs-pv-legacy-openshift-storage
EOF
```

- **<1>** The storage capacity of the PV that you are creating in the **openshift-storage** namespace must be the same as the original PV.
- **<2>** The volume handle for the target PV that you create in **openshift-storage** needs to have a different handle than the original application PV, for example, add **-clone** at the end of the volume handle.
- **<3>** The storage capacity of the PVC that you are creating in the **openshift-storage** namespace must be the same as the original PVC.

c. Create the PV and PVC in the **openshift-storage** namespace using the YAML file specified in the previous step:

```
$ oc create -f _<YAML_file>_
```

<YAML_file>

Specify the name of the YAML file.
For example:

```
$ oc create -f pv-openshift-storage.yaml

persistentvolume/cephfs-pv-legacy-openshift-storage created
persistentvolumeclaim/cephfs-pvc-legacy created
```

d. Ensure that the PVC is available in the **openshift-storage** namespace:

```
$ oc get pvc -n openshift-storage

NAME                          STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS                AGE
cephfs-pvc-legacy             Bound    cephfs-pv-legacy-openshift-storage         10Gi
RWX                                    14s
```

e. Navigate into the **openshift-storage** project:

```
$ oc project openshift-storage

Now using project "openshift-storage" on server "https://api.cluster-
5f6ng.5f6ng.sandbox65.opentlc.com:6443".
```

f. Create the NSFS namespacestore:

```
$ odf noobaa namespacestore create nsfs <nsfs_namespacestore> --pvc-
name='<cephfs_pvc_name>' --fs-backend='CEPH_FS'
```

<nsfs_namespacestore>

Specify the name of the NSFS namespacestore.

<cephfs_pvc_name>

Specify the name of the CephFS PVC in the **openshift-storage** namespace.
For example:

```
$ odf noobaa namespacestore create nsfs legacy-namespace --pvc-name='cephfs-
pvc-legacy' --fs-backend='CEPH_FS'
```

g. Ensure that the noobaa-endpoint pod restarts and that it successfully mounts the PVC at the NSFS namespacestore, for example, **/nsfs/legacy-namespace** mountpoint:

```
$ oc exec -it <noobaa_endpoint_pod_name> -- df -h /nsfs/<nsfs_namespacestore>
```

<noobaa_endpoint_pod_name>

Specify the name of the noobaa-endpoint pod.
For example:

```
$ oc exec -it noobaa-endpoint-5875f467f5-546c6 -- df -h /nsfs/legacy-namespace

Filesystem
Size  Used Avail Use% Mounted on
172.30.202.87:6789,172.30.120.254:6789,172.30.77.247:6789:/volumes/csi/csi-vol-
cc416d9e-dbf3-11ec-b286-0a580a810213/edcfe4d5-bdcb-4b8e-8824-8a03ad94d67c
10G     0   10G   0% /nsfs/legacy-namespace
```

h. Create a MCG user account:

```
$ odf noobaa account create <user_account> --allow_bucket_create=true --
new_buckets_path='/' --nsfs_only=true --nsfs_account_config=true --gid <gid_number> -
-uid <uid_number> --default_resource='legacy-namespace'
```

<user_account>

Specify the name of the MCG user account.

<gid_number>

Specify the NSFS GID.

<uid_number>

Specify the NSFS UID.



IMPORTANT

Use the same **UID** and **GID** as that of the legacy application. You can find it from the previous output.

For example:

```
$ odf noobaa account create leguser --allow_bucket_create=true --
new_buckets_path='/' --nsfs_only=true --nsfs_account_config=true --gid 0 --uid
1000660000 --default_resource='legacy-namespace'
```

i. Create a MCG bucket.

i. Create a dedicated folder for S3 inside the NSFS share on the CephFS PV and PVC of the legacy application pod:

```
$ oc exec -it <pod_name> -- mkdir <mount_path>/nsfs
```

For example:

```
$ oc exec -it cephfs-write-workload-generator-no-cache-1-cv892 -- mkdir
/mnt/pv/nsfs
```

ii. Create the MCG bucket using the **nsfs/** path:

```
$ odf noobaa api bucket_api create_bucket '{
  "name": "<bucket_name>",
  "namespace":{
    "write_resource": { "resource": "<nsfs_namespacestore>", "path": "nsfs/" },
    "read_resources": [ { "resource": "<nsfs_namespacestore>", "path": "nsfs/" }]
  }
}'
```

For example:

```
$ odf noobaa api bucket_api create_bucket '{
  "name": "legacy-bucket",
  "namespace":{
    "write_resource": { "resource": "legacy-namespace", "path": "nsfs/" },
    "read_resources": [ { "resource": "legacy-namespace", "path": "nsfs/" }]
  }
}'
```

j. Check the SELinux labels of the folders residing in the PVCs in the legacy application and **openshift-storage** namespaces:

```
$ oc exec -it <noobaa_endpoint_pod_name> -n openshift-storage -- ls -ltraZ
/nsfs/<nsfs_namespacstore>
```

For example:

```
$ oc exec -it noobaa-endpoint-5875f467f5-546c6 -n openshift-storage -- ls -ltraZ
/nsfs/legacy-namespace

total 567
drwxrwxrwx. 3 root       root system_u:object_r:container_file_t:s0:c0,c26      2 May 25
06:35 .
-rw-r--r--. 1 1000660000 root system_u:object_r:container_file_t:s0:c0,c26 580138 May
25 06:35 fs_write_cephfs-write-workload-generator-no-cache-1-cv892-data.log
drwxrwxrwx. 3 root       root system_u:object_r:container_file_t:s0:c0,c26     30 May 25
06:35 ..
```

```
$ oc exec -it _<pod_name>_ -- ls -latrZ _<mount_path>_
```

For example:

```
$ oc exec -it cephfs-write-workload-generator-no-cache-1-cv892 -- ls -latrZ /mnt/pv/

total 567
drwxrwxrwx. 3 root       root system_u:object_r:container_file_t:s0:c26,c5      2 May 25
06:35 .
-rw-r--r--. 1 1000660000 root system_u:object_r:container_file_t:s0:c26,c5 580138 May
25 06:35 fs_write_cephfs-write-workload-generator-no-cache-1-cv892-data.log
drwxrwxrwx. 3 root       root system_u:object_r:container_file_t:s0:c26,c5     30 May 25
06:35 ..
```

In these examples, you can see that the SELinux labels are not the same which results in permission denied or access issues.

9. Ensure that the legacy application and **openshift-storage** pods use the same SELinux labels on the files.
   You can do this in one of the following ways:

   - Section 5.4.3.1, “Changing the default SELinux label on the legacy application project to match the one in the openshift-storage project”.
   - Section 5.4.3.2, “Modifying the SELinux label only for the deployment config that has the pod which mounts the legacy application PVC”.

10. Delete the NSFS namespacestore:

    a. Delete the MCG bucket:

    ```
    $ odf noobaa bucket delete <bucket_name>
    ```

    For example:

    ```
    $ odf noobaa bucket delete legacy-bucket
    ```

    b. Delete the MCG user account:

    ```
    $ odf noobaa account delete <user_account>
    ```

    For example:

```
$ odf noobaa account delete leguser
```

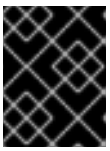
c. Delete the NSFS namespacestore:

```
$ odf noobaa namespacestore delete <nsfs_namespacestore>
```

For example:

```
$ odf noobaa namespacestore delete legacy-namespace
```

11. Delete the PV and PVC:



IMPORTANT

Before you delete the PV and PVC, ensure that the PV has a retain policy configured.

```
$ oc delete pv <cephfs_pv_name>
```

```
$ oc delete pvc <cephfs_pvc_name>
```

<cephfs_pv_name>

Specify the CephFS PV name of the legacy application.

<cephfs_pvc_name>

Specify the CephFS PVC name of the legacy application.
For example:

```
$ oc delete pv cephfs-pv-legacy-openshift-storage
```

```
$ oc delete pvc cephfs-pvc-legacy
```

5.4.3.1. Changing the default SELinux label on the legacy application project to match the one in the openshift-storage project

Use this procedure to change the default SELinux label on the legacy application project to match the one used in the **openshift-storage** project.

Procedure

1. Display the current **openshift-storage** namespace with **sa.scc.mcs**:

```
$ oc get ns openshift-storage -o yaml | grep sa.scc.mcs

openshift.io/sa.scc.mcs: s0:c26,c0
```

2. Edit the legacy application namespace, and modify the **sa.scc.mcs** with the value from the **sa.scc.mcs** of the **openshift-storage** namespace:

```
$ oc edit ns _<appplication_namespace>_
```

For example:

```
$ oc edit ns testnamespace
```

```
$ oc get ns _<application_namespace>_ -o yaml | grep sa.scc.mcs
```

For example:

```
$ oc get ns testnamespace -o yaml | grep sa.scc.mcs

openshift.io/sa.scc.mcs: s0:c26,c0
```

3. Restart the legacy application pod. A relabel of all the files take place and now the SELinux labels match with the **openshift-storage** deployment.

5.4.3.2. Modifying the SELinux label only for the deployment config that has the pod which mounts the legacy application PVC

Use this procedure to modify the SELinux label only for the deployment config that has the pod which mounts the legacy application PVC.

Procedure

1. Create a new **scc** with the **MustRunAs** and **seLinuxOptions** options, with the Multi Category Security (MCS) that the **openshift-storage** project uses.
   Example YAML file:

```
$ cat << EOF >> scc.yaml
allowHostDirVolumePlugin: false
allowHostIPC: false
allowHostNetwork: false
allowHostPID: false
allowHostPorts: false
allowPrivilegeEscalation: true
allowPrivilegedContainer: false
allowedCapabilities: null
apiVersion: security.openshift.io/v1
defaultAddCapabilities: null
fsGroup:
  type: MustRunAs
groups:
- system:authenticated
kind: SecurityContextConstraints
metadata:
  annotations:
  name: restricted-pvselinux
priority: null
readOnlyRootFilesystem: false
requiredDropCapabilities:
- KILL
- MKNOD
```

```
- SETUID
- SETGID
runAsUser:
  type: MustRunAsRange
seLinuxContext:
  seLinuxOptions:
    level: s0:c26,c0
  type: MustRunAs
supplementalGroups:
  type: RunAsAny
users: []
volumes:
- configMap
- downwardAPI
- emptyDir
- persistentVolumeClaim
- projected
- secret
EOF
```

```
$ oc create -f scc.yaml
```

2. Create a service account for the deployment and add it to the newly created **scc**.

   a. Create a service account:

   ```
   $ oc create serviceaccount _<service_account_name>_
   ```

   <service_account_name>`

   Specify the name of the service account.
   For example:

   ```
   $ oc create serviceaccount testnamespacesa
   ```

   b. Add the service account to the newly created **scc**:

   ```
   $ oc adm policy add-scc-to-user restricted-pvselinux -z <service_account_name>
   ```

   For example:

   ```
   $ oc adm policy add-scc-to-user restricted-pvselinux -z testnamespacesa
   ```

3. Patch the legacy application deployment so that it uses the newly created service account. This allows you to specify the SELinux label in the deployment:

   ```
   $ oc patch dc/<pod_name> '{"spec":{"template":{"spec":{"serviceAccountName":
   "<service_account_name>"}}}}'
   ```

   For example:

   ```
   $ oc patch dc/cephfs-write-workload-generator-no-cache --patch '{"spec":{"template":{"spec":
   {"serviceAccountName": "testnamespacesa"}}}}'
   ```

4. Edit the deployment to specify the security context to use at the SELinux label in the deployment configuration:

```
$ oc edit dc <pod_name> -n <application_namespace>
```

Add the following lines:

```
spec:
 template:
   metadata:
     securityContext:
       seLinuxOptions:
         Level: <security_context_value>
```

<security_context_value>

You can find this value when you execute the command to create a dedicated folder for S3 inside the NSFS share, on the CephFS PV and PVC of the legacy application pod.
For example:

```
$ oc edit dc cephfs-write-workload-generator-no-cache -n testnamespace
```

```
spec:
 template:
   metadata:
     securityContext:
       seLinuxOptions:
         level: s0:c26,c0
```

1. Ensure that the security context to be used at the SELinux label in the deployment configuration is specified correctly:

```
$ oc get dc <pod_name> -n <application_namespace> -o yaml | grep -A 2 securityContext
```

For example"

```
$ oc get dc cephfs-write-workload-generator-no-cache -n testnamespace -o yaml | grep -A 2
securityContext

      securityContext:
        seLinuxOptions:
          level: s0:c26,c0
```

The legacy application is restarted and begins using the same SELinux labels as the **openshift-storage** namespace.

5.5. CONFIGURING OR MODIFYING THE PUBLICACCESSBLOCK CONFIGURATION FOR S3 BUCKET

To apply Public Access Block to the bucket, you need to first apply a policy which grants public access, and then apply a Public Access Block. The policy is applied through **put-bucket-policy** S3 API, which is similar to Amazon S3. For more information, see

https://docs.aws.amazon.com/AmazonS3/latest/API/API_PutPublicAccessBlock.html.



IMPORTANT

When MCG evaluates the PublicAccessBlock configuration for a bucket or an object, it checks the PublicAccessBlock configuration for both the bucket (or the bucket that contains the object) and the bucket owner's account.



NOTE

Only bucket-wide policy is supported and not account-wide policy. Also, there is no support for access control lists (ACLs) in the policy.

Public access block is applied through **put-public-access-block** using the following steps:

Procedure

1. Allow general access policy.

```
{
 "Version": "2025-06-25",
 "Statement": [
  {
   "Effect": "Allow",
   "Principal": "*",
   "Action": "s3:*",
   "Resource": [
    "arn:aws:s3:::<bucket_name>/*",
    "arn:aws:s3:::<bucket_name>"
   ]
  }
 ]
}
```

2. Put Public Access Block.

   - Default public access block configuration

```
{
    {
        "BlockPublicPolicy": false,
        "RestrictPublicBuckets": false
    }
}
```

   - Restrict public buckets

```
{
    {
        "RestrictPublicBuckets": true
    }
}
```

   - Block new public policies

```
{
    {
        "BlockPublicPolicy": true
    }
}
```

- Block both public policy and public

```
{
    {
        "BlockPublicPolicy": true,
        "RestrictPublicBuckets": true
    }
}
```

- Configuration that is not supported

```
{
    {
        "BlockPublicAcls": false, <-- Not allowed even as false
        "IgnorePublicAcls": false, <-- Not allowed even as false
        "BlockPublicPolicy": true,
        "RestrictPublicBuckets": true
    }
}
```

To set the PublicAccessBlock, use the following command:

Example command:

```
$ alias s3api=AWS_ACCESS_KEY_ID=<access_key> AWS_SECRET_ACCESS_KEY=
<secret_key> aws s3api --no-verify-ssl --endpoint <endpoint>  --no-verify put-public-
access-block --bucket <bucket_name> --public-access-block-configuration
file://<configuration_file_path>. * *
```

CHAPTER 6. SECURING MULTICLOUD OBJECT GATEWAY

Secure the Multicloud Object Gateway by rotating default account credentials and configuring TLS certificates to protect S3 endpoint access in OpenShift Data Foundation.

6.1. CHANGING THE DEFAULT ACCOUNT CREDENTIALS TO ENSURE BETTER SECURITY IN THE MULTICLOUD OBJECT GATEWAY

Change and rotate your Multicloud Object Gateway (MCG) account credentials using the command-line interface to prevent issues with applications, and to ensure better account security.

6.2. REGENERATING THE S3 CREDENTIALS FOR THE ACCOUNTS

Prerequisites

- A running OpenShift Data Foundation cluster.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Get the account name.
   For listing the accounts, run the following command:

   ```
   $ odf noobaa account list
   ```

   Example output:

   ```
   NAME          ALLOWED_BUCKETS   DEFAULT_RESOURCE             PHASE   AGE
   account-test  [*]               noobaa-default-backing-store  Ready   14m17s
   test2         [first.bucket]    noobaa-default-backing-store  Ready   3m12s
   ```

   Alternatively, run the **oc get noobaaaccount** command from the terminal:

   ```
   $ oc get noobaaaccount
   ```

   Example output:

   ```
   NAME          PHASE   AGE
   account-test  Ready   15m
   test2         Ready   3m59s
   ```

2. To regenerate the noobaa account S3 credentials, run the following command:

   ```
   $ odf noobaa account regenerate <noobaa_account_name> [options]
   ```

```
$ odf noobaa account regenerate
FATA[0000]   Missing expected arguments: <noobaa-account-name>

Usage:
   odf noobaa account regenerate <noobaa-account-name> [flags] [options]

Use "odf noobaa options" for a list of global command-line options (applies to all commands).
```

3. Once you run the **odf noobaa account regenerate** command it will prompt a warning that says **"This will invalidate all connections between S3 clients and NooBaa which are connected using the current credentials."**, and ask for confirmation:
Example:

```
$ odf noobaa account regenerate account-test
```

Example output:

```
INFO[0000] You are about to regenerate an account's security credentials.
INFO[0000] This will invalidate all connections between S3 clients and NooBaa which are
connected using the current credentials.
INFO[0000] are you sure? y/n
```

4. On approving, it will regenerate the credentials and eventually print them:

```
INFO[0015]   Exists: Secret "noobaa-account-account-test"
Connection info:
AWS_ACCESS_KEY_ID      : ***
AWS_SECRET_ACCESS_KEY  : ***
```

6.3. REGENERATING THE S3 CREDENTIALS FOR THE OBC

Prerequisites

- A running OpenShift Data Foundation cluster.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. To get the OBC name, run the following command:

```
$ odf noobaa obc list
```

Example output:

```
NAMESPACE   NAME       BUCKET-NAME                                  STORAGE-CLASS
```

```
BUCKET-CLASS                 PHASE
default     obc-test   obc-test-35800e50-8978-461f-b7e0-7793080e26ba   default.noobaa.io
noobaa-default-bucket-class   Bound
```

Alternatively, run the **oc get obc** command from the terminal:

```
$ oc get obc
```

Example output:

```
NAME       STORAGE-CLASS       PHASE   AGE
obc-test   default.noobaa.io   Bound   38s
```

2. To regenerate the noobaa OBC S3 credentials, run the following command:

```
$ odf noobaa obc regenerate <bucket_claim_name> [options]
```

```
$ odf noobaa obc regenerate
FATA[0000]   Missing expected arguments: <bucket-claim-name>

Usage:
  odf noobaa obc regenerate <bucket-claim-name> [flags] [options]

Use "odf noobaa options" for a list of global command-line options (applies to all commands).
```

3. Once you run the **odf noobaa obc regenerate** command it will prompt a warning that says **"This will invalidate all connections between the S3 clients and noobaa which are connected using the current credentials."**, and ask for confirmation:
Example:

```
$ odf noobaa obc regenerate obc-test
```

Example output:

```
INFO[0000] You are about to regenerate an OBC's security credentials.
INFO[0000] This will invalidate all connections between S3 clients and NooBaa which are
connected using the current credentials.
INFO[0000] are you sure? y/n
```

4. On approving, it will regenerate the credentials and eventually print them:

```
INFO[0022]   RPC: bucket.read_bucket() Response OK: took 95.4ms

ObjectBucketClaim info:
  Phase                : Bound
  ObjectBucketClaim    : kubectl get -n default objectbucketclaim obc-test
  ConfigMap            : kubectl get -n default configmap obc-test
  Secret               : kubectl get -n default secret obc-test
  ObjectBucket         : kubectl get objectbucket obc-default-obc-test
  StorageClass         : kubectl get storageclass default.noobaa.io
  BucketClass          : kubectl get -n default bucketclass noobaa-default-bucket-class

Connection info:
```

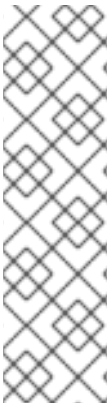
```
 BUCKET_HOST              : s3.default.svc
 BUCKET_NAME              : obc-test-35800e50-8978-461f-b7e0-7793080e26ba
   BUCKET_PORT              : 443
   AWS_ACCESS_KEY_ID      : ***
   AWS_SECRET_ACCESS_KEY  : ***

Shell commands:
  AWS S3 Alias          : alias s3='AWS_ACCESS_KEY_ID=***
AWS_SECRET_ACCESS_KEY=*** aws s3 --no-verify-ssl --endpoint-url ***'

Bucket status:
  Name                : obc-test-35800e50-8978-461f-b7e0-7793080e26ba
  Type                : REGULAR
  Mode                : OPTIMAL
  ResiliencyStatus       : OPTIMAL
  QuotaStatus            : QUOTA_NOT_SET
  Num Objects            : 0
  Data Size              : 0.000 B
  Data Size Reduced      : 0.000 B
  Data Space Avail       : 13.261 GB
  Num Objects Avail      : 9007199254740991
```

6.4. ENABLING SECURED MODE DEPLOYMENT FOR MULTICLOUD OBJECT GATEWAY

You can specify a range of IP addresses that should be allowed to reach the Multicloud Object Gateway (MCG) load balancer services to enable secure mode deployment. This helps to control the IP addresses that can access the MCG services.



NOTE

You can disable the MCG load balancer usage by setting the **disableLoadBalancerService** variable in the storagecluster custom resource definition (CRD) while deploying OpenShift Data Foundation using the command-line interface. This helps to restrict MCG from creating any public resources for private clusters and to disable the MCG service **EXTERNAL-IP**. For more information, see the Red Hat Knowledgebase article Install Red Hat OpenShift Data Foundation 4.X in internal mode using command-line interface. For information about disabling MCG load balancer service after deploying OpenShift Data Foundation, see Disabling Multicloud Object Gateway external service after deploying OpenShift Data Foundation.

Prerequisites

- A running OpenShift Data Foundation cluster.
- In case of a bare metal deployment, ensure that the load balancer controller supports setting the **loadBalancerSourceRanges** attribute in the Kubernetes services.

Procedure

- Edit the NooBaa custom resource (CR) to specify the range of IP addresses that can access the MCG services after deploying OpenShift Data Foundation.

```
$ oc edit noobaa -n openshift-storage noobaa
```

noobaa

The NooBaa CR type that controls the NooBaa system deployment.

noobaa

The name of the NooBaa CR.
For example:

```
...
spec:
  ...
  loadBalancerSourceSubnets:
    s3: ["10.0.0.0/16", "192.168.10.0/32"]
    sts:
      - "10.0.0.0/16"
      - "192.168.10.0/32"
...
```

loadBalancerSourceSubnets

A new field that can be added under **spec** in the NooBaa CR to specify the IP addresses that should have access to the NooBaa services.
In this example, all the IP addresses that are in the subnet 10.0.0.0/16 or 192.168.10.0/32 will be able to access MCG S3 and security token service (STS) while the other IP addresses are not allowed to access.

Verification

- To verify if the specified IP addresses are set, in the OpenShift Web Console, run the following command and check if the output matches with the IP addresses provided to MCG:

```
$ oc get svc -n openshift-storage <s3 | sts> -o=go-template='{{
.spec.loadBalancerSourceRanges }}'
```

6.5. DISABLING ROUTES THAT ENABLE EXTERNAL ACCESS TO MULTICLOUD OBJECT GATEWAY

You can disable **S3**, **STS**, and **noobaa-mgmt** routes by setting the following option in the storagecluster CR:

```
spec:
  multiCloudGateway:
    disableRoutes: true
```

This stops reconciling of all the three routes so that the routes are not recreated when they are deleted.

Procedure

1. Edit the storagecluster CR in one of the following ways:
   - Using the **oc patch** command:

```
oc patch storagecluster ocs-storage -n openshift-storage --type=merge -p '{"spec":
{"multiCloudGateway":{"disableRoutes":true}}}'
```

- Using the interactive method:

```
oc edit storagecluster ocs-storage -n openshift-storage
```

  - Add the following under **.spec:**

```
multiCloudGateway:
  disableRoutes: true
```

Verification

To verify, delete the existing routes and check if they are listed:

```
oc delete routes --all -n openshift-storage
oc get routes -n openshift-storage
```

The **S3**, **STS**, and **noobaa-mgmt** routes are no longer reconciled by the operator.

6.6. DISABLING HTTP ACCESS FOR NOOBAA S3 ROUTE

You can disable insecure HTTP access to the NooBaa S3 endpoint by configuring the **denyHTTP** option in the StorageCluster custom resource. This ensures that only secure HTTPS connections are allowed to access the Multicloud Object Gateway (MCG) S3 endpoint, improving the security posture and compliance of your object storage deployment.

Prerequisites

- Administrator access to OpenShift Data Foundation.
- OpenShift Data Foundation is deployed with Multicloud Object Gateway.

Procedure

1. Enable the **denyHTTP** option to disable HTTP access:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type=merge -p '{"spec":
{"multiCloudGateway":{"denyHTTP":true}}}'
```

   This command patches the StorageCluster custom resource to set **spec.multiCloudGateway.denyHTTP** to **true**.

2. Wait for the S3 route to reconcile. The operator will update the route configuration to deny HTTP traffic.

Verification

1. Verify that the S3 route's **insecureEdgeTerminationPolicy** is set to **None**:

```
$ oc get route s3 -n openshift-storage -o
jsonpath='{.spec.tls.insecureEdgeTerminationPolicy}{"\n"}'
```

Example output

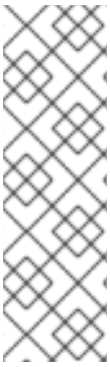
```
None
```

Re-enabling HTTP access

To re-enable HTTP access, set the **denyHTTP** option to **false**:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type=merge -p '{"spec":
{"multiCloudGateway":{"denyHTTP":false}}}'
```

After applying this change, the S3 route's **insecureEdgeTerminationPolicy** will be set back to **Allow**, permitting both HTTP and HTTPS traffic.



NOTE

- The **denyHTTP** setting only affects the S3 route. The STS and noobaa-mgmt routes are not affected by this configuration.
- Existing S3 client connections using HTTPS will not be interrupted when enabling or disabling this feature.
- Applications using HTTP endpoints will need to be reconfigured to use HTTPS endpoints when **denyHTTP** is enabled.

6.7. DISABLING HTTP ROUTE FOR RADOS OBJECT GATEWAY

You can disable insecure HTTP access to the RADOS Object Gateway (RGW) by configuring the **disableHTTP** option in the StorageCluster custom resource. This ensures that only secure HTTPS connections are allowed to access the RGW S3 endpoint, improving the security posture of your object storage deployment. You can disable HTTP route after the deployment of the cluster.

Prerequisites

- Administrator access to OpenShift Data Foundation.
- OpenShift Data Foundation is deployed with a CephObjectStore.

Procedure

- Edit the StorageCluster custom resource to disable the HTTP route using the **oc patch** command:

```
$ oc patch storagecluster ocs-storagecluster -n openshift-storage --type=merge -p '{"spec":
{"managedResources":{"cephObjectStores":{"disableHTTP":true}}}}'
```

Verification

1. Verify that the HTTP route has been deleted:

```
$ oc get routes -n openshift-storage | grep rgw
```

Example output

```
rook-ceph-rgw-ocs-storagecluster-cephobjectstore-https   rook-ceph-rgw-ocs-storagecluster-
cephobjectstore-https-openshift-storage.apps.example.com          rook-ceph-rgw-ocs-storagecluster-
cephobjectstore   https   reencrypt/Redirect     None
```

CHAPTER 7. MIRRORING DATA FOR HYBRID AND MULTICLOUD BUCKETS

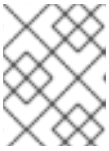
You can use the simplified process of the Multicloud Object Gateway (MCG) to span data across cloud providers and clusters. Before you create a bucket class that reflects the data management policy and mirroring, you must add a backing storage that can be used by the MCG.

You can set up mirroring data by using the OpenShift UI, YAML or command-line interface.

7.1. CREATING BUCKET CLASSES TO MIRROR DATA USING THE COMMAND-LINE INTERFACE

Prerequisites

- Download the OpenShift Data Foundation command-line interface binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. From the command-line interface, run the following command to create a bucket class with a mirroring policy:

```
$ odf noobaa bucketclass create placement-bucketclass mirror-to-aws --
backingstores=azure-resource,aws-resource --placement Mirror
```

2. Set the newly created bucket class to a new bucket claim to generate a new bucket that will be mirrored between two locations:

```
$ odf noobaa obc create  mirrored-bucket --bucketclass=mirror-to-aws
```

7.2. CREATING BUCKET CLASSES TO MIRROR DATA USING A YAML

Use this procedure to create bucket classes for mirroring data using a YAML configuration file.

Procedure

1. Apply the following YAML. This YAML is a hybrid example that mirrors data between local Ceph storage and AWS:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: noobaa
  name: <bucket-class-name>
  namespace: openshift-storage
spec:
```

```
placementPolicy:
  tiers:
  - backingStores:
    - <backing-store-1>
    - <backing-store-2>
    placement: Mirror
```

2. Add the following lines to your standard Object Bucket Claim (OBC):

```
additionalConfig:
  bucketclass: mirror-to-aws
```

For more information about OBCs, see Chapter 12, *Object Bucket Claim*.

CHAPTER 8. BUCKET POLICIES IN THE MULTICLOUD OBJECT GATEWAY

Bucket policies in the Multicloud Object Gateway use AWS S3-compatible syntax to control user access permissions to buckets and objects stored in OpenShift Data Foundation.

OpenShift Data Foundation supports AWS S3 bucket policies. Bucket policies allow you to grant users access permissions for buckets and the objects in them.

8.1. INTRODUCTION TO BUCKET POLICIES

Bucket policies are an access policy option available for you to grant permission to your AWS S3 buckets and objects. Bucket policies use JSON-based access policy language.

Additional resources

- AWS Access Policy Language Overview

8.2. USING BUCKET POLICIES IN MULTICLOUD OBJECT GATEWAY

Prerequisites

- A running OpenShift Data Foundation Platform.
- Access to the Multicloud Object Gateway (MCG), see Chapter 2, *Accessing the Multicloud Object Gateway with your applications*
- A valid Multicloud Object Gateway user account. See Creating a user in the Multicloud Object Gatewayfor instructions to create a user account.

Procedure

1. Create the bucket policy in JSON format.
   For example:

```
{
    "Version": "NewVersion",
    "Statement": [
        {
            "Sid": "Example",
            "Effect": "Allow",
            "Principal": {
                "AWS":  "john.doe@example.com"
            },
            "Action": [
                "s3:GetObject"
            ],
            "Resource": [
                "arn:aws:s3:::john_bucket"
            ]
        }
    ]
}
```

Replace **john.doe@example.com** with a valid Multicloud Object Gateway user account.

2. Using AWS S3 client, use the **put-bucket-policy** command to apply the bucket policy to your S3 bucket:

```
# aws --endpoint ENDPOINT --no-verify-ssl s3api put-bucket-policy --bucket MyBucket --policy file://BucketPolicy
```

   a. Replace ***ENDPOINT*** with the S3 endpoint.

   b. Replace ***MyBucket*** with the bucket to set the policy on.

   c. Replace ***BucketPolicy*** with the bucket policy JSON file.

   d. Add **--no-verify-ssl** if you are using the default self signed certificates.
   For example:

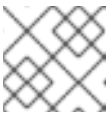
```
# aws --endpoint https://s3-openshift-storage.apps.gogo44.noobaa.org --no-verify-ssl s3api put-bucket-policy -bucket MyBucket --policy file://BucketPolicy
```

   For more information on the **put-bucket-policy** command, see the AWS CLI Command Reference for put-bucket-policy.



NOTE

The principal element specifies the user that is allowed or denied access to a resource, such as a bucket. Currently, Only NooBaa accounts can be used as principals. In the case of object bucket claims, NooBaa automatically create an account **obc-account.<generated bucket name>@noobaa.io**.



NOTE

Bucket policy conditions are not supported.

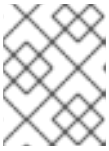
Additional resources

- AWS Access Policy Language Overview
- AWS Bucket Policy Examples
- IAM JSON policy elements reference

8.3. CREATING A USER IN THE MULTICLOUD OBJECT GATEWAY

Prerequisites

- A running OpenShift Data Foundation Platform.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

- Execute the following command to create an MCG user account:

```
$ odf noobaa account create <noobaa-account-name> [--allow_bucket_create=true] [--default_resource=''] [--force_md5_etag=false] [--gid=-1] [--new_buckets_path='/'] [--nsfs_account_config=false] [--nsfs_only=true] [--uid=-1]
```

<allow_bucket_create>

Specify if this account be allowed to create new buckets.

<default_resource>

Specify the default resource on which new buckets are created.

<force_md5_etag>

Specify if **md5 etag** calculation be enabled for the account.

<gid_number>

Specify the NSFS GID.

<new_buckets_path>

Specify the path where the new buckets are created.

<nsfs_account_config>

Specify if NSFS account needs to be created.

<nsfs_only>

Set if this account is used only for NSFS.

<uid_number>

Specify the NSFS UID.

CHAPTER 9. BUCKET NOTIFICATION IN MULTICLOUD OBJECT GATEWAY

Bucket notification allows you to send notification to an external server whenever there is an event in the bucket. Multicloud Object Gateway (MCG) supports several bucket event notification types. You can receive bucket notifications for activities such as flow creation, triggering data digestion, and so on. MCG supports the following event types that can be specified in the notification configuration:

- s3:TestEvent
- s3:ObjectCreated:*
- s3:ObjectCreated:Put
- s3:ObjectCreated:Post
- s3:ObjectCreated:Copy
- s3:ObjectCreated:CompleteMultipartUpload
- s3:ObjectRemoved:*
- s3:ObjectRemoved:Delete
- s3:ObjectRemoved:DeleteMarkerCreated
- s3:LifecycleExpiration:*
- s3:LifecycleExpiration:Delete
- s3:LifecycleExpiration:DeleteMarkerCreated
- s3:ObjectRestore:*
- s3:ObjectRestore:Post
- s3:ObjectRestore:Completed
- s3:ObjectRestore:Delete
- s3:ObjectTagging:*
- s3:ObjectTagging:Put
- s3:ObjectTagging:Delete

9.1. CONFIGURING BUCKET NOTIFICATION IN MULTICLOUD OBJECT GATEWAY

Prerequisites

- Ensure to have one of the following before configuring:
  - Kafka cluster is deployed.

For example, you can deploy Kafka using **AMQ/strimzi** by referring to Getting Started with AMQ Streams on OpenShift.

- An HTTP(s) server is connected.
  For example, you can set up an HTTP server to log incoming HTTP requests so that you can observe them using the **oc logs** command as follows:

```
$ cat http_logging_server.yaml
apiVersion: v1
kind: List
metadata: {}
items:
- apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: http-logger
  spec:
    replicas: 1
    selector:
      matchLabels:
        app: http-logger
    template:
      metadata:
        labels:
          app: http-logger
      spec:
        containers:
        - name: http-logger
          image: registry.redhat.io/ubi9/python-39:latest
          command:
            - /bin/sh
            - -c
            - |
              set -e  # Fail on any error
              mkdir -p /tmp/app
              pip install flask
              cat <<EOF > /tmp/app/server.py
              from flask import Flask, request
              app = Flask(__name__)
              @app.route("/", methods=["POST", "GET"])
              def log_request():
                  body = request.get_data(as_text=True)
                  print(body)  # Simple one-line logging per request
                  return "", 200
              if __name__ == "__main__":
                  app.run(host="0.0.0.0", port=8676, debug=True)
              EOF
              exec python /tmp/app/server.py
          ports:
          - containerPort: 8676
            protocol: TCP
          securityContext:
            runAsNonRoot: true
            allowPrivilegeEscalation: false
- apiVersion: v1
  kind: Service
```

```
  metadata:
    name: http-logger
    labels:
      app: http-logger
  spec:
    selector:
      app: http-logger
    ports:
      - port: 8676
        targetPort: 8676
        protocol: TCP
        name: http

$ oc create -f http_logging_server.yaml -n <http-server-namespace>
```

- Make sure that server is accessible from the MCG core pod from which the notifications are sent.
- Ensure to fetch MCG's credentials:

```
NOOBAA_ACCESS_KEY=$(oc extract secret/noobaa-admin -n openshift-storage --
keys=AWS_ACCESS_KEY_ID --to=- 2>/dev/null); \
NOOBAA_SECRET_KEY=$(oc extract secret/noobaa-admin -n openshift-storage --
keys=AWS_SECRET_ACCESS_KEY --to=- 2>/dev/null); \
S3_ENDPOINT=https://$(oc get route s3 -n openshift-storage -o json | jq -r ".spec.host")
alias aws_alias='AWS_ACCESS_KEY_ID=$NOOBAA_ACCESS_KEY
AWS_SECRET_ACCESS_KEY=$NOOBAA_SECRET_KEY aws --endpoint
$S3_ENDPOINT --no-verify-
```

Procedure

1. Describe connection using a **json** file, for example, **connection.json** on your local machine:
   For Kafka:

```
{
  "name": "kafka_notif_conn_file" <-- any string
  "notification_protocol": "kafka",
  "metadata.broker.list": "<kafka-service-name>.<project>.svc.cluster.local:<kafka-service-
port>",
  "topic": "my-topic",<--  refer to an existing KafkaTopic resource
}
```

   The structure under **metadata.broker.list** must be **<service-name>.<namespace>.svc.cluster.local:9092**, topic must refer to the name of an existing **kafkatopic** resource in the namespace.

   For HTTP(s):

```
{
  "name": "http_notif_connection_config",
  "notification_protocol": "http", <-- or "https"
  "agent_request_object": {
    "host": "<http-service>.<http-server-namespace>.svc.cluster.local",
    "port": <http-server-port>
   }
```

```
}
```

Additional options:

request_options_object

Value is a JSON that is passed to nodejs' http(s) request (optional).

Any field supported by nodejs' http(s) request option can be used, for example:

'path'

Used to specify the url path

'auth'

Used for http simple auth. Syntax for the value of 'auth' is: <name>:<password>.

2. Create a secret from the file in the **openshift-storage** namespace:

```
$ oc create secret generic <connection-secret> --from-file=connect.json -n openshift-storage
```

3. Update the **NooBaa** CR in the **openshift-storage** namespace with the connection secret and an optional CephFS RWX PVC. If PVC is not provided, MCG creates one automatically as needed:

```
$ oc get pvc bn-pvc -n openshift-storage
NAME     STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS                VOLUMEATTRIBUTESCLASS   AGE
bn-pvc   Bound    pvc-24683f8d-48e4-4c6b-b108-d507ca2f4fd1   25Gi       RWX
ocs-storagecluster-cephfs   <unset>                 25h
```

```
$ oc patch noobaa noobaa --type='merge' -n openshift-storage -p '{
  "spec": {
    "bucketNotifications": {
      "connections": [
        {
          "name": <connection-secret>,
          "namespace": "openshift-storage"
        }
      ],
      "enabled": true,
      "pvc": "bn-pvc" <-- optional
    }
  }
}'
```

CAUTION

Names of the connection secret must be unique in the list even though the namespaces are different.

Wait for the **noobaa-core** and **noobaa-endpoint** pods to restart before proceeding with the next step.

4. Use **S3:PutBucketNotification** on an MCG bucket using the **noobaa-admin** credentials and S3 endpoint under an S3 alias.

```
$ aws_alias s3api put-bucket-notification --bucket first.bucket --notification-configuration '{
  "TopicConfiguration": {
    "Id": "<notif_event_kafka>", <-- Unique string
    "Events": ["s3:ObjectCreated:*"], <-- a filter for events
    "Topic": "<connection-secret/connect.json>"
  }
}'
```

Verification

1. Verify that the bucket notification configuration is set on the bucket:

```
$ aws_alias s3api get-bucket-notification-configuration --bucket first.bucket
{
    "TopicConfigurations": [
        {
            "Id": "notif_event_kafka",
            "TopicArn": "kafka-connection-secret/connect.json",
            "Events": [
                "s3:ObjectCreated:*"
            ]
        }
    ]
}
```

2. Add some objects on the bucket:

```
echo 'a' | aws_alias s3 cp - s3://first.bucket/a
```

Wait for a while and query the topic messages to verify the expected notification has been sent and received.

For example:

For Kafka

```
$ oc -n your-kafka-project rsh my-cluster-kafka-0 bin/kafka-console-consumer.sh \
  --bootstrap-server my-cluster-kafka-bootstrap.myproject.svc.cluster.local:9092 \
  --topic my-topic \
  --from-beginning --timeout-ms 10000 | grep '^{.*}' | jq -c '.' | jq
```

For HTTP(s)

```
$ oc logs deployment/http-logger -n <http-server-namespace> | grep '^{.*}' | jq
```

Output

```
{
  "Records": [
    {
      "eventVersion": "2.3",
```

```
      "eventSource": "noobaa:s3",
      "eventTime": "2024-11-27T12:44:21.987Z",
      "s3": {
        "s3SchemaVersion": "1.0",
        "object": {
          "sequencer": 10,
          "key": "a",
          "eTag": "60b725f10c9c85c70d97880dfe8191b3"
        },
        "bucket": {
          "name": "second.bucket",
          "ownerIdentity": {
            "principalId": "admin@noobaa.io"
          },
          "arn": "arn:aws:s3:::first.bucket"
        }
      },
      "eventName": "ObjectCreated:Put",
      "userIdentity": {
        "principalId": "noobaa"
      },
      "requestParameters": {
        "sourceIPAddress": "100.64.0.3"
      },
      "responseElements": {
        "x-amz-request-id": "m3zvo0cm-5239xd-bhj",
        "x-amz-id-2": "m3zvo0cm-5239xd-bhj"
      }
    }
  ]
}
```

CHAPTER 10. MULTICLOUD OBJECT GATEWAY BUCKET REPLICATION

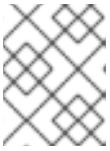
Multicloud Object Gateway bucket replication copies objects between buckets across cloud providers, enabling data redundancy and geo-distribution for S3-compatible workloads.

Data replication from one Multicloud Object Gateway (MCG) bucket to another MCG bucket provides higher resiliency and better collaboration options. These buckets can be either data buckets or namespace buckets backed by any supported storage solution (AWS S3, Azure, and so on).

A replication policy is composed of a list of replication rules. Each rule defines the destination bucket, and can specify a filter based on an object key prefix. Configuring a complementing replication policy on the second bucket results in bidirectional replication.

Prerequisites

- A running OpenShift Data Foundation Platform.
- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

To replicate a bucket, see Replicating a bucket to another bucket .

To set a bucket class replication policy, see Setting a bucket class replication policy .

10.1. REPLICATING A BUCKET TO ANOTHER BUCKET

You can set the bucket replication policy in two ways.

Additional resources

- Replicating a bucket to another bucket using the command-line interface
- Replicating a bucket to another bucket using a YAML

10.1.1. Replicating a bucket to another bucket using the command-line interface

You can set a replication policy for Multicloud Object Gateway (MCG) data bucket at the time of creation of object bucket claim (OBC). You must define the replication policy parameter in a JSON file.

Procedure

1. From the command-line interface, run the following command to create an OBC with a specific replication policy:

```
odf noobaa obc create <bucket-claim-name> -n openshift-storage --replication-policy
/path/to/json-file.json
```

<bucket-claim-name>

Specify the name of the bucket claim.

/path/to/json-file.json

Is the path to a JSON file which defines the replication policy.
Example JSON file:

```
[{ "rule_id": "rule-1", "destination_bucket": "first.bucket", "filter": {"prefix": "repl"}}]
```

"prefix"

Is optional. It is the prefix of the object keys that should be replicated, and you can even leave it empty, for example, **{"prefix": ""}**.
Example:

```
odf noobaa obc create my-bucket-claim -n openshift-storage --replication-policy
/path/to/json-file.json
```

10.1.2. Replicating a bucket to another bucket using a YAML

You can set a replication policy for Multicloud Object Gateway (MCG) data bucket at the time of creation of object bucket claim (OBC) or you can edit the YAML later. You must provide the policy as a JSON-compliant string that adheres to the format shown in the following procedure.

Procedure

- Apply the following YAML:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: <desired-bucket-claim>
  namespace: <desired-namespace>
spec:
  generateBucketName: <desired-bucket-name>
  storageClassName: openshift-storage.noobaa.io
  additionalConfig:
    replicationPolicy: {"rules": [{ "rule_id": "", "destination_bucket": "", "filter": {"prefix": ""}}]}
```

<desired-bucket-claim>

Specify the name of the bucket claim.

<desired-namespace>

Specify the namespace.

<desired-bucket-name>

Specify the prefix of the bucket name.

"rule_id"

Specify the ID number of the rule, for example, **{"rule_id": "rule-1"}**.

"destination_bucket"

Specify the name of the destination bucket, for example, **{"destination_bucket": "first.bucket"}**.

"prefix"

Is optional. It is the prefix of the object keys that should be replicated, and you can even leave it empty, for example, **{"prefix": ""}**.

Additional resources

- Object Bucket Claim

10.2. SETTING A BUCKET CLASS REPLICATION POLICY

It is possible to set up a replication policy that automatically applies to all the buckets created under a certain bucket class.

Additional resources

- Setting a bucket class replication policy using the command-line interface
- Setting a bucket class replication policy using a YAML

10.2.1. Setting a bucket class replication policy using the command-line interface

You can set a replication policy for Multicloud Object Gateway (MCG) data bucket at the time of creation of bucket class. You must define the **replication-policy** parameter in a JSON file. You can set a bucket class replication policy for the Placement and Namespace bucket classes.

You can set a bucket class replication policy for the Placement and Namespace bucket classes.

Procedure

- From the command-line interface, run the following command:

```
odf noobaa -n openshift-storage bucketclass create placement-bucketclass <bucketclass-name> --backingstores <backingstores> --replication-policy=/path/to/json-file.json
```

<bucketclass-name>

Specify the name of the bucket class.

<backingstores>

Specify the name of a backingstore. You can pass many backingstores separated by commas.

/path/to/json-file.json

Is the path to a JSON file which defines the replication policy.
Example JSON file:

```
[{ "rule_id": "rule-1", "destination_bucket": "first.bucket", "filter": {"prefix": "repl"}}]
```

"prefix"

Is optional. The prefix of the object keys gets replicated. You can leave it empty, for example, **{"prefix": ""}**.
For example:

```
odf noobaa -n openshift-storage bucketclass create placement-bucketclass bc --
backingstores azure-blob-ns --replication-policy=/path/to/json-file.json
```

This example creates a placement bucket class with a specific replication policy defined in the JSON file.

10.2.2. Setting a bucket class replication policy using a YAML

You can set a replication policy for Multicloud Object Gateway (MCG) data bucket at the time of creation of bucket class or you can edit their YAML later. You must provide the policy as a JSON-compliant string that adheres to the format shown in the following procedure.

Procedure

1. Apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  labels:
    app: <desired-app-label>
  name: <desired-bucketclass-name>
  namespace: <desired-namespace>
spec:
  placementPolicy:
    tiers:
    - backingstores:
      - <backingstore>
      placement: Spread
  replicationPolicy: [{ "rule_id": "<rule id>", "destination_bucket": "first.bucket", "filter": {"prefix":
"<object name prefix>"}}]
```

This YAML is an example that creates a placement bucket class. Each Object bucket claim (OBC) object that is uploaded to the bucket is filtered based on the prefix and is replicated to **first.bucket**.

<desired-app-label>

Specify a label for the app.

<desired-bucketclass-name>

Specify the bucket class name.

<desired-namespace>

Specify the namespace in which the bucket class gets created.

<backingstore>

Specify the name of a backingstore. You can pass many backingstores.

"rule_id"

Specify the ID number of the rule, for example, **{"rule_id": "rule-1"}**.

"destination_bucket"

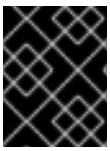
Specify the name of the destination bucket, for example, **{"destination_bucket": "first.bucket"}**.

"prefix"

Is optional. The prefix of the object keys gets replicated. You can leave it empty, for example, **{"prefix": ""}**.

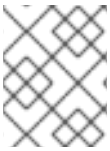
10.3. ENABLING LOG BASED BUCKET REPLICATION

When creating a bucket replication policy, you can use logs so that recent data is replicated more quickly, while the default scan-based replication works on replicating the rest of the data. Bucket replication policy using bucket logging is supported for MCG to MCG, MCG to AWS, and MCG to Azure.



IMPORTANT

This feature requires setting up bucket logs on AWS or Azure. The AWS logs bucket needs to be created in the same region as the source NamespaceStore AWS bucket.



NOTE

This feature is only supported in buckets that are backed by a NamespaceStore. Buckets backed by BackingStores cannot utilized log-based replication.

Additional resources

- Enabling Amazon S3 server access logging

10.3.1. Enabling log based bucket replication for new namespace buckets using OpenShift Web Console in Amazon Web Service environment

You can optimize replication by using the event logs of the Amazon Web Service(AWS) cloud environment. You enable log based bucket replication for new namespace buckets using the web console during the creation of namespace buckets.

Prerequisites

- Ensure that object logging is enabled in AWS. For more information, see the "Using the S3 console" section in Enabling Amazon S3 server access logging .
- Administrator access to OpenShift Web Console.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage → Object Bucket Claims**.
2. Click **Create ObjectBucketClaim**.
3. Enter the name of ObjectBucketName and select StorageClass and BucketClass.
4. Select the **Enable replication** check box to enable replication.
5. In the **Replication policy** section, select the **Optimize replication using event logs** checkbox.

6. Enter the name of the bucket that will contain the logs under **Event log Bucket**.
   If the logs are not stored in the root of the bucket, provide the full path without **s3://**

7. Enter a prefix to replicate only the objects whose name begins with the given prefix.

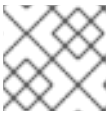
10.3.2. Enabling log based bucket replication for existing namespace buckets using YAML

You can enable log based bucket replication for the existing buckets that are created using the command-line interface or by applying an YAML, and not the buckets that are created using AWS S3 commands.

Procedure

- Edit the YAML of the bucket's OBC to enable log based bucket replication. Add the following under **spec**:

```
replicationPolicy: '{"rules":[{"rule_id":"<RULE ID>", "destination_bucket":"<DEST>", "filter": {"prefix": "<PREFIX>"}}], "log_replication_info": {"logs_location": {"logs_bucket": "<LOGS_BUCKET>"}}}'
```



NOTE

It is also possible to add this to the YAML of an OBC before it is created.

rule_id

Specify an ID of your choice for identifying the rule

destination_bucket

Specify the name of the target MCG bucket that the objects are copied to

(optional) {"filter": {"prefix": <>}}

Specify a prefix string that you can set to filter the objects that are replicated

log_replication_info

Specify an object that contains data related to log-based replication optimization. **{"logs_location": {"logs_bucket": <>}}** is set to the location of the AWS S3 server access logs.

10.3.3. Enabling log based bucket replication in Microsoft Azure

Prerequisites

- Refer to Microsoft Azure documentation and ensure that you have completed the following tasks in the Microsoft Azure portal:

  a. Ensure that have created a new application and noted down the name, application (client) ID, and directory (tenant) ID.
     For information, see Register an application.

  b. Ensure that a new a new client secret is created and the application secret is noted down.

  c. Ensure that a new Log Analytics workspace is created and its name and workspace ID is noted down.

For information, see Create a Log Analytics workspace.

d. Ensure that the **Reader** role is assigned under **Access control** and members are selected and the name of the application that you registered in the previous step is provided. For more information, see Assign Azure roles using the Azure portal.

e. Ensure that a new storage account is created and the Access keys are noted down.

f. In the **Monitoring** section of the storage account created, select a blob and in the **Diagnostic settings** screen, select only **StorageWrite** and **StorageDelete**, and in the destination details add the Log Analytics workspace that you created earlier. Ensure that a blob is selected in the **Diagnostic settings** screen of the **Monitoring** section of the storage account created. Also, ensure that only **StorageWrite** and **StorageDelete** is selected and in the destination details, the Log Analytics workspace that you created earlier is added. For more information, see Diagnostic settings in Azure Monitor.

g. Ensure that two new containers for object source and object destination are created.

- Administrator access to OpenShift Web Console.

Procedure

1. Create a secret with credentials to be used by the **namespacestores**.

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
type: Opaque
data:
    TenantID: <AZURE TENANT ID ENCODED IN BASE64>
    ApplicationID: <AZURE APPLICATIOM ID ENCODED IN BASE64>
    ApplicationSecret: <AZURE APPLICATION SECRET ENCODED IN BASE64>
    LogsAnalyticsWorkspaceID: <AZURE LOG ANALYTICS WORKSPACE ID ENCODED IN
BASE64>
    AccountName: <AZURE ACCOUNT NAME ENCODED IN BASE64>
    AccountKey: <AZURE ACCOUNT KEY ENCODED IN BASE64>
```

2. Create a **NamespaceStore** backed by a container created in Azure.
   For more information, see Adding a namespace bucket using the OpenShift Container Platform user interface.

3. Create a new Namespace-Bucketclass and OBC that utilizes it.

4. Check the object bucket name by looking in the YAML of target OBC, or by listing all S3 buckets, for example, **- s3 ls**.

5. Use the following template to apply an Azure replication policy on your source OBC by adding the following in its YAML, under **.spec**:

```
replicationPolicy:'{"rules":[ {"rule_id":"ID goes here", "sync_deletions": "<true or false>"",
"destination_bucket":object bucket name"}
 ], "log_replication_info":{"endpoint_type":"AZURE"}}'
```

sync_deletion

Specify a boolean value, **true** or **false**.

destination_bucket

Make sure to use the name of the object bucket, and not the claim. The name can be retrieved using the **s3 ls** command, or by looking for the value in an OBC's YAML.

Verification

1. Write objects to the source bucket.
2. Wait until MCG replicates them.
3. Delete the objects from the source bucket.
4. Verify the objects were removed from the target bucket.

10.3.4. Enabling log-based bucket replication deletion

Use this procedure to enable log-based bucket replication deletion for multicloud object gateway.

Prerequisites

- Administrator access to OpenShift Web Console.
- AWS Server Access Logging configured for the desired bucket.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage → Object Bucket Claims**.
2. Click **Create new Object bucket claim**
3. (Optional) In the **Replication rules** section, select the **Sync deletion** checkbox for each rule separately.
4. Enter the name of the bucket that will contain the logs under **Event log Bucket**.
   If the logs are not stored in the root of the bucket, provide the full path without **s3:**//
5. Enter a prefix to replicate only the objects whose name begins with the given prefix.

10.4. BUCKET LOGGING FOR MULTICLOUD OBJECT GATEWAY

Bucket logging helps you to record the S3 operations that are performed against the Multicloud Object Gateway (MCG) bucket for compliance, auditing, and optimization purposes.

Bucket logging supports the following two options:

- Best-effort - Bucket logging is recorded using UDP on the best effort basis
- Guaranteed - Bucket logging with this option creates a PVC attached to the MCG pods and saves the logs to this PVC on a Guaranteed basis, and then from the PVC to the log buckets. Using this option logging takes place twice for every S3 operation as follows:
  - At the start of processing the request

- At the end with the result of the S3 operation

10.4.1. Enabling bucket logging for Multicloud Object Gateway using the Best-effort option

You can enable Best-effort bucket logging, which records S3 operations using UDP on a best effort basis.

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to MCG.
  For information, see Accessing the Multicloud Object Gateway with your applications .

Procedure

1. Create a data bucket where you can upload the objects.

```
odf noobaa bucket create data.bucket
```

2. Create a log bucket where you want to store the logs for bucket operations by using the following command:

```
odf noobaa bucket create log.bucket
```

3. Configure bucket logging on data bucket with log bucket in one of the following ways:
   - Using the NooBaa API

```
odf noobaa api bucket_api put_bucket_logging '{
  "name": "data.bucket",
  "log_bucket": "log.bucket",
  "log_prefix": "data-bucket-logs"
}'
```

   - Using the S3 API

```
alias s3api_alias='AWS_ACCESS_KEY_ID=$NOOBAA_ACCESS_KEY
AWS_SECRET_ACCESS_KEY=$NOOBAA_SECRET_KEY aws --endpoint
https://localhost:10443 --no-verify-ssl s3api'
```

     a. Create a file called **setlogging.json** in the following format:

```
{
  "LoggingEnabled": {
    "TargetBucket": "<log-bucket-name>",
    "TargetPrefix": "<prefix/empty-string>"
  }
}
```

     b. Run the following command:

```
s3api_alias put-bucket-logging --endpoint <ep> --bucket <source-bucket> --bucket-logging-status file://setlogging.json --no-verify-ssl
```

4. Verify if the bucket logging is set for the data bucket in one of the following ways:

   - Using the NooBaa API

     ```
     odf noobaa api bucket_api get_bucket_logging '{
       "name": "data.bucket"
     }'
     ```

   - Using the S3 API

     ```
     s3api_alias get-bucket-logging --no-verify-ssl --endpoint <ep> --bucket <source-bucket>
     ```

     The S3 operations can take up to 24 hours to get recorded in the logs bucket. The following example shows the recorded logs and how to download them:

     Example

```
s3_alias cp s3://logs.bucket/data-bucket-logs/logs.bucket.bucket_data-bucket-logs_1719230150.log - | tail -n 2

Jun 24 14:00:02 10-XXX-X-XXX.sts.openshift-storage.svc.cluster.local {"noobaa_bucket_logging":"true","op":"GET","bucket_owner":"operator@noobaa.io","source_bucket":"data.bucket","object_key":"/data.bucket?list-type=2&prefix=data-bucket-logs&delimiter=%2F&encoding-type=url","log_bucket":"logs.bucket","remote_ip":"100.XX.X.X","request_uri":"/data.bucket?list-type=2&prefix=data-bucket-logs&delimiter=%2F&encoding-type=url","request_id":"luv2XXXX-ctyg2k-12gs"} Jun 24 14:00:06 10-XXX-X-XXX.s3.openshift-storage.svc.cluster.local {"noobaa_bucket_logging":"true","op":"PUT","bucket_owner":"operator@noobaa.io","source_bucket":"data.bucket","object_key":"/data.bucket/B69EC83F-0177-44D8-A8D1-4A10C5A5AB0F.file","log_bucket":"logs.bucket","remote_ip":"100.XX.X.X","request_uri":"/data.bucket/B69EC83F-0177-44D8-A8D1-4A10C5A5AB0F.file","request_id":"luv2XXXX-9syea5-x5z"}
```

1. (Optional) To disable bucket logging, use the following command:

   ```
   odf noobaa api bucket_api delete_bucket_logging '{
     "name": "data.bucket"
   }'
   ```

10.4.2. Enabling bucket logging using the Guaranteed option

You can enable Guaranteed bucket logging, which creates a PVC attached to the MCG pods and saves logs on a guaranteed basis. Using this option, logging takes place twice for every S3 operation: at the start of processing the request, and at the end with the result.

Prerequisites

- Openshift Container Platform with OpenShift Data Foundation operator installed.
- Access to MCG.
  For information, see Accessing the Multicloud Object Gateway with your applications .

Procedure

- Enable Guaranteed bucket logging using the NooBaa CR in one of the following ways:
  - Using the default CephFS storage class update the NooBaa CR spec:

```
bucketLogging:
{
loggingType: guaranteed
}
```

  - Using the RWX PVC that you created:



NOTE

Make sure that the PVC supports RWX

```
bucketLogging:
{
loggingType: guaranteed
bucketLoggingPVC: <pvc-name>
}
```

10.5. SYNCHRONIZING VERSIONS IN MULTICLOUD OBJECT GATEWAY BUCKET REPLICATION

Learn about version synchronization mechanisms in Multicloud Object Gateway bucket replication.

Prerequisites

- A Multicloud Object Gateway (MCG) source bucket, which is created from an object bucket claim (OBC) and any MCG target bucket. For example, you can create the two buckets using OBCs using the command-line interface (CLI):
  - Create a source bucket using an OBC:

```
$ odf noobaa obc create source-bucket --exact
$ odf noobaa obc create target-bucket --exact
```

    where **--exact** is optional.

- Ensure that S3 client aliases with MCG credentials and endpoint are set up.

```
$ NOOBAA_ACCESS_KEY=$(oc extract secret/noobaa-admin -n openshift-storage --
keys=AWS_ACCESS_KEY_ID --to=- 2>/dev/null); \
NOOBAA_SECRET_KEY=$(oc extract secret/noobaa-admin -n openshift-storage --
keys=AWS_SECRET_ACCESS_KEY --to=- 2>/dev/null); \
S3_ENDPOINT=https://$(oc get route s3 -n openshift-storage -o json | jq -r ".spec.host")
```

```
$ alias common_s3='AWS_ACCESS_KEY_ID=$NOOBAA_ACCESS_KEY
AWS_SECRET_ACCESS_KEY=$NOOBAA_SECRET_KEY aws --endpoint
$S3_ENDPOINT --no-verify-ssl'; \
```

```
alias s3_alias='common_s3 s3'; \
alias s3api_alias='common_s3 s3api'
```

- Make sure to enable versioning on both the source and target bucket by using the **put-bucket-versioning** command in the AWS S3 client:

```
$ s3api_alias put-bucket-versioning --bucket source-bucket --versioning-configuration
Status=Enabled
```

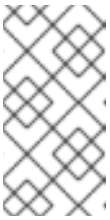
```
$ s3api_alias put-bucket-versioning --bucket target-bucket --versioning-configuration
Status=Enabled
```

Procedure

1. Patch or edit a replication policy with **sync_versions: true** to the source bucket's OBC:

```
$ oc patch obc source-bucket -n openshift-storage --type=merge -p '{"spec":
{"additionalConfig": {"replicationPolicy": "{\"rules\":[{\"rule_id\":\"replication-rule-
a\",\"destination_bucket\":\"target-bucket\", \"sync_versions\": true}]}"}}}'
```

Normal bucket replication replicates only the latest version, and the use of **sync_versions** add the functionality of replicating even the older versions, and in their original order.



NOTE

- Delete markers are replicated if configured and using log base replication.
- Delete version IDs are not replicated to avoid human errors.

Verification

- Verify the replication to the target bucket by adding a few versions under an object key and waiting for them to get replicated to the target bucket:

```
$ echo 'version_a' | s3_alias cp - s3://source-bucket/versioned_obj.txt
```

```
$ echo 'version_b' | s3_alias cp - s3://source-bucket/versioned_obj.txt
```

```
$ echo 'version_c' | s3_alias cp - s3://source-bucket/versioned_obj.txt
```

After a few minutes, compare the versions of the object on both the buckets:

```
$ s3api_alias list-object-versions --bucket source-bucket --prefix versioned_obj.txt | jq -r
".Versions[].ETag"
"aaabf266d38a8e995cef03c13ee9a7f1"
"181d8a23f59939c0cddec1692e05cdf3"
"ebc538cc6ffa04a39263f4b1be2f832f"
```

```
$ s3api_alias list-object-versions --bucket target-bucket --prefix versioned_obj.txt | jq -r
".Versions[].ETag"
"aaabf266d38a8e995cef03c13ee9a7f1"
```

```
"181d8a23f59939c0cddec1692e05cdf3"
"ebc538cc6ffa04a39263f4b1be2f832f"
```

10.6. OBTAINING METRICS TO REFLECT BUCKET REPLICATION STATE

To determine if the data is safe or available on the secondary site, the replication progress per bucket is provided using the following metrics :

- Total number of objects scanned in the last replication cycle per bucket – **bucket_last_cycle_total_objects_num {bucket="bucket-name"}**
- Number of objects successfully replicated in the last cycle per bucket – **bucket_last_cycle_replicated_objects_num{bucket="bucket-name"}**
- Number of objects that failed replication in the last cycle per bucket – **bucket_last_cycle_error_objects_num {bucket="bucket-name"}**

Procedure

- Use the following commands to obtain the metrics:

```
JWT_TOKEN=$(oc get secret noobaa-metrics-auth-secret -n openshift-storage -o jsonpath="
{.data.metrics_token}
" | base64 -d)

oc -n openshift-storage exec noobaa-core-0 – curl -k -H "Authorization: Bearer
${JWT_TOKEN}" localhost:8080/metrics/bg_workers
```

For example:

```
noobaa-operator$ oc -n openshift-storage exec -it noobaa-core-0 -- curl -k -H "Authorization:
Bearer ${JWT_TOKEN}" http://localhost:8080/metrics/bg_workers | grep -E "bucket_name"|
grep -v "NooBaa_replication_status"
Defaulted container "core" out of: core, noobaa-log-processor

NooBaa_bucket_last_cycle_total_objects_num{bucket_name="test-buck1"} 15
NooBaa_bucket_last_cycle_total_objects_num{bucket_name="test-buck2"} 25
NooBaa_bucket_last_cycle_replicated_objects_num{bucket_name="test-buck1"} 15
NooBaa_bucket_last_cycle_replicated_objects_num{bucket_name="test-buck2"} 25
NooBaa_bucket_last_cycle_error_objects_num{bucket_name="test-buck1"} 0
NooBaa_bucket_last_cycle_error_objects_num{bucket_name="test-buck2"} 0
```

- Use the following syntax to view the metrics in the metrics section:

```
metric_name {bucket_name='bucket_name'}
```

For example:

For **NooBaa_bucket_last_cycle_total_objects_num**

```
NooBaa_bucket_last_cycle_total_objects_num {bucket_name='bubck1'}
```

For **NooBaa_bucket_last_cycle_replicated_objects_num**

```
NooBaa_bucket_last_cycle_replicated_objects_num {bucket_name='bubck1'}
```

For NooBaa_bucket_last_cycle_error_objects_num `

```
NooBaa_bucket_last_cycle_error_objects_num {bucket_name='bubck1'}
```

CHAPTER 11. MANAGING S3 VECTORS

S3 Vectors provide native vector-database storage capabilities within the Multicloud Object Gateway (MCG), enabling you to store and manage vector embeddings for machine learning and AI applications.

This section describes how to create and manage vector buckets, vector indexes, and related resources.

11.1. S3 VECTORS IN MULTICLOUD OBJECT GATEWAY

S3 Vectors provide native vector-database storage capabilities within the Multicloud Object Gateway (MCG). Vector buckets enable you to store and manage vector embeddings for machine learning and AI applications, backed by NSFS (NamespaceStore FileSystem) storage.

Vector buckets are specialized buckets designed for storing vector data, which is commonly used in machine learning model embeddings, semantic search applications, recommendation systems, natural language processing (NLP) workloads, and AI-powered applications. They are provisioned through the standard ObjectBucketClaim (OBC) flow and managed using the AWS S3 Vectors CLI.

Currently, MCG supports **Lance** as the vector database type. Lance is a modern columnar data format optimized for machine learning workloads.

Vector buckets integrate seamlessly with MCG's object storage infrastructure, with vector data stored on filesystem-based NamespaceStores for optimal performance. You can use familiar ObjectBucketClaim workflows to create vector buckets and manage vector data using standard AWS CLI tools.

Vector buckets require the following components: an NSFS NamespaceStore that provides the underlying filesystem storage for vector data, a Vector BucketClass that defines the vector policy including the backing NamespaceStore and vector database type, and a Vector ObjectBucketClaim that provisions the vector bucket with appropriate configuration.

Limitations

- Only NSFS NamespaceStores are supported as the backing resource.
- Only **lance** is supported as the vector database type.
- Bucket tagging and quota are not yet supported for vector buckets.
- Vector bucket class updates (spec changes) are not supported.

11.2. CREATING A VECTOR BUCKETCLASS

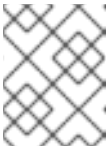
A vector BucketClass defines the storage policy for vector buckets, specifying the NSFS NamespaceStore resource and the vector database type.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- An NSFS NamespaceStore is created. For more information, see Creating a NamespaceStore to use a filesystem.

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage → Bucket Class** tab.
2. Click **Create Bucket Class**.
3. On the **Create new Bucket Class** page:
   a. Enter a **Bucket Class Name**.
   b. Select **Vector** as the **BucketClass type**.
   c. Click **Next**.
4. On the **Placement Policy** page:
   a. Select the NSFS NamespaceStore from the **Read and Write NamespaceStore** dropdown list.



NOTE

Only filesystem (NSFS) NamespaceStores are available for vector bucket classes. Other NamespaceStore types are not supported.

   b. Click **Next**.
5. Review your configuration and click **Create Bucket Class**.

Verification

1. Navigate to **Storage → Object Storage → Bucket Class** tab.
2. Verify that your newly created vector bucket class is in the **Ready** phase.

Additional resources

- Creating vector buckets using ObjectBucketClaim

11.3. CREATING VECTOR BUCKETS USING OBJECTBUCKETCLAIM

You can create vector buckets using ObjectBucketClaim (OBC) to provision storage for vector data in machine learning and AI applications.

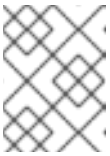
Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- A vector BucketClass is created. For more information, see Creating a vector BucketClass .
- AWS CLI v2.34.23 or later is installed (for **s3vectors** subcommand support).

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage → ObjectBucketClaims** tab.
2. Click the **S3 Vector** card.

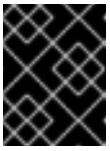
3. On the **Create ObjectBucketClaim** page:
   a. Enter an **ObjectBucketClaim Name**.
   b. Select the **StorageClass** as **openshift-storage.noobaa.io**.
   c. Select the vector **BucketClass** that you created earlier from the dropdown list.



NOTE

Only BucketClasses that define a **vectorPolicy** are listed. BucketClasses without **vectorPolicy** are excluded.

   d. (Optional) Enter a **Path** in the **Additional Configuration** section. This specifies a subdirectory path inside the NSFS PVC where vector data is stored.



IMPORTANT

If you specify a path, ensure that the directory exists on the filesystem under the PV mount path.

   e. Click **Create**.

Verification

1. Navigate to **Storage → Object Storage → ObjectBucketClaims** tab.
2. Verify that your ObjectBucketClaim is in the **Bound** phase.
3. Click on the ObjectBucketClaim name to view details, including the bucket name and credentials.

Next steps

- To manage vector indexes in your vector bucket, see Managing vector indexes.

11.4. CREATING VECTOR BUCKETS USING S3 API

You can create vector buckets directly using the S3 API through the MCG object browser interface.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- A vector BucketClass is created. For more information, see Creating a vector BucketClass .
- Access to the MCG object browser. For more information, see Accessing the Object Browser.

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Object Storage page, choose **Buckets** tab, then click the **MultiCloud Object Gateway(MCG)** tab.

3. Click the **S3 Vector** tab.
4. Click **Create bucket**.
5. Select **Create via S3 API**
6. On the bucket creation form:
   a. Enter a **Bucket Name**.
   b. Select the vector **BucketClass** from the dropdown list.



NOTE

Only BucketClasses that define a **vectorPolicy** are listed. When creating vector buckets via S3 API, the NamespaceStore list is restricted to filesystem (NSFS) NamespaceStores.

   c. (Optional) Configure additional settings as needed.
   d. Click **Create**.

Verification

1. Navigate to **Storage → Object Storage → Buckets** tab → **MultiCloud Object Gateway(MCG) → S3 Vector**.
2. Verify that your newly created vector bucket is listed.
3. Click on the bucket name to view details and access the **Vector indexes** tab.

Next steps

- To manage vector indexes in your vector bucket, see Managing vector indexes.

11.5. MANAGING VECTOR INDEXES

Vector indexes store and organize vector embeddings within a vector bucket. You can create, list, view details, and delete vector indexes using the MCG object browser.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- A vector bucket is created.
- Access to the MCG object browser.

Additional resources

- Creating vector buckets using ObjectBucketClaim
- Accessing the Object Browser

11.5.1. Listing vector indexes

You can view all vector indexes within a vector bucket using the MCG object browser.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- A vector bucket is created. For more information, see Creating vector buckets using ObjectBucketClaim.
- Access to the MCG object browser. For more information, see Accessing the Object Browser.

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Object Storage page, choose **Buckets** tab, then click the **MultiCloud Object Gateway(MCG)** tab.
3. Click on the required vector bucket name.
4. Select the **Vector indexes** tab.
   The page displays up to 100 vector indexes. If there are more than 100 vector indexes, use the **Next** and **Previous** buttons on the top-right of the table to navigate between pages.

11.5.2. Creating a vector index

You can create a new vector index within a vector bucket using the MCG object browser.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- A vector bucket is created. For more information, see Creating vector buckets using ObjectBucketClaim.
- Access to the MCG object browser. For more information, see Accessing the Object Browser.

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Object Storage page, choose **Buckets** tab, then click the **MultiCloud Object Gateway(MCG)** tab.
3. Click on the required vector bucket name.
4. Select the **Vector indexes** tab.
5. Click **Create index**.
6. On the **Create index** form:
   a. Enter an **Index Name**.
   b. Provide the **Dimension**.

c. Proved the **Distance Metric**.

d. Provide the Metadata keys up to 10.

e. Click **Create**.

Verification

The newly created vector index appears in the **Vector indexes** tab list.

11.5.3. Viewing vector index details

You can view the details of a vector index within a vector bucket using the MCG object browser.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- A vector bucket is created. For more information, see Creating vector buckets using ObjectBucketClaim.
- Access to the MCG object browser. For more information, see Accessing the Object Browser.

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Object Storage page, choose **Buckets** tab, then click the **MultiCloud Object Gateway(MCG)** tab.
3. Click on the required vector bucket name.
4. Select the **Vector indexes** tab.
5. Click on the row corresponding to the vector index you want to view.
   The vector index details page displays information about the index, including its name, description, creation time, and other properties.

11.5.4. Deleting a vector index

You can delete a vector index from a vector bucket using the MCG object browser.



IMPORTANT

You must delete all vector indexes in a vector bucket before you can delete the bucket itself.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- A vector bucket is created. For more information, see Creating vector buckets using ObjectBucketClaim.
- Access to the MCG object browser. For more information, see Accessing the Object Browser.

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Object Storage page, choose **Buckets** tab, then click the **MultiCloud Object Gateway(MCG)** tab.
3. Click on the required vector bucket name.
4. Select the **Vector indexes** tab.
5. Select the kebab menu (⋮) corresponding to the vector index you want to delete.
6. Select **Delete index**.
7. In the confirmation dialog, type the vector index name to confirm deletion.
8. Click **Delete**.

Verification

The deleted vector index no longer appears in the **Vector indexes** tab list.

11.6. DELETING VECTOR BUCKETS

You can delete vector buckets when they are no longer needed. Before deleting a vector bucket, you must first delete all vector indexes within the bucket.

Prerequisites

- A running OpenShift Data Foundation cluster with Multicloud Object Gateway (MCG).
- Access to the MCG object browser. For more information, see Accessing the Object Browser.
- All vector indexes in the bucket are deleted. For more information, see Managing vector indexes.

Procedure

1. From the OpenShift Web Console, navigate to **Storage → Object Storage**.
2. From the Object Storage page, choose **Buckets** tab, then click the **MultiCloud Object Gateway(MCG)** tab.
3. Select the kebab menu (⋮) corresponding to the vector bucket you want to delete.
4. Select **Delete bucket**.
5. In the confirmation dialog, type the vector bucket name to confirm deletion.
6. Click **Delete bucket**.



NOTE

If the bucket contains vector indexes, you will receive an error message. Delete all vector indexes first before attempting to delete the bucket.

Verification

The deleted vector bucket no longer appears in the S3 Vector buckets list.

CHAPTER 12. OBJECT BUCKET CLAIM

Object Bucket Claims (OBCs) provide applications with S3-compatible object storage access by automatically provisioning buckets and credentials in OpenShift Data Foundation.

12.1. OBJECT BUCKET CLAIM

An Object Bucket Claim can be used to request an S3 compatible bucket backend for your workloads.

You can create an Object Bucket Claim in three ways. An object bucket claim creates a new bucket and an application account in NooBaa with permissions to the bucket, including a new access key and secret access key. The application account is allowed to access only a single bucket and can't create new buckets by default.

Additional resources

- Section 12.2, "Dynamic Object Bucket Claim"
- Section 12.3, "Creating an Object Bucket Claim using the command-line interface"
- Section 12.4, "Creating an Object Bucket Claim using the OpenShift Web Console"

12.2. DYNAMIC OBJECT BUCKET CLAIM

Similar to Persistent Volumes, you can add the details of the Object Bucket claim (OBC) to your application's YAML, and get the object service endpoint, access key, and secret access key available in a configuration map and secret. It is easy to read this information dynamically into environment variables of your application.



NOTE

The Multicloud Object Gateway endpoints uses self-signed certificates only if OpenShift uses self-signed certificates. Using signed certificates in OpenShift automatically replaces the Multicloud Object Gateway endpoints certificates with signed certificates. Get the certificate currently used by Multicloud Object Gateway by accessing the endpoint via the browser. See Accessing the Multicloud Object Gateway with your applications for more information.

Procedure

1. Add the following lines to your application YAML:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: <obc-name>
spec:
  generateBucketName: <obc-bucket-name>
  storageClassName: openshift-storage.noobaa.io
```

These lines are the OBC itself.

   a. Replace **<obc-name>** with the a unique OBC name.

b. Replace **<obc-bucket-name>** with a unique bucket name for your OBC.

2. To automate the use of the OBC add more lines to the YAML file.
For example:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: testjob
spec:
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - image: <your application image>
          name: test
          env:
            - name: BUCKET_NAME
              valueFrom:
                configMapKeyRef:
                  name: <obc-name>
                  key: BUCKET_NAME
            - name: BUCKET_HOST
              valueFrom:
                configMapKeyRef:
                  name: <obc-name>
                  key: BUCKET_HOST
            - name: BUCKET_PORT
              valueFrom:
                configMapKeyRef:
                  name: <obc-name>
                  key: BUCKET_PORT
            - name: AWS_ACCESS_KEY_ID
              valueFrom:
                secretKeyRef:
                  name: <obc-name>
                  key: AWS_ACCESS_KEY_ID
            - name: AWS_SECRET_ACCESS_KEY
              valueFrom:
                secretKeyRef:
                  name: <obc-name>
                  key: AWS_SECRET_ACCESS_KEY
```

The example is the mapping between the bucket claim result, which is a configuration map with data and a secret with the credentials. This specific job claims the Object Bucket from NooBaa, which creates a bucket and an account.

a. Replace all instances of **<obc-name>** with your OBC name.

b. Replace **<your application image>** with your application image.

3. Apply the updated YAML file:

```
# oc apply -f <yaml.file>
```

Replace **<yaml.file>** with the name of your YAML file.

4. To view the new configuration map, run the following:

```
# oc get cm <obc-name> -o yaml
```

Replace **obc-name** with the name of your OBC.

You can expect the following environment variables in the output:

- **BUCKET_HOST** - Endpoint to use in the application.
- **BUCKET_PORT** - The port available for the application.
  - The port is related to the **BUCKET_HOST**. For example, if the **BUCKET_HOST** is https://my.example.com, and the **BUCKET_PORT** is 443, the endpoint for the object service would be https://my.example.com:443.
- **BUCKET_NAME** - Requested or generated bucket name.
- **AWS_ACCESS_KEY_ID** - Access key that is part of the credentials.
- **AWS_SECRET_ACCESS_KEY** - Secret access key that is part of the credentials.



IMPORTANT

Retrieve the **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY**. The names are used so that it is compatible with the AWS S3 API. You need to specify the keys while performing S3 operations, especially when you read, write or list from the Multicloud Object Gateway (MCG) bucket. The keys are encoded in Base64. Decode the keys before using them.

```
# oc get secret <obc_name> -o yaml
```

<obc_name>

Specify the name of the object bucket claim.

12.3. CREATING AN OBJECT BUCKET CLAIM USING THE COMMAND-LINE INTERFACE

When creating an Object Bucket Claim (OBC) using the command-line interface, you get a configuration map and a Secret that together contain all the information your application needs to use the object storage service.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Use the command-line interface to generate the details of a new bucket and credentials. Run the following command:

```
# odf noobaa obc create <obc-name> -n openshift-storage
```

Replace **<obc-name>** with a unique OBC name, for example, **myappobc**.

Additionally, you can use the **--app-namespace** option to specify the namespace where the OBC configuration map and secret will be created, for example, **myapp-namespace**.

For example:

```
INFO[0001]   Created: ObjectBucketClaim "test21obc"
```

The command-line interface has created the necessary configuration and has informed OpenShift about the new OBC.

2. Run the following command to view the OBC:

```
# oc get obc -n openshift-storage
```

For example:

```
NAME       STORAGE-CLASS                 PHASE   AGE
test21obc  openshift-storage.noobaa.io   Bound   38s
```

3. Run the following command to view the YAML file for the new OBC:

```
# oc get obc test21obc -o yaml -n openshift-storage
```

For example:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  creationTimestamp: "2019-10-24T13:30:07Z"
  finalizers:
  - objectbucket.io/finalizer
  generation: 2
  labels:
    app: noobaa
    bucket-provisioner: openshift-storage.noobaa.io-obc
    noobaa-domain: openshift-storage.noobaa.io
  name: test21obc
  namespace: openshift-storage
  resourceVersion: "40756"
  selfLink: /apis/objectbucket.io/v1alpha1/namespaces/openshift-
storage/objectbucketclaims/test21obc
  uid: 64f04cba-f662-11e9-bc3c-0295250841af
spec:
  ObjectBucketName: obc-openshift-storage-test21obc
  bucketName: test21obc-933348a6-e267-4f82-82f1-e59bf4fe3bb4
```

```
  generateBucketName: test21obc
  storageClassName: openshift-storage.noobaa.io
status:
  phase: Bound
```

4. Inside of your **openshift-storage** namespace, you can find the configuration map and the secret to use this OBC. The CM and the secret have the same name as the OBC.
Run the following command to view the secret:

```
# oc get -n openshift-storage secret test21obc -o yaml
```

For example:

```
apiVersion: v1
data:
  AWS_ACCESS_KEY_ID: c0M0R2xVanF3ODR3bHBkVW94cmY=
  AWS_SECRET_ACCESS_KEY:
Wi9kcFluSWxHRzlWaFlzNk1hc0xma2JXcjM1MVhqa051SlBleXpmOQ==
kind: Secret
metadata:
  creationTimestamp: "2019-10-24T13:30:07Z"
  finalizers:
  - objectbucket.io/finalizer
  labels:
    app: noobaa
    bucket-provisioner: openshift-storage.noobaa.io-obc
    noobaa-domain: openshift-storage.noobaa.io
  name: test21obc
  namespace: openshift-storage
  ownerReferences:
  - apiVersion: objectbucket.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: ObjectBucketClaim
    name: test21obc
    uid: 64f04cba-f662-11e9-bc3c-0295250841af
  resourceVersion: "40751"
  selfLink: /api/v1/namespaces/openshift-storage/secrets/test21obc
  uid: 65117c1c-f662-11e9-9094-0a5305de57bb
type: Opaque
```

The secret gives you the S3 access credentials.

5. Run the following command to view the configuration map:

```
# oc get -n openshift-storage cm test21obc -o yaml
```

For example:

```
apiVersion: v1
data:
  BUCKET_HOST: 10.0.171.35
  BUCKET_NAME: test21obc-933348a6-e267-4f82-82f1-e59bf4fe3bb4
  BUCKET_PORT: "31242"
  BUCKET_REGION: ""
```

```
  BUCKET_SUBREGION: ""
kind: ConfigMap
metadata:
  creationTimestamp: "2019-10-24T13:30:07Z"
  finalizers:
  - objectbucket.io/finalizer
  labels:
    app: noobaa
    bucket-provisioner: openshift-storage.noobaa.io-obc
    noobaa-domain: openshift-storage.noobaa.io
  name: test21obc
  namespace: openshift-storage
  ownerReferences:
  - apiVersion: objectbucket.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: ObjectBucketClaim
    name: test21obc
    uid: 64f04cba-f662-11e9-bc3c-0295250841af
  resourceVersion: "40752"
  selfLink: /api/v1/namespaces/openshift-storage/configmaps/test21obc
  uid: 651c6501-f662-11e9-9094-0a5305de57bb
```

The configuration map contains the S3 endpoint information for your application.

12.4. CREATING AN OBJECT BUCKET CLAIM USING THE OPENSHIFT WEB CONSOLE

You can create an Object Bucket Claim (OBC) using the OpenShift Web Console.

Prerequisites

- Administrative access to the OpenShift Web Console.
- In order for your applications to communicate with the OBC, you need to use the configmap and secret. For more information about this, see Section 12.2, “Dynamic Object Bucket Claim” .

Procedure

1. Log into the OpenShift Web Console.
2. On the left navigation bar, click **Storage → Object Storage → Object Bucket Claims**.
3. Choose the bucket type:
   - Click the **General** card to create a standard object bucket claim.
   - Click the **S3 Vector** card to create a vector bucket claim for storing vector embeddings. For more information, see Creating vector buckets using ObjectBucketClaim .
4. Click **Create Object Bucket Claim**.
   a. Enter a name for your object bucket claim and select the appropriate storage class based on your deployment, internal or external, from the dropdown menu:

      Internal mode

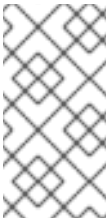
The following storage classes, which were created after deployment, are available for use:

- **ocs-storagecluster-ceph-rgw** uses the Ceph Object Gateway (RGW)
- **openshift-storage.noobaa.io** uses the Multicloud Object Gateway (MCG)

External mode

The following storage classes, which were created after deployment, are available for use:

- **ocs-external-storagecluster-ceph-rgw** uses the RGW
- **openshift-storage.noobaa.io** uses the MCG



NOTE

The RGW OBC storage class is only available with fresh installations of OpenShift Data Foundation version 4.5. It does not apply to clusters upgraded from previous OpenShift Data Foundation releases.

b. Click **Create**.
Once you create the OBC, you are redirected to its detail page.



NOTE

For RGW OBCs, the OBC-specific credentials are required to access the newly created bucket. RGW administrator credentials cannot be used to access this bucket.

12.5. ATTACHING AN OBJECT BUCKET CLAIM TO A DEPLOYMENT

Once created, Object Bucket Claims (OBCs) can be attached to specific deployments.

Prerequisites

- Administrative access to the OpenShift Web Console.

Procedure

1. On the left navigation bar, click **Storage → Object Storage → Object Bucket Claims**.
2. Click the Action menu **(⋮)** next to the OBC you created.
   a. From the drop-down menu, select **Attach to Deployment**
   b. Select the desired deployment from the Deployment Name list, then click **Attach**.

12.6. VIEWING OBJECT BUCKETS USING THE OPENSHIFT WEB CONSOLE

You can view the details of object buckets created for Object Bucket Claims (OBCs) using the OpenShift Web Console.

Prerequisites

- Administrative access to the OpenShift Web Console.

Procedure

1. Log into the OpenShift Web Console.
2. On the left navigation bar, click **Storage → Object Storage → Object Buckets**.
   Optonal: You can also navigate to the details page of a specific OBC, and click the **Resource** link to view the object buckets for that OBC.
3. Select the object bucket of which you want to see the details. Once selected you are navigated to the **Object Bucket Details** page.

12.7. DELETING OBJECT BUCKET CLAIMS

Prerequisites

- Administrative access to the OpenShift Web Console.

Procedure

1. On the left navigation bar, click **Storage → Object Storage → Object Bucket Claims**.
2. Click the Action menu **(⋮)** next to the Object Bucket Claim (OBC) you want to delete.
   a. Select **Delete Object Bucket Claim**.
   b. Click **Delete**.

CHAPTER 13. MANAGING BUCKET QUOTA

Bucket quota allows you to set limits on the number of objects and total storage size for buckets in the Multicloud Object Gateway (MCG). This helps prevent resource starvation and enables better capacity planning and cost control.

This section describes how to configure and manage quota for Object Bucket Claims and buckets.

13.1. BUCKET QUOTA IN MULTICLOUD OBJECT GATEWAY

Multicloud Object Gateway (MCG) supports quota configuration for Object Bucket Claims (OBC) and buckets to prevent resource starvation and optimize storage usage. Quotas help administrators control storage consumption and ensure fair resource allocation across multiple applications.

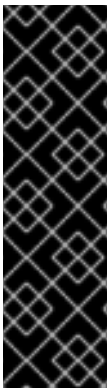
Quota configuration provides the following benefits:

- **Resource Management**: Prevents individual buckets from consuming excessive storage resources.
- **Cost Control**: Limits storage usage to control costs in cloud environments.
- **Multi-tenancy**: Ensures fair resource allocation across multiple applications and teams.
- **Capacity Planning**: Enables better prediction and management of storage requirements.

13.1.1. Quota types

MCG supports two types of quota limits:

- **maxObjects**: Maximum number of objects allowed in a bucket.
- **maxSize**: Maximum total size of all objects in a bucket. You can specify the size using standard units such as **Gi** (gibibytes) or **Mi** (mebibytes).



IMPORTANT

MCG uses a **non-strict quota** mechanism:

- Quota calculations are based on statistics aggregation.
- Statistics calculation cycle runs every 2 minutes.
- Quota enforcement occurs after the next statistics calculation cycle completes.
- Clients may temporarily exceed quota limits until the next cycle runs.

13.1.2. Quota configuration levels

MCG supports quota configuration at three different levels:

BucketClass level

Applies to all OBCs and buckets created from that BucketClass. This is useful for setting default quotas for a category of buckets.

Object Bucket Claim (OBC) level

Applies to a specific OBC and its associated bucket. This provides fine-grained control for individual bucket claims.

Bucket level (Direct)

Applies directly to a specific bucket. This provides the highest level of control.

13.1.3. Quota precedence rules

When quotas are configured at both the OBC and BucketClass levels, the most restrictive (minimum) value is applied per dimension. For example, if a BucketClass sets **maxObjects: 1000** and the OBC sets **maxObjects: 2000**, the resulting bucket will have a quota of 1000 objects.

Quotas set directly on a bucket via the CLI take effect immediately, but might be overwritten on the next OBC reconciliation (for example, OBC edit, operator restart). For OBC-managed buckets, always set quotas through the OBC or BucketClass rather than directly on the bucket.

13.1.4. Supported bucket types

Quota is only supported on regular (placement-based) buckets. Quota configuration is **not supported** on the following bucket types:

- Namespace buckets
- Cache buckets

13.2. SETTING QUOTA ON A BUCKETCLASS

You can set a default quota on a BucketClass so that every new bucket created under it inherits an individual quota limit. This is not a shared quota across buckets. Each bucket gets its own copy of the configured limits.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.
- An existing backingstore or the ability to create one.

Additional resources

- OpenShift Data Foundation CLI binary download

13.2.1. Creating a BucketClass with quota using YAML

You can create a BucketClass with quota limits using a YAML definition.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.
- An existing backingstore or the ability to create one.

Procedure

1. Create a YAML file with the BucketClass definition:

```
apiVersion: noobaa.io/v1alpha1
kind: BucketClass
metadata:
  name: <bucketclass-name>
  namespace: openshift-storage
spec:
  placementPolicy:
    tiers:
    - backingStores:
      - <backingstore-name>
  quota:
    maxSize: "<max-size>"          # for example, "10Gi"
    maxObjects: "<max-objects>"    # for example, "1000"
```

2. Apply the YAML file:

```
$ oc apply -f <bucketclass-file>.yaml
```

Verification

- Check the BucketClass status to verify the quota configuration:

```
$ odf noobaa bucketclass status <bucketclass-name> -n openshift-storage
```

The output should show the quota values in the BucketClass configuration.

13.2.2. Creating a BucketClass with quota using the CLI

You can create a BucketClass with quota limits using the command-line interface.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.
- An existing backingstore or the ability to create one.

Procedure

1. Create a BucketClass with quota limits:

```
$ odf noobaa bucketclass create placement-bucketclass <bucketclass-name> \
    --backingstores <backingstore-name> \
    --max-size <max-size> --max-objects <max-objects> \
    -n openshift-storage
```

For example, to create a BucketClass with both object and size limits:

```
$ odf noobaa bucketclass create placement-bucketclass limited-bucketclass \
    --backingstores noobaa-default-backing-store \
    --max-size 10Gi --max-objects 1000 \
    -n openshift-storage
```

Verification

- Check the BucketClass status to verify the quota configuration:

```
$ odf noobaa bucketclass status <bucketclass-name> -n openshift-storage
```

  Confirm the quota values appear in the BucketClass status output.



NOTE

If both the BucketClass and the OBC define quota, the most restrictive (minimum) value is applied per dimension.

13.3. SETTING QUOTA ON OBJECT BUCKET CLAIM USING YAML

You can set quota limits on an Object Bucket Claim (OBC) to control the maximum number of objects and total storage size for a bucket. Quotas set at the OBC level override any quotas defined at the BucketClass level.

Prerequisites

- Access to the OpenShift Container Platform cluster.
- An existing BucketClass or the default storage class.

Procedure

1. Create a YAML file for the OBC with quota configuration in the **spec.additionalConfig** section:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: my-limited-bucket
  namespace: my-app
spec:
  generateBucketName: my-bucket
  storageClassName: openshift-storage.noobaa.io
  additionalConfig:
    maxObjects: '20000'
    maxSize: 1Gi
```

   Where:

   - **maxObjects**: Maximum number of objects allowed in the bucket. Specify as a string value.
   - **maxSize**: Maximum total size of all objects in the bucket.

2. Apply the YAML file:

```
$ oc apply -f <obc-quota-file>.yaml
```

3. Verify that the OBC was created successfully:

```
$ oc get obc -n <namespace>
```

13.3.1. Updating quota on an existing Object Bucket Claim

You can update the quota limits on an existing OBC using the **oc patch** command.

Prerequisites

- Access to the OpenShift Container Platform cluster.
- An existing OBC with quota configured.

Procedure

1. To update the **maxObjects** quota:

```
$ oc patch objectbucketclaim <obc-name> -n <namespace> \
  --type merge \
  -p '{"spec":{"additionalConfig": {"maxObjects":"50000"}}}'
```

2. To update the **maxSize** quota:

```
$ oc patch objectbucketclaim <obc-name> -n <namespace> \
  --type merge \
  -p '{"spec":{"additionalConfig": {"maxSize":"10Gi"}}}'
```

3. To update both quota parameters:

```
$ oc patch objectbucketclaim <obc-name> -n <namespace> \
  --type merge \
  -p '{"spec":{"additionalConfig": {"maxObjects":"30000", "maxSize":"5Gi"}}}'
```

Verification

1. Verify the quota configuration by viewing the OBC details:

```
$ oc get obc <obc-name> -n <namespace> -o yaml
```

The output should show the quota values in the **spec.additionalConfig** section.

13.4. SETTING QUOTA ON OBJECT BUCKET CLAIM USING THE CLI

You can create an Object Bucket Claim (OBC) with quota limits using the Multicloud Object Gateway (MCG) command-line interface. This allows you to set the maximum number of objects and total storage size for a bucket at the time of creation.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.

Procedure

1. Create an OBC with quota limits using the MCG CLI:

```
$ odf noobaa obc create <obc-name> \
  --max-objects=<number> \
  --max-size=<size> \
  -n openshift-storage
```

Where:

- **--max-objects**: Maximum number of objects allowed in the bucket (optional).
- **--max-size**: Maximum total size of all objects in the bucket.
  For example, to create an OBC with both object and size limits:

```
$ odf noobaa obc create my-limited-bucket \
  --max-objects=20000 \
  --max-size=1Gi \
  -n openshift-storage
```

Example output:

```
INFO[0001]   Created: ObjectBucketClaim "my-limited-bucket"
```

Verification

- View the OBC details to confirm the quota configuration:

```
$ oc get obc <obc-name> -n openshift-storage -o yaml
```

The output should show the quota values in the **spec.additionalConfig** section:

```
spec:
  additionalConfig:
    maxObjects: "20000"
    maxSize: 1Gi
```

13.5. SETTING QUOTA ON BUCKETS

You can set quota limits directly on buckets using the Multicloud Object Gateway (MCG) command-line interface. This provides the highest level of control and overrides any quotas set at the Object Bucket Claim (OBC) level.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.
- An existing bucket or the ability to create a new one.

Additional resources

- OpenShift Data Foundation CLI binary download

13.5.1. Creating a bucket with quota

You can create a new bucket with quota limits using the **noobaa bucket create** command.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.
- An existing bucket or the ability to create a new one.

Procedure

1. Create a bucket with quota limits:

```
$ odf noobaa bucket create <bucket-name> \
  --max-objects=<number> \
  --max-size=<size>
```

Where:

- **--max-objects**: Maximum number of objects allowed in the bucket (optional).
- **--max-size**: Maximum total size of all objects in the bucket (optional).
  For example, to create a bucket with both object and size limits:

```
$ odf noobaa bucket create production-data \
  --max-objects=50000 \
  --max-size=20Gi
```

For example, to create a bucket with size limit only:

```
$ odf noobaa bucket create media-storage \
  --max-size=100Gi
```

For example, to create a bucket with object limit only:

```
$ odf noobaa bucket create log-archive \
  --max-objects=1000000
```

Verification

- Check the bucket status to verify the quota configuration:

```
$ odf noobaa bucket status <bucket-name>
```

13.5.2. Updating quota on an existing bucket

You can update the quota limits on an existing bucket using the **noobaa bucket update** command.

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.
- An existing bucket with or without quota configured.

Procedure

1. Update the quota limits on an existing bucket:

```
$ odf noobaa bucket update <bucket-name> \
  --max-objects=<number> \
  --max-size=<size>
```

   For example, to update both quota parameters:

```
$ odf noobaa bucket update production-data \
  --max-objects=100000 \
  --max-size=50Gi
```

Verification

- Check the bucket status to verify the updated quota configuration:

```
$ odf noobaa bucket status <bucket-name>
```

CHAPTER 14. CACHING POLICY FOR OBJECT BUCKETS

Cache buckets connect a local Multicloud Object Gateway bucket to a large S3-compatible hub target such as AWS or IBM COS, reducing latency for frequently accessed objects.

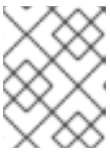
A cache bucket is a namespace bucket with a hub target and a cache target. The hub target is an S3 compatible large object storage bucket. The cache bucket is the local Multicloud Object Gateway (MCG) bucket. You can create a cache bucket that caches an AWS bucket or an IBM COS bucket.

- AWS S3
- IBM COS

14.1. CREATING AN AWS CACHE BUCKET

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Create a NamespaceStore resource. A NamespaceStore represents an underlying storage to be used as a read or write target for the data in the MCG namespace buckets. From the command-line interface, run the following command:

```
odf noobaa namespacestore create aws-s3 <namespacestore> --access-key <AWS ACCESS KEY> --secret-key <AWS SECRET ACCESS KEY> --target-bucket <bucket-name>
```

   a. Replace **<namespacestore>** with the name of the namespacestore.

   b. Replace **<AWS ACCESS KEY>** and **<AWS SECRET ACCESS KEY>** with an AWS access key ID and secret access key you created for this purpose.

   c. Replace **<bucket-name>** with an existing AWS bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
   You can also add storage resources by applying a YAML. First create a secret with credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
type: Opaque
data:
```

```
  AWS_ACCESS_KEY_ID: <AWS ACCESS KEY ID ENCODED IN BASE64>
  AWS_SECRET_ACCESS_KEY: <AWS SECRET ACCESS KEY ENCODED IN
BASE64>
```

You must supply and encode your own AWS access key ID and secret access key using Base64, and use the results in place of **<AWS ACCESS KEY ID ENCODED IN BASE64>** and **<AWS SECRET ACCESS KEY ENCODED IN BASE64>**.

Replace **<namespacestore-secret-name>** with a unique name.

Then apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <namespacestore>
  namespace: openshift-storage
spec:
  awsS3:
    secret:
      name: <namespacestore-secret-name>
      namespace: <namespace-secret>
    targetBucket: <target-bucket>
  type: aws-s3
```

d. Replace **<namespacestore>** with a unique name.

e. Replace **<namespacestore-secret-name>** with the secret created in the previous step.

f. Replace **<namespace-secret>** with the namespace used to create the secret in the previous step.

g. Replace **<target-bucket>** with the AWS S3 bucket you created for the namespacestore.

2. Run the following command to create a bucket class:

```
odf noobaa bucketclass create namespace-bucketclass cache <my-cache-bucket-class> --
backingstores <backing-store> --hub-resource <namespacestore>
```

a. Replace **<my-cache-bucket-class>** with a unique bucket class name.

b. Replace **<backing-store>** with the relevant backing store. You can list one or more backingstores separated by commas in this field.

c. Replace **<namespacestore>** with the namespacestore created in the previous step.

3. Run the following command to create a bucket using an Object Bucket Claim (OBC) resource that uses the bucket class defined in step 2.

```
$ odf noobaa obc create <my-bucket-claim> my-app --bucketclass <custom-bucket-class>
```

a. Replace **<my-bucket-claim>** with a unique name.

b. Replace **<custom-bucket-class>** with the name of the bucket class created in step 2.

14.2. CREATING AN IBM COS CACHE BUCKET

Prerequisites

- Download the OpenShift Data Foundation (ODF) CLI binary from the customer portal and make it executable.



NOTE

Choose the correct product variant according to your architecture. Available platforms are Linux(x86_64), Windows, and Mac OS.

Procedure

1. Create a NamespaceStore resource. A NamespaceStore represents an underlying storage to be used as a read or write target for the data in the MCG namespace buckets. From the command-line interface, run the following command:

```
odf noobaa namespacestore create ibm-cos <namespacestore> --endpoint <IBM COS ENDPOINT> --access-key <IBM ACCESS KEY> --secret-key <IBM SECRET ACCESS KEY> --target-bucket <bucket-name>
```

a. Replace **<namespacestore>** with the name of the NamespaceStore.

b. Replace **<IBM ACCESS KEY>**, **<IBM SECRET ACCESS KEY>**, **<IBM COS ENDPOINT>** with an IBM access key ID, secret access key and the appropriate regional endpoint that corresponds to the location of the existing IBM bucket.

c. Replace **<bucket-name>** with an existing IBM bucket name. This argument tells the MCG which bucket to use as a target bucket for its backing store, and subsequently, data storage and administration.
You can also add storage resources by applying a YAML. First, Create a secret with the credentials:

```
apiVersion: v1
kind: Secret
metadata:
  name: <namespacestore-secret-name>
type: Opaque
data:
  IBM_COS_ACCESS_KEY_ID: <IBM COS ACCESS KEY ID ENCODED IN BASE64>
  IBM_COS_SECRET_ACCESS_KEY: <IBM COS SECRET ACCESS KEY ENCODED IN BASE64>
```

You must supply and encode your own IBM COS access key ID and secret access key using Base64, and use the results in place of **<IBM COS ACCESS KEY ID ENCODED IN BASE64>** and **<IBM COS SECRET ACCESS KEY ENCODED IN BASE64>**.

Replace **<namespacestore-secret-name>** with a unique name.

Then apply the following YAML:

```
apiVersion: noobaa.io/v1alpha1
kind: NamespaceStore
metadata:
  finalizers:
  - noobaa.io/finalizer
  labels:
    app: noobaa
  name: <namespacestore>
  namespace: openshift-storage
spec:
  s3Compatible:
    endpoint: <IBM COS ENDPOINT>
    secret:
      name: <backingstore-secret-name>
      namespace: <namespace-secret>
    signatureVersion: v2
    targetBucket: <target-bucket>
  type: ibm-cos
```

d. Replace **<namespacestore>** with a unique name.

e. Replace **<IBM COS ENDPOINT>** with the appropriate IBM COS endpoint.

f. Replace **<backingstore-secret-name>** with the secret created in the previous step.

g. Replace **<namespace-secret>** with the namespace used to create the secret in the previous step.

h. Replace **<target-bucket>** with the AWS S3 bucket you created for the namespacestore.

2. Run the following command to create a bucket class:

```
odf noobaa bucketclass create namespace-bucketclass cache <my-bucket-class> --
backingstores <backing-store> --hubResource <namespacestore>
```

a. Replace **<my-bucket-class>** with a unique bucket class name.

b. Replace **<backing-store>** with the relevant backing store. You can list one or more backingstores separated by commas in this field.

c. Replace **<namespacestore>** with the namespacestore created in the previous step.

3. Run the following command to create a bucket using an Object Bucket Claim resource that uses the bucket class defined in step 2.

```
$ odf noobaa obc create <my-bucket-claim> my-app --bucketclass <custom-bucket-class>
```

a. Replace **<my-bucket-claim>** with a unique name.

b. Replace **<custom-bucket-class>** with the name of the bucket class created in step 2.

CHAPTER 15. LIFECYCLE BUCKET CONFIGURATION IN MULTICLOUD OBJECT GATEWAY

Multicloud Object Gateway (MCG) lifecycle provides a way to reduce storage costs due to accumulated data objects.

Deletion of expired objects is a simplified way that enables handling of unused data. Data expiration is a part of Amazon Web Services (AWS) lifecycle management and sets an expiration date for automatic deletion. The minimal time resolution of the lifecycle expiration is one day.

AWS S3 API is used to configure the lifecycle bucket in MCG.

The limitation with the expiration rule API for MCG in comparison with AWS is that there is no support for transition rules (both regular and noncurrent).

You can configure the bucket lifecycle using the MCG object browser.

Additional resources

- Expiring objects
- Support of Multicloud Object Gateway data bucket APIs
- Creating new lifecycle rules using MCG object browser

CHAPTER 16. SCALING MULTICLOUD OBJECT GATEWAY PERFORMANCE

Increase Multicloud Object Gateway throughput for S3-intensive workloads by scaling the number of NooBaa endpoints to match application demand.

The Multicloud Object Gateway (MCG) performance may vary from one environment to another. In some cases, specific applications require faster performance which can be easily addressed by scaling S3 endpoints.

The MCG resource pool is a group of NooBaa daemon containers that provide two types of services enabled by default:

- Storage service
- S3 endpoint service

S3 endpoint service The S3 endpoint is a service that every Multicloud Object Gateway (MCG) provides by default that handles the heavy lifting data digestion in the MCG. The endpoint service handles the inline data chunking, deduplication, compression, and encryption, and it accepts data placement instructions from the MCG.

16.1. AUTOMATIC SCALING OF MULTICLOUD OBJECT GATEWAY ENDPOINTS

The number of MultiCloud Object Gateway (MCG) endpoints scale automatically when the load on the MCG S3 service increases or decreases. OpenShift Data Foundation clusters are deployed with one active MCG endpoint. Each MCG endpoint pod is configured by default with 1 CPU and 2Gi memory request, with limits matching the request. When the CPU load on the endpoint crosses over an 80% usage threshold for a consistent period of time, a second endpoint is deployed lowering the load on the first endpoint. When the average CPU load on both endpoints falls below the 80% threshold for a consistent period of time, one of the endpoints is deleted. This feature improves performance and serviceability of the MCG.

You can scale the Horizontal Pod Autoscaler (HPA) for **noobaa-endpoint** using the following **oc patch** command, for example:

```
# oc patch -n openshift-storage storagecluster ocs-storagecluster \
    --type merge \
    --patch '{"spec": {"multiCloudGateway": {"endpoints": {"minCount": 3,"maxCount": 10}}}}'
```

The example above sets the **minCount** to **3** and the **maxCount to `10**.



WARNING

Changing **maxCount** in the **StorageCluster** CR triggers a PostgreSQL restart, which causes a temporary S3 outage of approximately 30–60 seconds. This occurs because the noobaa-operator dynamically calculates the PostgreSQL **max_connections** value based on the configured maximum endpoint count, and PostgreSQL requires a restart to apply the new value.

- Changing only **minCount** does not trigger a PostgreSQL restart.
- HPA-driven autoscaling within the already-configured **minCount** and **maxCount** range does not trigger a PostgreSQL restart.

If S3 availability is critical to your workload, perform **maxCount** changes during a scheduled maintenance window.

16.2. AUTOSCALING RGW IN OPENSHIFT DATA FOUNDATION USING HPA AND KEDA

This section describes how to enable autoscaling for RGW (RADOS Gateway) in OpenShift Data Foundation by integrating with OpenShift Horizontal Pod Autoscaler (HPA) - KEDA (Kubernetes Event-Driven Autoscaling).

Prerequisites

- All resources described in this procedure (Roles, RoleBindings, TriggerAuthentication, ScaledObject, and so on) must be created in the **openshift-storage** namespace.
- Custom Metrics Autoscaler Operator version must be 2.6 or later.

Procedure

1. Install the Custom Metrics Autoscaler Operator.
   a. Log in to the OpenShift Web Console.
   b. Click **Ecosystem → Software Catalog**.
   c. Scroll or type Custom Metrics Autoscaler into the Filter by keyword box to find the Custom Metrics Autoscaler Operator.
   d. Click **Install**.
2. Create a Custom Metrics Autoscaler controller and verify whether the following pods are running in the **openshift-keda** namespace:

```
oc get pods -n openshift-keda
NAME                                      READY   STATUS    RESTARTS   AGE
custom-metrics-autoscaler-operator-6cb698bb4d-nnpmj   1/1     Running   0          103m
keda-admission-c6d879546-twprp                        1/1     Running   0          103m
keda-metrics-apiserver-68788fbd9b-h7l5n               1/1     Running   0          103m
```

```
keda-operator-664dc57859-mb49f                1/1     Running   0          20m
```

3. Configure Custom Metrics Autoscaler with the Thanos Query Service.
   Prometheus scaler that is supported by Custom Metrics Autoscaler collects metrics through the Thanos Query service provided by OpenShift Monitoring. Because Thanos Query requires authentication, a ServiceAccount token must be used for bearer authentication.

   a. Create a Service Account.

   ```
   oc create serviceaccount thanos -n openshift-storage
   ```

   b. Create a secret to generate the token.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: thanos-token-secret
     namespace: openshift-storage
     annotations:
       kubernetes.io/service-account.name: "thanos"
   type: kubernetes.io/service-account-token
   ```

   ```
   oc apply -f secret.yaml
   ```

   c. Add the cluster-monitoring-view cluster role to the created Service Account in the openshift-storage namespace.

   ```
   oc adm policy add-cluster-role-to-user cluster-monitoring-view -z thanos
   ```

   d. Create a triggerAuthentication CRD for the bearer authentication using the secret name.

   ```
   apiVersion: keda.sh/v1alpha1
   kind: TriggerAuthentication
   metadata:
     name: keda-trigger-auth-prometheus
     namespace: openshift-storage
   spec:
     secretTargetRef:
     - parameter: bearerToken
       name: thanos-token-secret
       key: token
     - parameter: ca
       name: thanos-token-secret
       key: ca.crt
   ```

4. Enable RGW autoscaling.

   ```
   oc annotate storagecluster ocs-storagecluster ocs.openshift.io/enable-rgw-autoscale="true" -n openshift-storage
   ```

   This behavior takes precedence over manual scaling. You can revert this change by performing the following command:

```
oc annotate storagecluster ocs-storagecluster ocs.openshift.io/enable-rgw-autoscale- -n
openshift-storage
```

5. Create ScaledObject for autoscaling RGW.
   Autoscaling using the HPA feature of OpenShift triggers scaling based on custom metrics. The prometheus metrics related to RGW exported by the Ceph manager is used. The following example uses **ceph_rgw_put_obj_ops** metrics and scale up to five pods.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: rgw-scale-namespace
  namespace: openshift-storage
spec:
  scaleTargetRef:
    apiVersion: ceph.rook.io/v1
    kind: CephObjectStore
    name: ocs-storagecluster-cephobjectstore
    namespace: openshift-storage
  minReplicaCount: 1
  maxReplicaCount: 5
  triggers:
    - type: prometheus
      metadata:
        serverAddress: https://thanos-querier.openshift-monitoring.svc.cluster.local:9091
        metricName: ceph_rgw_put_obj_ops
        query: |
          sum(rate(ceph_rgw_op_put_obj_ops[2m]))
        threshold: "1"
        authModes: "bearer"
        namespace: openshift-storage
      authenticationRef:
        name: keda-trigger-auth-prometheus
```

16.3. INCREASING CPU AND MEMORY FOR PV POOL RESOURCES

MCG default configuration supports low resource consumption. However, when you need to increase CPU and memory to accommodate specific workloads and to increase MCG performance for the workloads, you can configure the required values for CPU and memory in the OpenShift Web Console.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → Object Storage → Backing Store**.
2. Select the relevant backing store and click on YAML.
3. Scroll down until you find **spec:** and update **pvPool** with CPU and memory. Add a new property of limits and then add cpu and memory.
   Example reference:

```
spec:
  pvPool:
    resources:
      limits:
```

```
      cpu: 1000m
      memory: 4000Mi
    requests:
      cpu: 800m
      memory: 800Mi
      storage: 50Gi
```

4. Click **Save**.

Verification

- To verfiy, you can check the resource values of the PV pool pods.

CHAPTER 17. ACCESSING THE RADOS OBJECT GATEWAY S3 ENDPOINT

You can access the RADOS Object Gateway (RGW) endpoint directly. The RGW route is created by default and is named **rook-ceph-rgw-ocs-storagecluster-cephobjectstore**.

For information about RGW S3 supported and unsupported verbs, see S3 supported and unsupported verbs in the *Red Hat Ceph Storage Developer Guide* .

CHAPTER 18. USING TLS CERTIFICATES FOR APPLICATIONS ACCESSING RGW

Most of the S3 applications require TLS certificate in the forms such as an option included in the Deployment configuration file, passed as a file in the request, or stored in **/etc/pki** paths.

TLS certificates for RADOS Object Gateway (RGW) are stored as Kubernetes Secret and you need to fetch the details from the secret.

Prerequisites

A running OpenShift Data Foundation cluster.

Procedure

- **For internal RGW server**
  - Get the TLS certificate and key from the Kubernetes Secret:

```
$ oc get secrets/<secret_name> -o jsonpath='{.data..tls\.crt}' | base64 -d

$ oc get secrets/<secret_name> -o jsonpath='{.data..tls\.key}' | base64 -d
```

    <secret_name>

    The default Kubernetes Secret name is **<objectstore_name>-cos-ceph-rgw-tls-cert**. Specify the name of the object store.

- **For external RGW server**
  - Get the the TLS certificate from the Kubernetes Secret:

```
$ oc get secrets/<secret_name> -o jsonpath='{.data.cert}' | base64 -d
```

    <secret_name>

    The default Kubernetes Secret name is **ceph-rgw-tls-cert** and it is an opaque type of secret. The key value for storing the TLS certificates is **cert**.

18.1. ACCESSING EXTERNAL RGW SERVER IN OPENSHIFT DATA FOUNDATION

Procedure

1. To access using Object Bucket Claims, fetch the S3 credentials from the secret generated by the OBC creation:

```
# oc get secret <object bucket claim name> -o
jsonpath='{.data.AWS_SECRET_ACCESS_KEY}' | base64 --decode
# oc get secret <object bucket claim name> -o jsonpath='{.data.AWS_ACCESS_KEY_ID}' |
base64 --decode
```

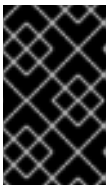
   Fetch the endpoint details from the **configmap** of the OBC:

```
# oc get cm <object bucket claim name> -o jsonpath='{.data.BUCKET_HOST}'
```

```
# oc get cm <object bucket claim name> -o jsonpath='{.data.BUCKET_PORT}'
# oc get cm <object bucket claim name> -o jsonpath='{.data.BUCKET_NAME}'
```

2. To access using the **Ceph Object Store User** CR, fetch the S3 credentials and endpoint details from the generated secret:

```
# oc get secret rook-ceph-object-user-<object-store-cr-name>-<object-user-cr-name> -o
jsonpath='{.data.AccessKey}' | base64 --decode
# oc get secret rook-ceph-object-user-<object-store-cr-name>-<object-user-cr-name> -o
jsonpath='{.data.SecretKey}' | base64 --decode
# oc get secret rook-ceph-object-user-<object-store-cr-name>-<object-user-cr-name> -o
jsonpath='{.data.Endpoint}' | base64 --decode
```



IMPORTANT

For both the access mechanisms, you can either request for new certificates from the administrator or reuse the certificates from the Kubernetes Secret, **ceph-rgw-tls-cert**.

CHAPTER 19. USING THE MULTICLOUD OBJECT GATEWAY'S SECURITY TOKEN SERVICE TO ASSUME THE ROLE OF ANOTHER USER

Multicloud Object Gateway (MCG) provides support to a security token service (STS) similar to the one provided by Amazon Web Services.



NOTE

This procedure describes the Multicloud Object Gateway (MCG) Security Token Service role-assumption workflow.

To allow other users to assume the role of a certain user, you need to assign a role configuration to the user. You can manage the configuration of roles using the MCG CLI tool.

Procedure

1. Configure role assignment. The following example shows role configuration that allows two MCG users (**assumer@mcg.test** and **assumer2@mcg.test**) to assume a certain user's role:

```
'{"role_name": "AllowTwoAssumers", "assume_role_policy": {"version": "2012-10-17",
"statement": [ {"action": ["sts:AssumeRole"], "effect": "allow", "principal":
["assumer@mcg.test", "assumer2@mcg.test"]}]}}'
```

2. Assign the role configuration by using the MCG CLI tool.

```
odf noobaa sts assign-role --email <assumed user's username> --role_config '{"role_name":
"AllowTwoAssumers", "assume_role_policy": {"version": "2012-10-17", "statement": [
{"action": ["sts:AssumeRole"], "effect": "allow", "principal": ["assumer@mcg.test",
"assumer2@mcg.test"]}]}}'
```

3. Collect the following information before proceeding to assume the role as it is needed for the subsequent steps:

   - The access key ID and secret access key of the assumer (the user who assumes the role)
   - The MCG STS endpoint, which can be retrieved by using the command:

     ```
     $ oc -n openshift-storage get route
     ```

   - The access key ID of the assumed user.
   - The value of the **role_name** value in your role configuration.
   - A name of your choice for the role session

4. After the configuration role is ready, assign it to the appropriate user (fill with the data described in the previous step):

```
AWS_ACCESS_KEY_ID=<aws-access-key-id> AWS_SECRET_ACCESS_KEY=<aws-
secret-access-key1> aws --endpoint-url <mcg-sts-endpoint> sts assume-role --role-arn
arn:aws:sts::<assumed-user-access-key-id>:role/<role-name> --role-session-name <role-
session-name>
```



NOTE

Adding **--no-verify-ssl** might be necessary depending on your cluster's configuration.

The resulting output contains the access key ID, secret access key, and session token that can be used for executing actions while assuming the other user's role.

You can use the credentials generated after the assume role steps as shown in the following example:

```
AWS_ACCESS_KEY_ID=<aws-access-key-id> AWS_SECRET_ACCESS_KEY=<aws-secret-access-key1> AWS_SESSION_TOKEN=<session token> aws --endpoint-url <mcg-s3-endpoint> s3 ls
```